ETHERNET
SENSOR
BUS



User Guide

ESB Protocol

ESB
Nov 11, 2026 - Rev. 1.0.0

Table of contents

1 Introduction **1**
- 1.1 Operating principle . . . 2
  - 1.1.1 Endpoints . . . 3
- 1.2 Messages . . . 3
  - 1.2.1 Directives . . . 4
  - 1.2.2 Reports . . . 5
  - 1.2.3 Unsolicited reports . . . 5
  - 1.2.4 Malformed messages . . . 5
  - 1.2.5 Compatibility . . . 5
  - 1.2.6 Example . . . 6
- 1.3 Network . . . 6
  - 1.3.1 Message size . . . 7
  - 1.3.2 Raw Ethernet . . . 7
  - 1.3.3 UDP/IP . . . 8
  - 1.3.4 Clients . . . 9
  - 1.3.5 Support protocols . . . 9
- 1.4 Scheduler . . . 9
  - 1.4.1 Synchronous and asynchronous actions . . . 9
  - 1.4.2 Life of an action . . . 10
  - 1.4.3 Triggers . . . 10
  - 1.4.4 Repetition . . . 11
  - 1.4.5 Time base . . . 12
  - 1.4.6 Late actions . . . 12
  - 1.4.7 Results and replies . . . 12
  - 1.4.8 Identifying and canceling actions . . . 12
  - 1.4.9 Example . . . 13
- 1.5 Higher-level behavior . . . 13
  - 1.5.1 Persistent storage . . . 14
  - 1.5.2 Fail-safe operation . . . 15
  - 1.5.3 Example . . . 15
- 1.6 Usage hints . . . 15
  - 1.6.1 Discover the server first . . . 15
  - 1.6.2 Make directives easy to match and to repeat . . . 16
  - 1.6.3 Timing and scheduling . . . 16
  - 1.6.4 Safety and storage . . . 16
  - 1.6.5 Several servers and the network . . . 16
  - 1.6.6 Handling messages . . . 17
- 1.7 Protobuf encoding . . . 17
  - 1.7.1 Fields and records . . . 17
  - 1.7.2 Values . . . 17
  - 1.7.3 Field presence and default values . . . 18
  - 1.7.4 Worked example . . . 18
- 1.8 Code generators . . . 18
  - 1.8.1 The Protocol Buffers compiler . . . 18
  - 1.8.2 Generators from Google . . . 19
  - 1.8.3 Generators for small systems . . . 19
  - 1.8.4 Further reading . . . 19

2 RCP Messages **20**
- 2.1 Example . . . 20
- 2.2 Protocol . . . 20
  - 2.2.1 Messages . . . 20

2.2.2 Enumerations 29

3 Common Definitions **32**
3.1 Endpoints 32
3.2 Timestamps and versions 32
3.3 Protocol 32
3.3.1 Messages 32
3.3.2 Enumerations 33

4 Server **34**
4.1 RCP Server 34
4.1.1 Configuration, commands, and queries 34
4.1.2 Example 34
4.1.3 Protocol 35
4.2 Identification and Capabilities 38
4.2.1 Protocol 39
4.3 Network Configuration 40
4.3.1 Subnet 40
4.3.2 VLANs 40
4.3.3 Clients 41
4.3.4 PHY registers 41
4.3.5 Example 41
4.3.6 Protocol 41
4.4 Pins 45
4.4.1 Pin descriptors and functions 46
4.4.2 Pin configuration 46
4.4.3 Example 46
4.4.4 Protocol 47
4.5 PLCA 50
4.5.1 PLCA cycle 50
4.5.2 Protocol 50
4.6 PTP 51
4.6.1 Synchronization 51
4.6.2 Example 52
4.6.3 Protocol 52
4.7 Topology Discovery 53
4.7.1 Distance 53
4.7.2 Example 54
4.7.3 Protocol 54
4.8 Core Operations 56
4.8.1 Server identification 56
4.8.2 Network watchdog 56
4.8.3 Aggregation of notifications 57
4.8.4 Protocol 57
4.9 Vendor-Specific Features 60
4.9.1 Protocol 60

5 Endpoints **62**
5.1 GPIO Driver 62
5.1.1 Pins and masks 62
5.1.2 Pin change events 63
5.1.3 Example 63
5.1.4 Protocol 63
5.2 SPI Driver 65
5.2.1 Clock polarity and phase 65
5.2.2 Transactions 65
5.2.3 Example 66
5.2.4 Protocol 66
5.3 I2C Driver 68

5.3.1 Bit timing . . . 68
5.3.2 Transactions . . . 68
5.3.3 Example . . . 69
5.3.4 Protocol . . . 69
5.4 PWM Driver . . . 70
5.4.1 Protocol . . . 71
5.5 UART Driver . . . 72
5.5.1 Transmission timing . . . 72
5.5.2 Reception . . . 74
5.5.3 Reception filtering . . . 74
5.5.4 Flow control . . . 75
5.5.5 Example . . . 76
5.5.6 Protocol . . . 76
5.6 PCM Audio Driver . . . 78
5.6.1 Frame formats . . . 78
5.6.2 Streaming . . . 79
5.6.3 Example . . . 79
5.6.4 Protocol . . . 79
5.7 PDM Driver . . . 81
5.7.1 Principle . . . 81
5.7.2 Clock and sampling edge . . . 81
5.7.3 Streaming . . . 82
5.7.4 Example . . . 82
5.7.5 Protocol . . . 82
5.8 ADC Driver . . . 83
5.8.1 Processing chain . . . 83
5.8.2 Streaming and thresholds . . . 84
5.8.3 External ADC . . . 84
5.8.4 Example . . . 84
5.8.5 Protocol . . . 85
5.9 SENT Driver . . . 89
5.9.1 Message format . . . 89
5.9.2 Streaming . . . 89
5.9.3 Example . . . 89
5.9.4 Protocol . . . 89
5.10 LIN Driver . . . 90
5.10.1 Frame . . . 91
5.10.2 Commands . . . 91
5.10.3 Example . . . 91
5.10.4 Protocol . . . 91

Chapter 1
Introduction

Ethernet Sensor Bus (ESB)

The Ethernet Sensor Bus (ESB) is a remote control protocol (RCP). A client sends directives over Ethernet to RCP servers. The servers execute them on hardware endpoints, such as SPI, I2C, GPIO, or UART controllers, and report the results back. ESB is an open protocol. It was first released in 2026.

This documentation specifies the messages of ESB as a Protocol Buffers (proto3) schema, and the behavior of clients and servers. The schema files, from which this documentation is generated, are in the GitHub repository ethernet-sensor-bus.

Motivation

A distributed system usually puts a microcontroller with firmware next to every sensor, actuator, or bus interface. The application logic is then split across many firmware images, and each of them has its own development, test, and update cycle.

With ESB, the application runs on a central processor and reaches the hardware of the remote RCP servers over Ethernet. An RCP server runs no application software. It interprets the protocol and drives its endpoints. A change of the application affects only the central processor.

Scope

This documentation specifies the messages that a client and an RCP server exchange, how they are carried over Ethernet, and how an RCP server behaves when it receives them.

The datasheet of a server defines what this documentation leaves open, such as the endpoints and optional features of the server and its limits, e.g., the maximum clock speed of the SPI controller or the depth of the UART transmit buffer.

Conventions

Normative text states a requirement in the present tense, e.g., “The server accepts the broadcast address.” The verbal forms *should*, *may*, and *can* express a recommendation, a permission, and a possibility or capability, respectively. *Should not* expresses the opposite of *should*.

The pages *Operating principle*, *Usage hints*, *Protobuf encoding*, and *Code generators* are informative and do not contain requirements. Notes, warnings, and examples are informative as well. All other text is normative.

Hexadecimal numbers have the prefix 0x and binary numbers the prefix 0b. All other numbers are decimal. Byte sequences are written as pairs of hexadecimal digits without prefix, e.g., `0D 01`. A name in code font is a name from the schema or a literal, such as a keyword or a file name. A name from the schema is qualified by its package where needed, e.g., *`rcp.net.fldConfig.vlanIds`*.

Terms and definitions

- *Client*: a system that controls RCP servers. It sends directives and receives reports, e.g., as software on a central processor.
- *RCP server*, short *server*: a system that clients control. It contains the endpoints, executes directives on them, and sends reports.
- *Endpoint*: a hardware block of an RCP server that exchanges low-level data with the outside world, e.g., a SPI controller, a GPIO controller, or a UART, see *Operating principle*.
- *Handle*: the number that addresses an endpoint, see *Operating principle*.
- *Directive*: the top-level message from a client to an RCP server. It holds a list of operations.

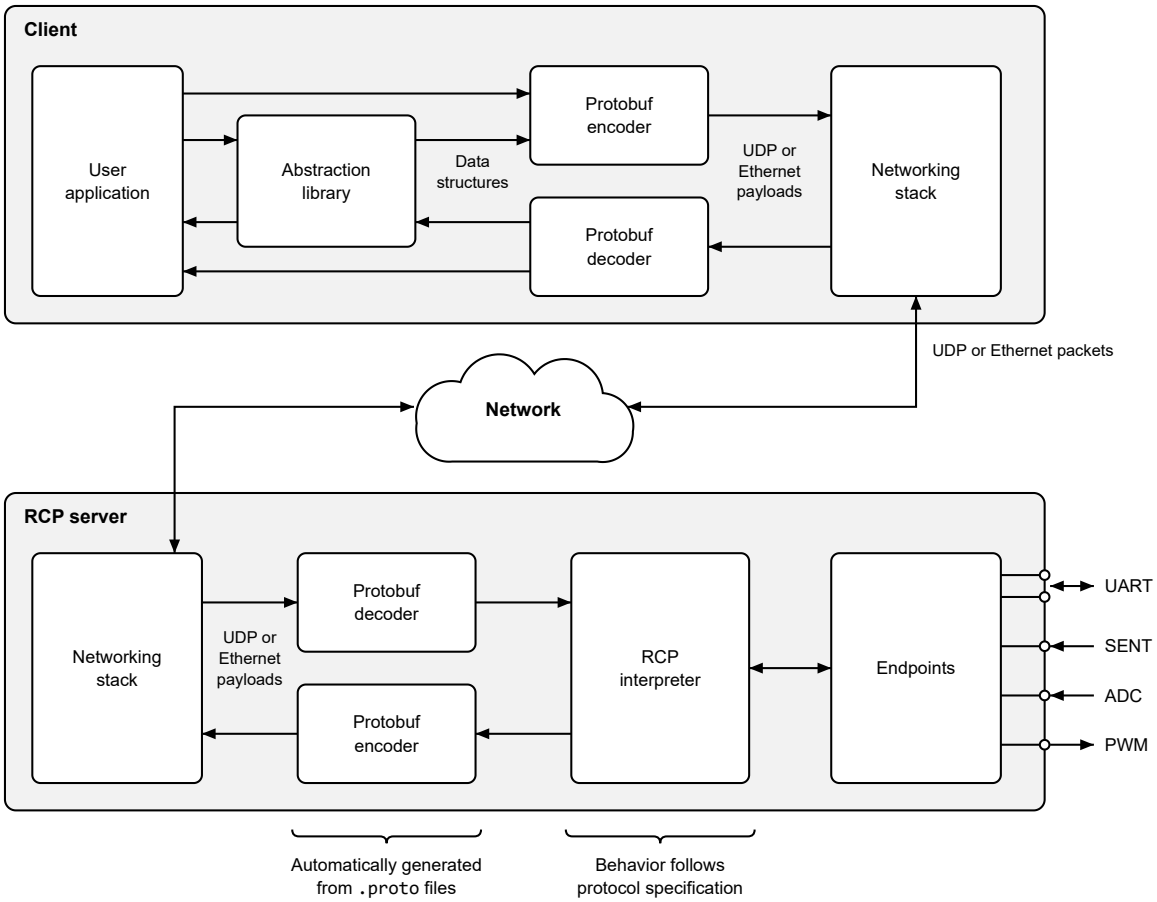
- *Operation*: an entry in a directive, see *Messages*.
- *Report*: the top-level message from an RCP server to a client. It holds results and notifications.
- *Notification*: data that an endpoint sends on its own, see *Messages*.
- *RCP message*: a directive or a report.

Normative references

- IEEE 802.3-2022, Ethernet
- IEEE 802.1Q-2022, Bridges and bridged networks
- IEEE 802.1AS-2020, Timing and synchronization for time-sensitive applications
- RFC 768, User Datagram Protocol
- RFC 791, Internet Protocol
- RFC 826, Address Resolution Protocol
- RFC 951, Bootstrap Protocol
- RFC 2131, Dynamic Host Configuration Protocol
- SAE J2716, SENT (Single Edge Nibble Transmission) for Automotive Applications
- Protocol Buffers Language Guide (proto3)

1.1 Operating principle

The client builds a directive, usually through an abstraction library, encodes it with code that a Protocol Buffers (protobuf) compiler generated from the `.proto` files, and sends it through its networking stack. The RCP server decodes the directive, executes its operations on the endpoints, and answers with a report. It also sends reports that the client has not requested, e.g., when a scheduled action has run. The figure shows both sides. The RCP interpreter of the server contains no application logic.



Messages describes the messages, *Network* their transport, and *Scheduler* how the server runs actions. The encoding is explained in *Protobuf encoding* and the code generators in *Code generators*.

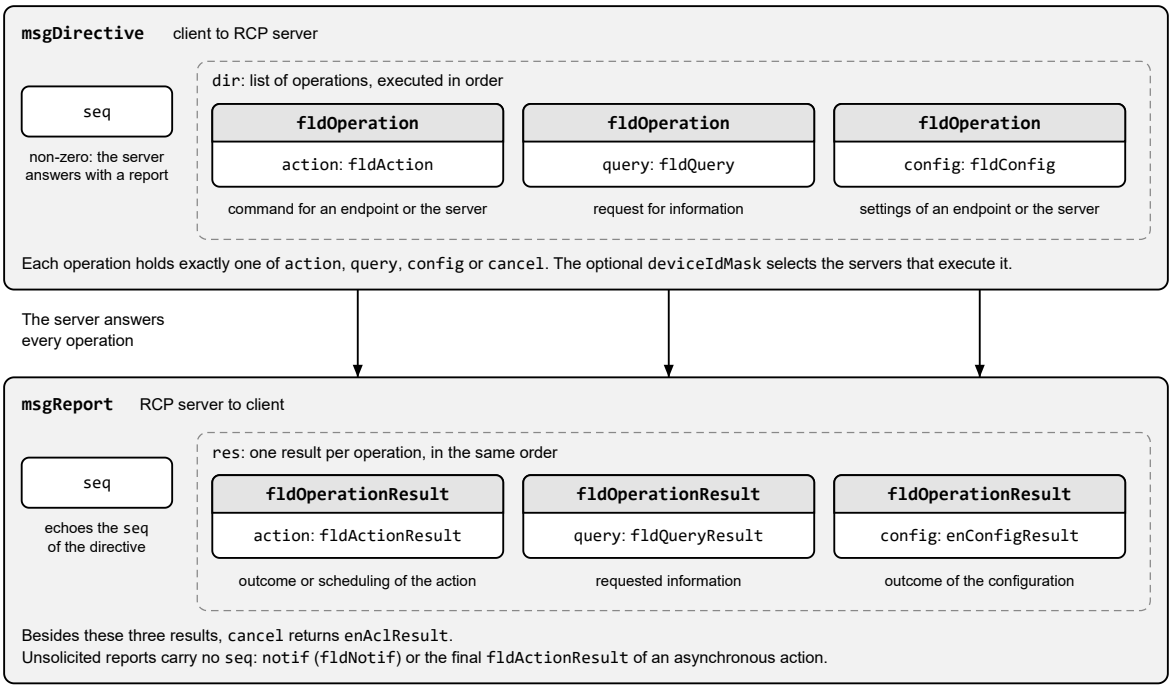
1.1.1 Endpoints

An endpoint is a hardware block of the server that exchanges low-level data with the outside world, e.g., a SPI controller, a GPIO controller, or a UART. The figure shows a UART, a SENT receiver, an ADC, and a PWM generator as examples. Each type of *endpoint* has a page of its own.

Each endpoint has a unique, non-zero handle, which a client uses to address it. The handle 0 addresses the server itself. The optional query `rcp.dev.qryInterfaces` lists the endpoints with their handles and types, see *Identification and Capabilities*.

1.2 Messages

The client and the RCP server exchange two types of top-level messages, the directive and the report. They are defined in *RCP Messages*. The page *Network* specifies how they are transmitted. The figure shows the structure of both messages.



1.2.1 Directives

A directive (*msgDirective*) holds a sequence number (*msgDirective.seq*) and a list of operations. The server handles the operations in the order in which they appear. It checks and schedules an asynchronous action at its position, and the action runs when its trigger fires, see *Scheduler*.

A directive is not a transaction. An operation error neither rolls back earlier operations nor prevents later operations from being handled. This differs from a decoding error, which rejects the whole directive, see Malformed messages.

The maximum size of a message, see *Network*, and the size of the operations limit the number of operations in a directive. A server may limit this number further.

An operation (*fldOperation*) is one of the following:

- An action commands an endpoint or the RCP server itself, e.g., a SPI transfer or the toggling of GPIO pins. It can run immediately, at a later time, repeatedly, or when an event occurs, see *Scheduler*.
- A query requests information, e.g., the identity, the configuration, or the status. The enumeration *rcp.dev.enQuery* marks the mandatory and the optional queries.
- A configuration changes the settings of the RCP server or of an endpoint.
- A cancel operation cancels scheduled actions that have not completed.

An action, a configuration, or a query contains the handle of the endpoint that it addresses, followed by the message for that endpoint. The message matches the type of the endpoint at the handle, see *Operating principle*.

If *msgDirective.seq* is non-zero, the server returns it in the report to the directive. If it is 0, the default, the server sends no report to the directive. This does not suppress unsolicited reports. A malformed directive is the exception, see Malformed messages. The sequence number correlates a reply, not an execution: receiving the same directive again can execute its operations again.

If a directive is broadcast or multicast to many RCP servers, the optional DID mask of an operation (*fldOperation.deviceIdMask*) selects the servers that execute it. A server executes the operation only if the bitwise AND of its identification number (DID) and the mask is not zero.

Otherwise, it ignores the operation and adds no result for it to the report. The DID is set with *rcp.core.fldServerConfig.did*. See *Core Operations* for an example.

1.2.2 Reports

A report (*msgReport*) holds the sequence number of the directive that it answers and one result for each of its operations, in the same order, except for an operation that the server ignores because of the DID mask. Each kind of result has its own enumeration of result codes. Some result codes are warnings. A warning means that the operation succeeded but deviated from the request. For example, *cfgParamAdjusted* tells that the server accepted a configuration but adjusted a value, such as a clock speed that its hardware cannot reproduce exactly. The client can read the actual setting back with a query. A warning counts as success, e.g., for the reply mode of an action and for the trigger type of a dependent action, see *Scheduler*.

1.2.3 Unsolicited reports

The server also sends reports that the client has not requested. These reports carry no *msgReport.seq*. A report can hold results of two kinds:

- The result of a scheduled action. If an action does not run immediately, the report to its directive only acknowledges that the action is pending. The server reports the result of each execution, unless *fldAction.replyMode* limits the reports.
- A notification. Some endpoints produce data on their own, e.g., when a GPIO pin changes, a UART receives bytes, or an ADC crosses a threshold.

Action execution results are sent to the client that scheduled the action. Each endpoint has at most one client that owns its notifications, and sends them only to that client. The query result *fldEndpointNotifInfo* identifies the owner.

To save bandwidth, the server may collect several notifications in one report, see *Core Operations*. Each endpoint numbers its notifications (*fldNotif.seq*) and each action its results (*fldActionResult.seq*), so a gap shows that a report is missing.

1.2.4 Malformed messages

A server ignores a frame that is not a valid RCP frame, see *Network*. It sends no report for it.

If the frame is valid but the server cannot process the encoded message, it rejects the whole directive. It does not execute any operation of the directive, not even the operations in front of the error. The server answers with a report that holds one notification. The code of the notification (*fldNotif.code*) tells the reason:

- *ntfProtocolError*: the message is not valid Protocol Buffers data, e.g., it ends inside a field or contains an invalid wire type.
- *ntfProtocolTooLarge*: the message exceeds a limit of the server, e.g., a list has more items than the server can store. The limits are implementation-specific.
- *ntfProtocolUnsupported*: the message contains a field that the server does not know, or it sets more than one member of a `oneof`.
- *ntfOutOfMemory*: the server ran out of memory while it decoded the message.

The report carries the sequence number of the directive only if the server read it before it detected the error. Otherwise, the sequence number is 0. The server sends the report also if the sequence number of the directive is 0.

1.2.5 Compatibility

New versions of the schema add fields, messages, and enumeration values. They never renumber an existing field or enumeration value and never reuse a number for another purpose, so a

message of an older version is still valid in a newer version. A client reads the version of the protocol that a server implements from *rcp.info.fldIdentity.rcpVersion*.

A client and a server can implement different versions, so a receiver can get content that it does not know:

- A server rejects a directive that contains a field that it does not know, see Malformed messages. It does not skip the field and execute the rest of the directive.
- A client skips a field of a report that it does not know. Protocol Buffers libraries do this by default.
- A server answers an operation that it does not implement with the result code *actNotSupported*, *cfgNotSupported*, or *qryNotSupported*.

1.2.6 Example

A directive with two operations: a query for the status of the RCP server and an action that reads the state of the pins 0 and 1 of the GPIO endpoint with the handle 1. Both run as soon as possible:

```
seq: 7
dir { query { rcp: qryServerStatus } }
dir { action { hndl: 1  gpio { get: 0x03 } } }
```

The report has the same sequence number and one result for each operation, in the same order. In this example, the power-on flag of the RCP server is still set, pin 0 is high, and pin 1 is low:

```
seq: 7
res { query { rcp { server { pwrOn: true } } } }
res { action { gpio: 1 } }
```

The examples use the text format of protobuf, see *Protobuf encoding*.

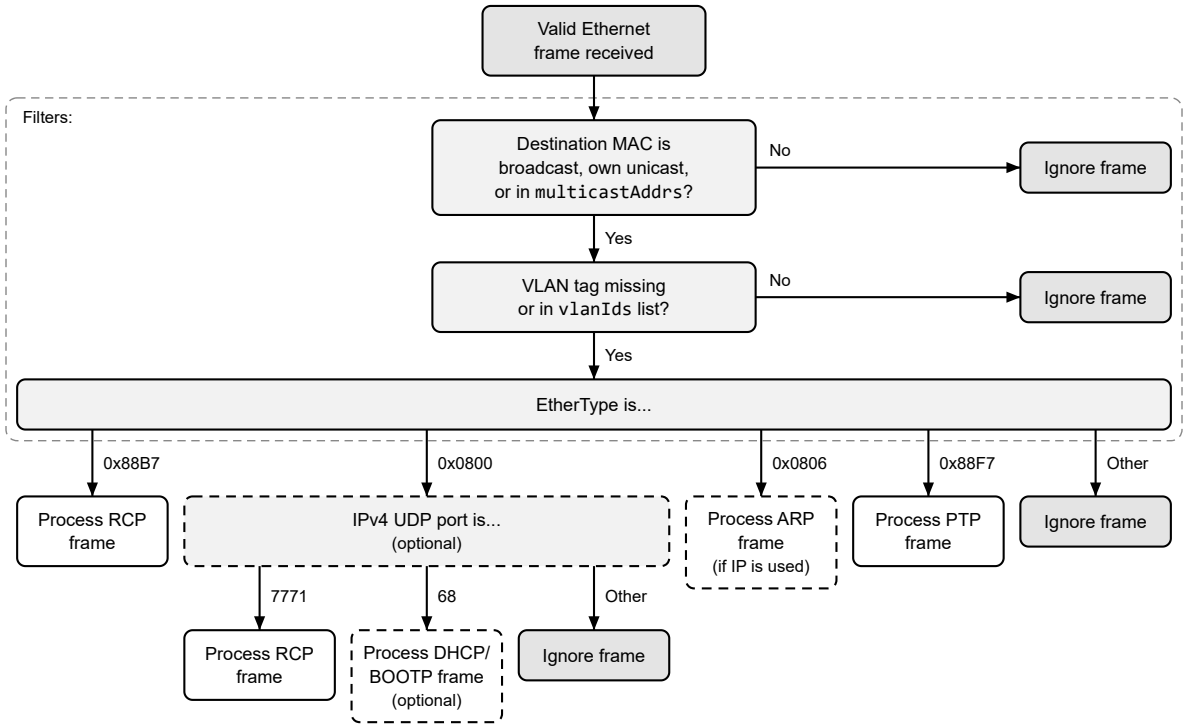
1.3 Network

An RCP server accepts RCP messages carried over raw Ethernet frames (layer 2 of the OSI model). A server that supports IP can also accept RCP messages carried over User Datagram Protocol (UDP) packets (layer 4).

Note

Raw Ethernet needs no IP configuration, but its frames do not pass routers. UDP/IP packets can be routed.

The figure shows how a server processes and filters the frames that it receives. Dashed boxes mark optional or conditional support. ARP is required if the server uses IP.



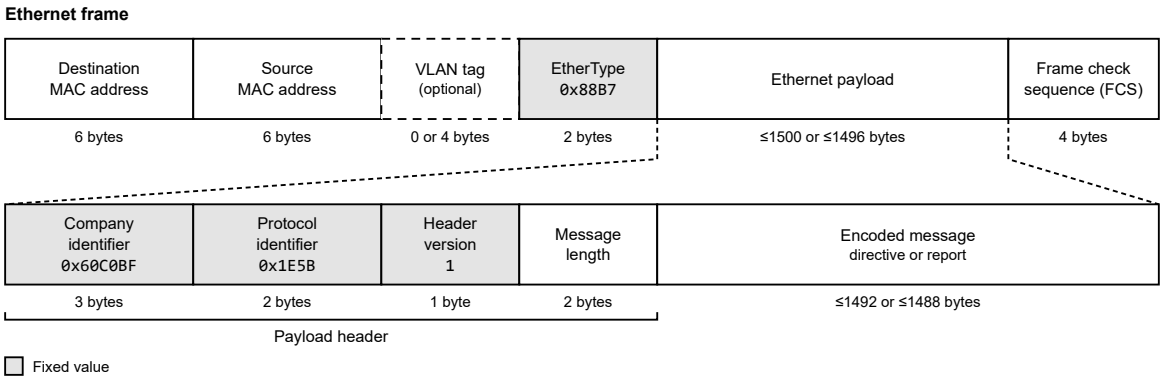
1.3.1 Message size

The figures in the sections Raw Ethernet and UDP/IP give the maximum size of the complete encoded directive or report, without and with a VLAN tag.

Fragmentation is optional. A server may reassemble fragmented IPv4 packets (RFC 791), which allows UDP messages that are larger than a frame. A server without fragmentation support ignores fragmented packets without notifying the client. Raw Ethernet has no fragmentation. A client should keep both its directives and the expected reports within one frame unless it knows that the server supports larger messages.

1.3.2 Raw Ethernet

A raw Ethernet frame that carries an RCP message is structured as defined in IEEE 802.3. The upper part of the figure shows the frame, and the lower part shows its payload. The figure applies to directives and reports.



The VLAN tag is defined in IEEE 802.1Q. The EtherType is the OUI Extended EtherType. The frame check sequence is the 32-bit cyclic redundancy check (CRC) value of the frame.

The server accepts the following destination MAC addresses:

- The broadcast address FF:FF:FF:FF:FF:FF. All servers on the Ethernet network accept it, regardless of their configuration.
- The individual address of the server. By default, it is the factory-default address (*rcp.info.fldIdentity.macAddr*), and *rcp.net.fldConfig.macAddr* replaces it.
- The multicast addresses of the server (*rcp.net.fldConfig.multicastAddrs*).

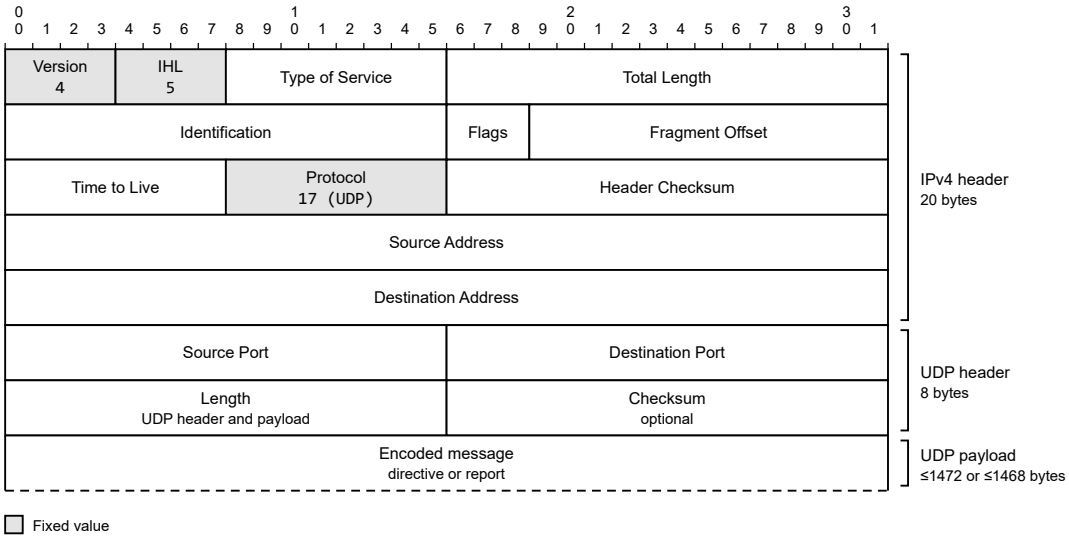
The fields of the payload header are transmitted with the most significant byte first. The company identifier is the organizationally unique identifier (OUI) of onsemi. The message length counts only the encoded message, excluding the eight-byte payload header and any Ethernet padding. The header version is independent of the version of the protocol that the server reports in *rcp.info.fldIdentity.rcpVersion*. The server ignores any bytes after the encoded message. It ignores a frame whose message length is 0 or exceeds the bytes available after the payload header. The server handles a message that it cannot decode as described in *Messages*.

The reply to a directive uses the same frame format. Its destination address is the source address of the received frame, and it uses the VLAN of the received frame. Its source address is the individual MAC address of the server, even if the received frame was addressed to a broadcast or multicast address.

1.3.3 UDP/IP

A server that supports IP also accepts RCP messages in UDP packets (RFC 768) that are carried in IPv4 packets (RFC 791). If *rcp.net.fldConfig.ipAddr* is 255.255.255.255, the IP networking stack, including UDP, is disabled and the server accepts raw Ethernet frames only.

The IPv4 header has no options. The UDP payload is the encoded message without payload header.



A client sends a directive to the UDP port 7771 (0×1E5B) of the server. The server sends its reports from the UDP port 7771 to the UDP source port that the client used.

The server accepts the following destination IP addresses:

- The limited broadcast address 255.255.255.255, as long as IP networking is enabled.
- The broadcast address of the local subnet. It has all bits after the network prefix set to one. The length of the network prefix is *rcp.net.fldConfig.subnetPrefixLength*.
- The unicast IP address of the server (*rcp.net.fldConfig.ipAddr*).
- The multicast addresses of the server (*rcp.net.fldConfig.multicastAddrs*).

1.3.4 Clients

The server assigns a unique client ID to each client that it learns from received messages or that is registered in advance. The page *Network Configuration* describes how the server identifies and registers clients.

1.3.5 Support protocols

A server that uses IP answers requests of the Address Resolution Protocol (ARP, RFC 826) for its IP address, so that a client can find its MAC address. If *rcp.net.fldConfig.gratuitousArpEnable* is set, the server also announces its address after every successful change of the IP address.

The server can acquire its IP address with the Dynamic Host Configuration Protocol (DHCP, RFC 2131) or the Bootstrap Protocol (BOOTP, RFC 951) if *rcp.net.fldConfig.ipAddr* is 0.0.0.0. Both are optional.

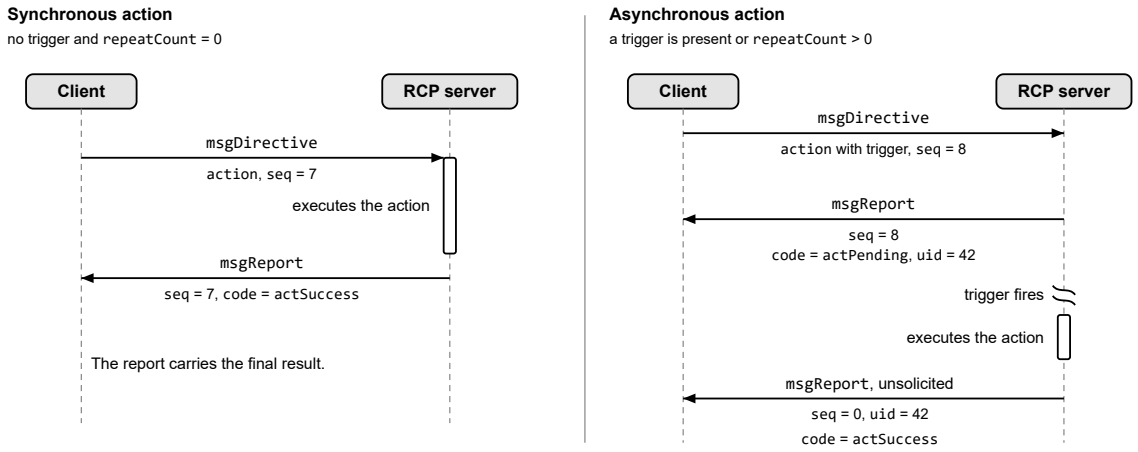
1.4 Scheduler

The scheduler is the part of an RCP server that decides when an action runs. A client can request that an action runs at a given time, repeatedly, when an event occurs, or when another action has finished. The server keeps the action in its scheduler until it is due, runs it, and reports the result. The messages are defined in *RCP Messages*.

1.4.1 Synchronous and asynchronous actions

An action without a trigger and with *fldAction.repeatCount* equal to 0 is synchronous. The server executes it as soon as the endpoint is free, and the report to the directive holds the final

result. Every other action is asynchronous, including one with an empty *timed* message. The report to the directive acknowledges scheduling (*actPending*), not execution. Execution results follow in unsolicited reports, subject to *fldAction.replyMode*. The figure shows both cases.

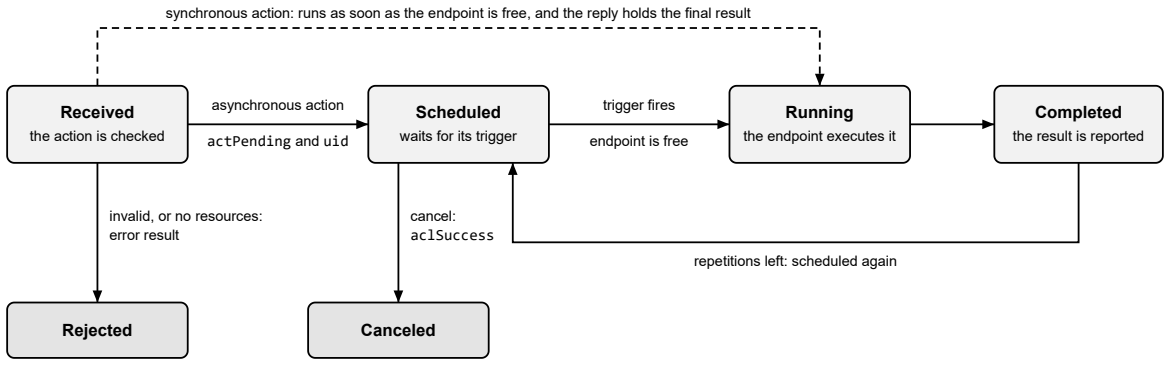


If a client sets *rcp.core.fldServerConfig.syncActionsEnable* to false, the server treats synchronous actions as asynchronous. It answers with *actPending* and runs the action as soon as the endpoint is free.

1.4.2 Life of an action

The server first checks an action. It rejects an invalid action with an error result. Examples are an invalid parameter (*actInvalidParam*), a command that the endpoint does not support (*actNotSupported*), and a scheduler with no room left (*actNoResources*). Otherwise, it schedules an asynchronous action, which then waits for its trigger. When the trigger fires and the endpoint is free, the action runs and the server reports its result. A repeating action is scheduled again. After the last repetition, the server removes the action.

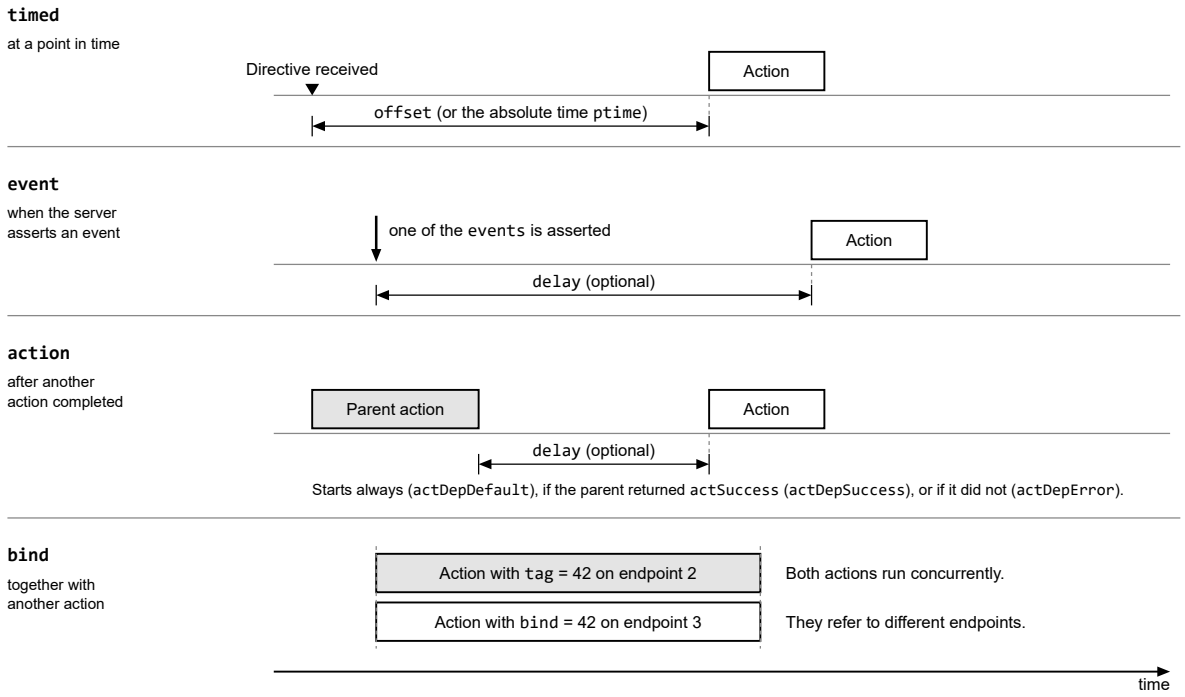
An endpoint executes one action at a time. An action that is due while its endpoint is busy waits, unless it has a hard deadline, see Late actions. Actions for different endpoints can run at the same time. The figure shows the states of an action.



A running action completes even if it is canceled (`aclPending`). After its last repetition, an action is removed from the scheduler.

1.4.3 Triggers

The trigger of an action (*fldAction.trigger*) decides when the action runs. An action without a trigger runs as soon as the endpoint is free (best effort).



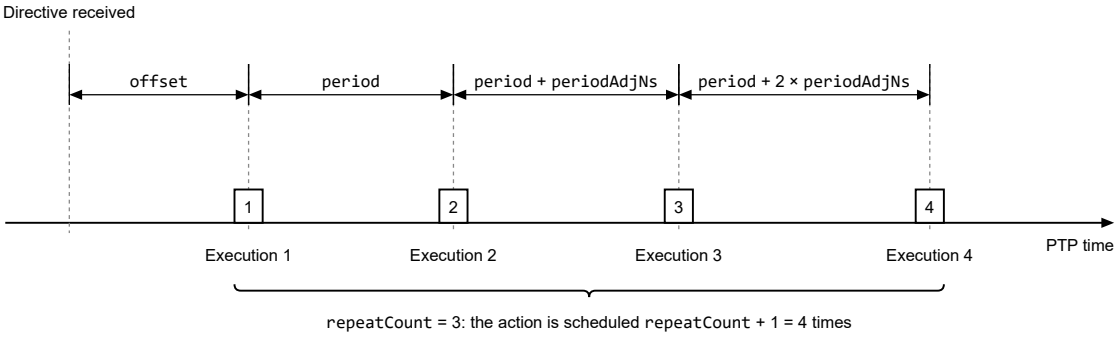
- A timed action (*fldActionTimed*) is due at its presentation time. It is either an absolute time (*fldActionTimed.ptime*) or an offset from the reception of the directive (*fldActionTimed.offset*). If neither is given, the action is due at once.
- An event action (*fldActionEvent*) runs when the server asserts one of the listed events, optionally after a delay. Examples are the change of a GPIO pin and the crossing of an ADC threshold. The events and their numbers are implementation-specific.
- A dependent action (*fldActionDependent*) runs when another action has finished, optionally after a delay. The trigger type selects whether the action runs after any result of the other action, only after a success, or only after an error.
- A bound action (*fldAction.bind*) runs together with another action, e.g., to start a PWM and a SPI controller together. A bound action addresses a different endpoint than the action it is bound to. Otherwise, the server rejects it.

1.4.4 Repetition

The field *fldAction.repeatCount* is the number of repetitions, so the action runs `repeatCount` plus one times. The value 0xFFFFFFFF repeats the action until it is canceled. A repeating action is always asynchronous.

For a timed action, *fldActionTimed.period* sets the interval of the repetitions. The server adds the period to the presentation time of the previous execution, so a delay of one execution does not shift the following ones. After each execution, the server adds the signed number of nanoseconds in *fldActionTimed.periodAdjNs* to the period. This makes sequences that become faster or slower. The figure shows an example.

The first execution is scheduled at the absolute time `ptime` or `offset` after the directive was received.



`periodAdjNs` is added to the period after every execution (here a positive value).

1.4.5 Time base

Timed actions use a timer of the server as their time base. The Precision Time Protocol (PTP) synchronizes the timer with the time of the network, see *PTP*. Servers that follow the same time master can run actions at the same time. The client can also set the timer with *`rcp.dev.opSetPtpTime`*, e.g., before it uses timed actions or enables PTP. The precision and the granularity of the timer are implementation-specific.

1.4.6 Late actions

An action is late if the server cannot run it at its presentation time, because that time is already in the past or because the endpoint is busy. The field *`fldActionTimed.hardDeadline`* decides what happens to a late action:

- If it is `false`, the default, the server runs the action anyway. A successful execution returns the warning *`actDelayed`*. A failed execution returns its error instead.
- If it is `true`, the server does not run the action. The result is the error *`actTooLate`*.

The result of an action can carry the time at which it ran (*`fldActionResult.time`*).

1.4.7 Results and replies

The field *`fldAction.replyMode`* filters the execution results of an asynchronous action: every execution, none, errors only, the last execution only, or errors and the last execution. It does not suppress the scheduling acknowledgement. The directive's *`msgDirective.seq`* independently controls that report, see *Messages*. A synchronous action ignores *`replyMode`*.

Warnings count as success, both for reply filtering and for dependent triggers. In particular, *`actDelayed`* triggers *`actDepSuccess`*, not *`actDepError`*, and does not by itself produce an *`actReplyOnError`* report.

1.4.8 Identifying and canceling actions

An action has two identifiers:

- The tag (*`fldAction.tag`*) is a label that the client chooses. It does not have to be unique, so several actions can share it. The client knows the tag in advance, e.g., to name the action in the trigger of another one. In a trigger, the tag 0 refers to the action that the server queued last, and a tag that several actions share refers to the first one found. The server returns the tag with every result.
- The unique ID (*`fldActionResult.uid`*) is generated by the server for every queued action and identifies exactly one action. The client learns it from the acknowledgement.

The cancel operation (*fldActionCancel*) selects actions by one of them. A unique ID selects one action, and a tag selects all live actions that carry it. An action is live while it is scheduled or running. The result is *aclSuccess* if a scheduled action is removed, *aclPending* if the action was already running and completes, and *aclNotFound* if no live action matches. The command *rcp.dev.opCancelAllActions* clears all scheduled actions, including those with repetitions left. A running action completes.

When an action is removed, dependents still waiting for its completion are removed without a notification or an action result. An already-triggered dependent, including one waiting out its delay, finishes its current execution under its reply mode, with no further repetitions.

The query *rcp.dev.qrySchedulerStatus* lists the actions that the scheduler holds (*rcp.core.fldSchedulerStatus*) with their IDs, tags, endpoints, remaining repetitions, and clients. The capacity of the scheduler is implementation-specific. If it is full, the server rejects a new asynchronous action with *actNoResources*.

The settings *fldAction.failsafe* and *fldAction.storeToNvm* are described on the page *Higher-level behavior*.

1.4.9 Example

The first operation of this directive sets GPIO pin 0, which is configured as an output, one second after the reception of the directive. The action has a hard deadline, so the server drops it if it is late, and it has the tag 7. The second operation clears the pin 10 ms after the first action has succeeded and names the first action by its tag. Both actions are asynchronous. The server answers each with *actPending* and reports the results later.

```
seq: 10
dir {
   action {
      hndl: 1
      gpio { set: 0x01 }
      timed { offset { sec: 1 }  hardDeadline: true }
      tag: 7
   }
}
dir {
   action {
      hndl: 1
      gpio { clr: 0x01 }
      action {
         tag: 7
         trigType: actDepSuccess
         delay { nsec: 10000000 }
      }
   }
}
```

The next directive cancels all actions with the tag 7 and then reads what the scheduler still holds:

```
seq: 11
dir { cancel { tag: 7 } }
dir { query { rcp: qrySchedulerStatus } }
```

1.5 Higher-level behavior

This page describes behavior that spans more than one message: persistent storage and fail-safe operation.

1.5.1 Persistent storage

An RCP server can have a non-volatile memory (NVM), e.g., an EEPROM. The query *rcp.dev.qryNvmStatus* tells whether it is present. An operation that needs the NVM fails with the result *actNvmNotPresent* on a server without it. The NVM holds the configuration, actions, and user data.

Configuration

The command *rcp.dev.opSaveConfig* saves the current configuration of the server. The configuration includes the network settings, such as the MAC address and the multicast addresses, and the registered clients.

At start-up, the server loads the saved configuration if there is a valid one. Otherwise, or without an NVM, it starts with its default configuration, so a change that is not saved is lost at a reset. A saved configuration that cannot be loaded, e.g., because its checksum does not match, is not applied. The status of the NVM reports the error (*rcp.core.fldNvmStatus.configError*).

The command *rcp.dev.opRestoreConfig* loads the saved configuration at any time. If it returns an error, the current configuration does not change. The command *rcp.dev.opRestoreFactoryConfig* erases the NVM and restarts the server with its default configuration.

The power-on flag of the status (*rcp.core.fldServerStatus.pwrOn*) is set at every start until the client clears it with *rcp.dev.opClearPwron*. A client that clears the flag once and polls it later can detect an unwanted reset.

Actions

An action whose field *fldAction.storeToNvm* is set is written to the NVM instead of being scheduled. The server loads the stored actions at start-up, so they run even if no client has reached the server yet. This allows fail-safe actions that exist from power-up.

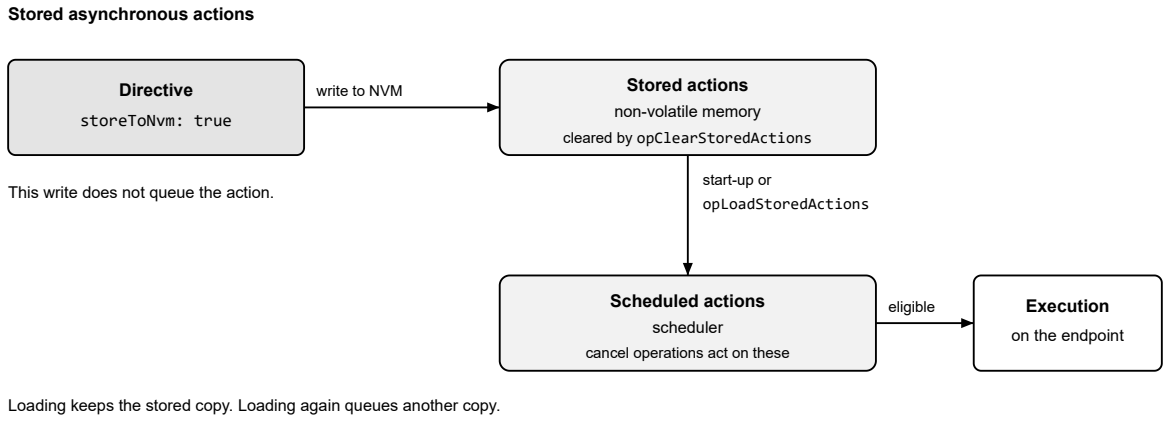


Fig. 1.1. Storage and scheduling of asynchronous actions

The command *rcp.dev.opLoadStoredActions* loads the stored actions at once. It does not remove the actions that are already loaded, so repeating it loads them again. The command *rcp.dev.opClearStoredActions* erases the stored actions, but not the loaded ones. The status of the NVM tells how many actions are stored and whether loading them failed (*rcp.core.fldNvmStatus*).

To receive reports from stored actions after a restart, register their client and save the client configuration, see *Network Configuration*.

User data

The NVM can also hold user data, which the server does not interpret. The command *rcp.dev.opManageUserData* reads or writes it (*rcp.core.fldNvmOperation*). The server rejects an access beyond the user-dedicated NVM.

1.5.2 Fail-safe operation

The network watchdog detects the loss of the client, see *Core Operations*. If it expires, the server enters the fail-safe mode and executes the fail-safe actions. These are the actions with the field *fldAction.failsafe* set. An example is an action that brings an output into a safe state. The client defines the fail-safe actions after the start of the server, or the server loads them from the NVM, see above. The server may also enter the mode for other implementation-specific reasons.

In fail-safe mode, the server rejects a synchronous action without the flag, and outside of the mode, it rejects a synchronous action with the flag. The result is *actFailsafeError* in both cases. An asynchronous fail-safe action waits in the scheduler until the server enters the mode.

What else the mode changes is implementation-specific, e.g., whether the server cancels the other scheduled actions and whether the PTP timer keeps running. The same holds for the way in which the server leaves the mode. The flag *rcp.core.fldServerStatus.liv* is set during normal operation and cleared in fail-safe mode. The command *rcp.dev.opEnterFailsafe* forces the mode. It can be used to test the fail-safe actions.

1.5.3 Example

The following directive enables and saves a one-second watchdog timeout, stores a fail-safe action, and loads it for use now. It also survives a restart. In fail-safe mode, the action toggles GPIO pin 0, already configured as an output, every 250 ms, e.g., to blink a lamp. Execution reports are suppressed.

```
seq: 12
dir { config { rcp { server { watchdogTimeoutMs: 1000 } } } }
dir { action { rcp { op: opSaveConfig } } }
dir {
   action {
      hndl: 1
      gpio { tgl: 0x01 }
      timed { period { nsec: 250000000 } }
      repeatCount: 4294967295
      replyMode: actReplyNever
      failsafe: true
      storeToNvm: true
   }
}
dir { action { rcp { op: opLoadStoredActions } } }
```

1.6 Usage hints

This page gives recommendations for the development of clients.

1.6.1 Discover the server first

Send one directive with the queries *rcp.dev.qryIdentity*, *rcp.dev.qryCapabilities*, and *rcp.dev.qryInterfaces*. The server answers them in the same order. The identity holds the protocol version, the capabilities list the optional features, and the interfaces give the handles and types of the endpoints, which every other operation needs. The last two queries are optional. A server that does not implement them answers *qryNotSupported*, and the client has to know

the endpoints in advance. Sent as a broadcast, the same directive finds all RCP servers on the network.

1.6.2 Make directives easy to match and to repeat

- Set a non-zero sequence number and use a new one for every directive whose reply you need.
- Raw Ethernet and UDP do not guarantee delivery. A missing report leaves the outcome unknown. Repeat the directive only if it is safe to execute it twice in the current state. Do not retry a transfer blindly.
- Batch operations to reduce round trips, but do not treat their order as a success condition. If an action depends on a successful configuration, check the configuration result before sending the action, see *Messages*.
- Check the result of every operation, also the warnings. A warning means that the operation succeeded, but the server did something other than requested.

1.6.3 Timing and scheduling

- Make sure that the timer of the server is valid before using timed actions. Wait until the PTP status reports it as synchronized, or set the timer, see *Scheduler*.
- Send timed actions early. Use `fldActionTimed.hardDeadline` only for an action that is useless when it is late.
- Use a repeating action with a period instead of many single actions, and a bound action to start different endpoints together.
- For a dependent action, queue its parent first and leave time to queue the dependent before the parent completes. A future timed parent and its dependent can share one directive. Use a distinct tag for each parent.
- Limit the reports of a repeating action with `fldAction.replyMode`.
- Give related actions a tag and cancel them by it. At the start of a session, look at `rcp.dev.qrySchedulerStatus`. The command `rcp.dev.opCancelAllActions` clears all pending actions, also those of other clients.

1.6.4 Safety and storage

- To reach a safe state when the client is lost, enable the network watchdog, define fail-safe actions, and store them in the NVM so that they exist from power-up, see *Core Operations* and *Higher-level behavior*.
- Send a message at least three times per watchdog timeout, so that a single lost message does not trigger the fail-safe mode. Test the fail-safe actions with `rcp.dev.opEnterFailsafe`.
- Save the configuration when it is final. Load stored actions only once.

1.6.5 Several servers and the network

- Give every server a DID, e.g., one bit per server or one bit per group, before you use DID masks, see *Core Operations*. The unique ID of its hardware (`rcp.info.fldIdentity.sid`) tells the servers apart before they have a DID.
- After changing the MAC address or the IP address of a server, send the following directives to the new address, see *Network*.
- Configure the VLANs of a server before moving it into a tagged network. With the default empty list, the server drops all tagged frames.

1.6.6 Handling messages

- Do not compare messages by their bytes. Two correct encoders can produce different bytes for the same message, see *Protobuf encoding*. Decode the messages and compare the fields.
- Expect fields in reports that your library does not know. A server that implements a newer version of the schema can send them, and Protocol Buffers libraries skip them.
- Send operations that use fields of a newer schema in a directive of their own. A server that does not know such a field rejects the whole directive, see *Messages*.

1.7 Protobuf encoding

Protocol Buffers (protobuf) encode a message as a compact sequence of bytes and decode it again. Generated code hides the encoding. The authoritative description is the Protocol Buffers guide Encoding.

1.7.1 Fields and records

A `.proto` file defines a message as a list of fields. Each field has a name, a type, and a *field number*:

```
message fldTstamp {
   fixed32 sec  = 1;
   fixed32 nsec = 2;
}
```

The encoding identifies a field by its number, not by its name. A field number never changes and is never reused.

An encoded message is a sequence of *records*, one for each field that is present. A record is a tag followed by the value. The tag holds the field number and the *wire type*, which defines how the value is stored:

```
tag = (field number << 3) | wire type
```

Table 1.1

Wire type	Number	Used for
`VARINT`	0	`int32`, `int64`, `uint32`, `uint64`, `sint32`, `sint64`, `bool`, and enums
`LEN`	2	`string`, `bytes`, embedded messages, and packed repeated fields
`I32`	5	`fixed32`, `sfixed32`, and `float`

The wire type also tells a decoder how long the value is, so that it can skip an unknown field. RCP clients skip unknown fields in reports, but RCP servers reject them in directives, see *Messages*.

1.7.2 Values

Most numbers are *varints*: each byte carries seven bits of the number, least significant group first, and its most significant bit indicates whether another byte follows. Values up to 127 take one byte, values up to 16383 take two bytes, and so on. For example, 150 is written as `96 01`:

```
150 in binary:                    1001 0110
in groups of seven bits:          0000001 0010110
least significant group first:    0010110 0000001
with the continuation bits:       10010110 00000001
                                  = 96 01
```

The types `fixed32` and `sfixed32` always take four bytes, least significant byte first. The schema uses `fixed32` for the sequence numbers (*`msgDirective.seq`*) and the fields of the timestamp (*`fldTstamp`*).

Strings, `bytes`, and embedded messages are stored as a length, written as a varint, followed by that many bytes. An embedded message is a complete message of its own, so a directive nests records several levels deep, from the operation to the action to the command of an endpoint. Each level adds a tag and a length.

1.7.3 Field presence and default values

A field that is not present takes no space. A singular scalar field without `optional` and outside a `oneof` does not track presence: the encoder omits its default value, i.e., 0, `false`, or empty, so the receiver cannot distinguish an omitted field from an explicitly sent default. For example, an omitted *`msgDirective.seq`* has the value 0.

Fields declared `optional`, message fields, and `oneof` members track presence. The decoder can tell whether such a field was present, even when its value is 0 or `false`. The schema uses `optional` for settings where “not set” means “leave unchanged” and a value means “change it”, e.g., *`rcp.ptp.fldConfig.enable`*. An empty *`fldAction.timed`* message still selects a timed trigger, and a selected `oneof` member with the value 0 is still encoded.

A `repeated` field stores a list, e.g., one record for each operation of a directive. Repeated numbers are normally packed into a `LEN` record. The protobuf data model does not distinguish an omitted list from an empty list.

1.7.4 Worked example

In the text format, the directive that queries the identity of the RCP server reads:

```
seq: 1
dir { query { rcp: qryIdentity } }
```

Its encoded form takes 11 bytes: `0D 01 00 00 00 12 04 1A 02 10 0A`.

Table 1.2

Bytes	Meaning
`0D`	Tag of *`msgDirective.seq`*: field number 1, wire type 5 (`I32`)
`01 00 00 00`	Value of `seq`: 1, as four bytes
`12`	Tag of *`msgDirective.dir`*: field number 2, wire type 2 (`LEN`)
`04`	Length of the first operation: 4 bytes
`1A`	Tag of *`fldOperation.query`*: field number 3, wire type 2 (`LEN`)
`02`	Length of the query: 2 bytes
`10`	Tag of *`fldQuery.rcp`*: field number 2, wire type 0 (`VARINT`)
`0A`	Value of *`fldQuery.rcp`*: 10, which is *`qryIdentity`*

The field *`fldQuery.hndl`* is 0, which addresses the RCP server itself, so it is not written. The maximum size of an encoded message depends on the frame type, see *Network*.

1.8 Code generators

A *code generator* reads the `.proto` files and writes a library for one programming language, consisting of message types, encoders, and decoders.

1.8.1 The Protocol Buffers compiler

`protoc`, the Protocol Buffers compiler, parses and checks the `.proto` files and runs the generators. The option `--<language>_out=<directory>` selects a built-in generator, and `--<name>_out` selects a plugin, i.e., a separate program named `protoc-gen-<name>`.

```
protoc -I protocol --python_out=generated/python \
    $(find protocol -name '*.proto')
```

1.8.2 Generators from Google

The built-in generators of `protoc` target general-purpose computers. The generated code needs the Google runtime library for the language, in a version compatible with `protoc` (see Cross-Version Runtime Guarantee), and the library allocates the messages dynamically.

The following Python script uses the code generated by `--python_out` and the `protobuf` package to build and encode the directive that queries the identity of an RCP server. It requires `generated/python` on the `PYTHONPATH`:

```
import rcp_pb2
from server import rcp_dev_pb2

directive = rcp_pb2.msgDirective(seq=1)
directive.dir.add().query.rcp = rcp_dev_pb2.qryIdentity
data = directive.SerializeToString()
# data: 0D 01 00 00 00 12 04 1A 02 10 0A
```

1.8.3 Generators for small systems

The firmware of a microcontroller often has no room for a general-purpose runtime and cannot allocate memory dynamically. Other projects provide generators for such targets, and the list of third-party add-ons on the Protocol Buffers website names many, e.g., Nanopb and protobuf-c for C and EmbeddedProto for C++. Verify that the generator supports `optional` fields.

1.8.4 Further reading

- Reference Guides: the guides of all languages that Google supports.
- Protoc Installation: how to install the compiler.

Chapter 2
RCP Messages

File: `rcp.proto`

Package: `esb`

The top-level messages of ESB, a remote control protocol (RCP), and the containers of the operations.

The client sends a directive (*`msgDirective`*) to the RCP server, and the server answers with a report (*`msgReport`*). The page *Messages* describes their structure and the page *Scheduler* how the server runs actions. The messages of the endpoints travel in the containers that this file defines. The endpoints are described on their own pages (see *Common Definitions* for the list), the RCP server itself on *RCP Server*.

The message *`msgRcp`* combines both top-level messages for tools such as Wireshark. Neither side sends it.

2.1 Example

A directive with a configuration, a synchronous action, and a query:

```
seq: 1
dir { config { hndl: 2  spi { cpol: false  cpha: false  speedHz:
1000000 } } }
dir { action { hndl: 2  spi { xfer { xft: xferWriteOnly  wdata:
"\x9F" }  xfer { xft: xferReadOnly  rsize: 2 } } } }
dir { query { rcp: qryIdentity } }
```

The report has the same *`msgReport.seq`* and holds three results, one for each operation. A directive that toggles GPIO #0, which is configured as an output, ten times at intervals of 500 ms, starting one second after its reception (an asynchronous action):

```
seq: 2
dir {
   action {
      hndl: 1
      gpio { tgl: 0x01 }
      timed { offset { sec: 1 }  period { nsec: 500000000 } }
      repeatCount: 9
   }
}
```

The server answers with *`actPending`* and the unique ID of the action, and reports the result of each execution in an unsolicited report, unless *`fldAction.replyMode`* limits it.

2.2 Protocol

2.2.1 Messages

fldConfig

Message `esb.fldConfig`

Container for all configuration operations.

Fields

- **hndl** — `uint32`
  Unique non-zero handle of the endpoint, or 0 for the RCP server itself.
- **dev** — *oneof*
  Configuration matching the type of the endpoint at *`hndl`*.
  - **rcp** — *`rcp.dev.fldConfig`*
    Configuration of the RCP server itself.
  - **gpio** — *`gpio.fldConfig`*
    Configuration of a GPIO controller.
  - **i2c** — *`i2c.fldConfig`*
    Configuration of an I2C controller.
  - **spi** — *`spi.fldConfig`*
    Configuration of a SPI controller.
  - **uart** — *`uart.fldConfig`*
    Configuration of a UART controller.
  - **pcm** — *`pcm.fldConfig`*
    Configuration of a PCM audio controller.
  - **pdm** — *`pdm.fldConfig`*
    Configuration of a PDM controller.
  - **adc** — *`adc.fldConfig`*
    Configuration of an ADC controller.
  - **sent** — *`sent.fldConfig`*
    Configuration of a SENT controller.
  - **lin** — *`lin.fldConfig`*
    Configuration of a LIN controller.
  - **pwm** — *`pwm.fldConfig`*
    Configuration of a PWM controller.

fldActionTimed

Message `esb.fldActionTimed`

Configuration of timed actions.

Fields

- **start** — *oneof*
  Presentation time of the first execution. If absent, the action runs as soon as the endpoint is free.

  > **Note**
  > The granularity and precision of the PTP timer are implementation-specific. Refer to the datasheet of the server for details.

  - **ptime** — *`fldTstamp`*
    Absolute time: the operation is executed when the internal PTP timer crosses this presentation time.
  - **offset** — *`fldTstamp`*
    Relative time: the RCP server captures the precise time at which the directive is received from the network, then schedules the operation to be executed at that time plus this offset.
- **period** — *`fldTstamp`*
  Repeat interval of this timed action. It is only used if the *`fldAction.repeatCount`* field is greater than 0. The RCP server then calculates each subsequent presentation time by

adding this interval to the previous presentation time. Delays in execution and duration of the action do not affect the interval. The *periodAdjNs* field can change the interval after every execution.

- **hardDeadline** — bool
  Decides whether a late action is dropped or executed. An action is late if it cannot be scheduled at the specified time. This happens if *ptime* lies in the past, or if the endpoint is busy executing other operations at the desired scheduling time.
  If set to true, a late action is not executed. Its result code is *actTooLate*, an error code.
  If set to false (or not set), a late action is always executed, even if the required deadline cannot be met. However, in this case its result code is *actDelayed*, a warning code, if execution succeeds. A failed execution returns its error instead.

  > **Note**
  > This field does not guarantee that the RCP server sends a reply. Whether and when the result code is reported depends on the *fldAction.replyMode* of the action. The warning *actDelayed* is treated as a successful execution, so *actReplyOnError* reports *actTooLate* but not *actDelayed*. Reply modes that report only on the last action of a series can defer the result.

- **periodAdjNs** — sint32
  If set, the specified adjustment time in nanoseconds is added/subtracted from the repeating period after each time the action is re-scheduled. This allows creating sequences of operations that repeat faster or slower in a linear fashion.

  > **Note**
  > This option is only used in conjunction with the *period* parameter. If this one is not specified, the *periodAdjNs* field is ignored.

fldActionEvent

Message esb.fldActionEvent

Configuration for event-triggered actions.

Fields

- **events** — *repeated* uint32
  Actions can optionally be triggered on specific events. If the *events* array is not empty, the action is executed when the RCP server asserts any of the events contained therein. The list of available events and associated IDs is implementation-specific. Refer to the datasheet of the server for details.
- **delay** — *fldTstamp*
  If set, this optional parameter defines the delay in PTP time format between the time the event was asserted and the time the action is scheduled.

fldActionDependent

Message esb.fldActionDependent

Configuration for an action triggered by the completion of a parent action. If the parent is removed while this action is waiting for its completion, this action is removed without a notification or an action result. An already-triggered execution, including its delay, completes under *fldAction.replyMode* but does not repeat.

Fields

- **tag** — uint32
  Tag of the parent action to wait for. The tag 0 refers to the previously queued action. See *fldAction.tag* for how a tag is resolved.
- **trigType** — *enActionDependentType*
  The trigger type (see *enActionDependentType*).
- **delay** — *fldTstamp*
  If set, the actual execution of the action is further delayed by the specified amount of time after the completion of the parent action.

fldAction

Message esb.fldAction

Container for an action that an endpoint or the RCP server executes.

Fields

- **trigger** — *oneof*
  Selects when the action runs. Without a trigger, it runs as soon as the endpoint is free. An action without a trigger and with *repeatCount* equal to 0 is synchronous, unless *rcp.core.fldServerConfig.syncActionsEnable* is false. Its execution result is returned in the reply to the directive. All other actions are asynchronous, including one with an empty *timed* message. A queued asynchronous action returns *actPending*. Its execution results follow in unsolicited reports, subject to *replyMode*.
  - **timed** — *fldActionTimed*
    Execute at a specific time, as defined by *fldActionTimed*.
  - **event** — *fldActionEvent*
    Execute upon the assertion of one or more events, as defined by *fldActionEvent*.
  - **action** — *fldActionDependent*
    Execute after the completion of another action, as defined by *fldActionDependent*.
  - **bind** — uint32
    Execute concurrently with the action that has the specified tag (see *tag*). The tag 0 refers to the previously queued action. Bound actions refer to different endpoints (handles). Otherwise, the RCP server rejects the action.
- **repeatCount** — uint32
  If set to a non-zero value, the RCP server automatically re-schedules the operation based on the specified trigger type. The total number of times the action is scheduled is *repeatCount* + 1. If *repeatCount* is set to 0, or not set (default), the action is executed once. If *repeatCount* is set to its maximum value (0xffffffff), the action is re-scheduled indefinitely, or until canceled.

  > **Note**
  > Setting this parameter to a non-zero value makes the action asynchronous regardless of the *trigger* type.

- **replyMode** — *enActionReplyMode*
  Reply mode for this action, see *enActionReplyMode*. If the action is synchronous, this parameter is ignored. For the definition of synchronous actions, see the *trigger* field documentation.
- **failsafe** — bool
  If set, the action is executed only if the RCP server is in fail-safe mode. Fail-safe mode can be triggered by the network watchdog or other implementation-specific conditions (refer to the datasheet of the server). For asynchronous operations (see the *trigger* field documentation), this parameter adds fail-safe mode as an additional condition for scheduling. For synchronous operations, if this parameter is set and the system is not

currently in fail-safe mode, the action is rejected with the *actFailsafeError* error code. If this parameter is not set, the action is rejected if the system is currently in fail-safe mode. This feature can be useful for implementing functional safety and safe operation of the server in case, for example, of loss of network connectivity.

- **storeToNvm** — bool
  If set, the action is saved in non-volatile memory and loaded on server POR or reboot.

  > **Warning**
  >
  > Actions stored to NVM are not loaded into RAM at the same time. The command *rcp.dev.opLoadStoredActions* loads all saved actions.

- **tag** — uint32
  Optional label that the client gives to the action. It does not have to be unique: several actions can carry the same tag, and 0 means that the action has no tag. The tag is how other operations refer to this action:
  - the triggers *action* (dependent) and *bind* of later actions
  - the cancel operation by tag, see *fldActionCancel.tag*

  When more than one live action carries the same tag, a trigger refers to the first one found, while canceling by tag cancels all of them. In contrast, the RCP server assigns every queued action a unique *fldActionResult.uid*, which the client only learns from the acknowledgement. It refers to exactly one action, see *fldActionCancel.uid*.
- **hndl** — uint32
  Handle of the endpoint, or 0 for the RCP server itself. Endpoint handles are unique and non-zero. The datasheet of the server and the query *rcp.dev.qryInterfaces* list the endpoints with their handles and types.
- **dev** — *oneof*
  The endpoint-specific command.
  - **rcp** — *rcp.dev.fldCmd*
    Commands for RCP functionality.
  - **gpio** — *gpio.fldCmd*
    Commands for GPIO interfaces.
  - **spi** — *spi.fldCmd*
    Commands for SPI interfaces.
  - **i2c** — *i2c.fldCmd*
    Commands for I2C interfaces.
  - **pwm** — *pwm.fldCmd*
    Commands for PWM interfaces.
  - **pcm** — *pcm.fldCmd*
    Commands for PCM audio interfaces.
  - **uart** — *uart.fldCmd*
    Commands for UART interfaces.
  - **pdm** — *pdm.fldCmd*
    Commands for PDM interfaces.
  - **adc** — *adc.fldCmd*
    Commands for ADC interfaces.
  - **sent** — *sent.fldCmd*
    Commands for SENT interfaces.
  - **lin** — *lin.fldCmd*
    Commands for LIN interfaces.

fldActionResult

Message esb.fldActionResult

Result of an action. The RCP server returns it as a direct reply to an action (*fldAction*). Such a reply reports either the execution of a synchronous action or the scheduling of an asynchronous

action. The server also returns it when an asynchronous action has run. Such an unsolicited result reports the execution.

Fields

- **code** — *enActionResult*
  Result code of the action.
- **tag** — *optional* `uint32`
  The tag that the client gave in *fldAction.tag*, only present if it was not zero. It is not necessarily unique, use the *uid* to tell actions apart.
- **seq** — *optional* `fixed32`
  The RCP server generates a sequence number for each reply of an asynchronous action (see *uid*). The number increases by one with each reply. Each action has its own series, so the number is not globally unique. The client can use it to detect missing or misordered replies of each action independently.
- **time** — *optional* *fldTstamp*
  Optional timestamp in PTP format that the RCP server captures when it executes specific types of actions (e.g., GPIO capture). The RCP server can also set it if an event triggered the action or if the action was late.
- **triggerId** — *optional* `uint32`
  For the result of an event-triggered action, this field specifies the trigger ID that became active.

  > **Warning**
  >
  > If multiple events trigger the action at the same time, only the one with the highest priority, i.e., the lowest event identifier, is reported as *triggerId*.

- **res** — *oneof*
  Endpoint-specific reply data.
  - **gpio** — `uint32`
    Result generated by GPIOs.
  - **data** — `bytes`
    Generic binary data (multiple endpoints).
  - **rcp** — *rcp.dev.fldCmdResult*
    Result of RCP commands.
- **uid** — *optional* `uint32`
  Unique non-zero ID generated by the RCP server for every queued action. It is included in the acknowledgement of the action and in every result of that action. Live actions never share a uid. Unlike the *tag*, which the client chooses, the client only learns it from the acknowledgement.

fldQuery

Message `esb.fldQuery`

Container for a query of the RCP server or of an endpoint.

Fields

- **hndl** — `uint32`
  Handle of the endpoint to query, or 0 for the RCP server itself. Endpoint handles are unique and non-zero. The datasheet of the server and the query *rcp.dev.qryInterfaces* list the endpoints with their handles and types.
- **qry** — *oneof*
  The RCP server supports synchronous queries to retrieve static or dynamic information about the RCP server itself or one of its endpoints. If none of the options is selected and *hndl* is non-zero, the query returns the configuration of the specified endpoint.

  - **rcp** — *rcp.dev.enQuery*
    Query of the RCP server itself.
  - **ep** — *enEndpointQuery*
    Query of an endpoint.

fldQueryResult

Message `esb.fldQueryResult`

Container for the result of a query.

Fields

- **code** — *enQueryResult*
  Return code of the query.
- **res** — *oneof*
  Content of the specific query reply.

  > **Note**
  >
  > *config* does not use the field *fldConfig.rcp*. To retrieve the configuration of the RCP server, use the individual queries and the field *rcp* of this message.

  - **rcp** — *rcp.dev.fldQueryResult*
    Reply to RCP server queries.
  - **config** — *fldConfig*
    Reply to endpoints config queries.
  - **epInfo** — *fldEndpointNotifInfo*
    Reply to endpoint status query.

fldNotif

Message `esb.fldNotif`

Notifications sent by the server, including endpoint events and received stream data. Each endpoint sends its notifications only to its notification owner, identified by *fldEndpointNotifInfo*. This message also carries directive-decoding errors.

Fields

- **code** — *enNotifResult*
  Result code of the notification.
- **hndl** — `uint32`
  Endpoint that generated the notification.
- **seq** — `fixed32`
  The RCP server generates a sequence number for each notification of an endpoint. The number increases by one with each notification. Each endpoint has its own series, so the number is not globally unique. The client can use it to detect missing or misordered notifications of each endpoint independently.
- **time** — *optional* *fldTstamp*
  Optional timestamp in PTP format, captured by the RCP server when triggering specific types of notifications (e.g., GPIO capture).
- **ntf** — *oneof*
  Data supplied by the notifying endpoint.
  - **gpio** — *gpio.fldNotif*
    GPIO change status notification.
  - **data** — `bytes`
    Endpoint bytes (streaming).

  - **adc** — *adc.fldNotif*
    ADC specific notifications.

fldActionCancel

Message `esb.fldActionCancel`

Selects live actions to cancel, either by tag or by uid. An action is live while scheduled or running. A scheduled action is removed. A running action completes its current execution without further repetitions.

Fields

- **by** — *oneof*
  Identifier used to select live actions.
  - **tag** — `uint32`
    Cancel all live actions that have this tag, see *fldAction.tag*.
  - **uid** — `uint32`
    Cancel the action that has this uid, see *fldActionResult.uid*.

fldOperation

Message `esb.fldOperation`

One operation of a directive.

Fields

- **deviceIdMask** — *optional* `uint32`
  Optional mask to select which RCP servers execute the operation. If not set, all receiving RCP servers execute the operation. If set, the receiving RCP server executes the operation only if the bitwise AND of its own DID and the mask is not zero. All other RCP servers receiving the directive ignore this operation and add no result for it to the report. For an example, see the page *Core Operations*. This feature is useful when using broadcast and multicast messages on the network containing operations for multiple RCP servers. This allows the client to save bandwidth and reduce latency while keeping strict control over which server does what. The DID is set with *rcp.core.fldServerConfig.did*.
- **op** — *oneof*
  Selects the operation type.
  - **action** — *fldAction*
    Action to perform.
  - **query** — *fldQuery*
    Query to perform.
  - **config** — *fldConfig*
    Configuration to apply.
  - **cancel** — *fldActionCancel*
    Actions to cancel.

fldOperationResult

Message `esb.fldOperationResult`

Container for the result of an operation. It is a direct response to an operation, an unsolicited action result, or a notification.

Fields

- **res** — *oneof*
  - **action** — *fldActionResult*
    Result of an action.

  - **query** — *fldQueryResult*
    Result of a query.
  - **config** — *enConfigResult*
    Result of a configuration.
  - **cancel** — *enAclResult*
    Result of an action cancel.
  - **notif** — *fldNotif*
    Unsolicited notification or directive-decoding error.

msgDirective

Message `esb.msgDirective`

Top-level message from the client to the RCP server. The server decodes the whole message before handling its operations. It rejects the whole directive for a decoding error. No operation is executed, and the reply contains one *fldNotif* describing the error.

Fields

- **seq** — `fixed32`
  Sequence number used to correlate directives and reports. A non-zero value is echoed in *msgReport.seq*. 0 (default) suppresses the normal reply, but not unsolicited reports or the error report for a directive rejected during decoding. The number does not suppress duplicate execution. Receiving the same directive again can execute it again.
- **dir** — *repeated* *fldOperation*
  Operations handled in list order. An asynchronous action is queued at its position, not awaited. An operation error neither rolls back earlier operations nor prevents later operations from being handled. If a reply is requested, it contains one result per operation in the same order, except for an operation ignored because of *fldOperation.deviceIdMask*.

msgReport

Message `esb.msgReport`

Top-level message from the RCP server to the client.

Fields

- **seq** — `fixed32`
  Echoes *msgDirective.seq* in a reply. In an unsolicited report, it is 0. For a directive rejected during decoding, it carries the directive's sequence number only if the server read it before detecting the error. Otherwise, it is 0.
- **res** — *repeated* *fldOperationResult*
  A reply contains one result per operation, in directive order, except for operations ignored because of *fldOperation.deviceIdMask*. A directive rejected during decoding produces one *fldNotif* instead. An unsolicited report contains asynchronous action results or endpoint notifications.

msgRcp

Message `esb.msgRcp`

Combination of the top-level messages *msgDirective* and *msgReport*. Neither side sends this message. Tools such as Wireshark use it to decode the messages of both directions, so the field set of *msgRcp* is the union of the fields of the two messages.

> **Note**
>
> The three messages use identical field numbers, so that Wireshark decodes them properly.

A client could use this message as a top-level message and omit the *res* or the *dir* field depending on the direction, but it should use the dedicated messages *msgDirective* and *msgReport*. The encoders and decoders inside the RCP server never use this message.

Fields

- **seq** — fixed32
  The interpretation depends on the direction.
- **dir** — *repeated* *fldOperation*
  List of operations to perform.
- **res** — *repeated* *fldOperationResult*
  List of operation results.

2.2.2 Enumerations

enActionDependentType

Enumeration esb.enActionDependentType

Actions can optionally be triggered on completion of other actions. This allows, for example, defining daisy chains of timed operations to be executed as a reaction to specific events, in sequence.

- **actDepDefault** — Execute as soon as the parent action finishes, regardless of its result code.
- **actDepSuccess** — Execute only if the parent action succeeded: *actSuccess* or a warning such as *actDelayed*.
- **actDepError** — Execute only if the parent action failed, i.e., its result code is an error.

enActionReplyMode

Enumeration esb.enActionReplyMode

Selects which execution results of an asynchronous action are sent in unsolicited reports. It does not affect the reply to the directive, which is controlled by *msgDirective.seq*. Warnings, including *actDelayed*, count as success rather than an error.

- **actReplyDefault** — Report every execution.
- **actReplyNever** — Report no executions.
- **actReplyOnError** — Report only executions that fail.
- **actReplyOnLast** — Report only the last execution of this action.
- **actReplyOnErrorOrLast** — Report executions that fail and the last execution of this action.

enActionResult

Enumeration esb.enActionResult

Global enum for action results.

- **actSuccess** — Action was successfully executed.
- **actPending** — Action has been successfully scheduled.
- **actDelayed** — Action was successful but executed past its due time.
- **actTooLate** — Action was not executed because it missed its hard deadline.

- **actNoResources** — Not enough server resources to perform the action.
- **actInvalidParam** — A parameter value is invalid or out of range, or a combination of parameters violates the constraints of the requested operation.
- **actNotSupported** — The requested operation or feature is not supported by this server.
- **actFailsafeError** — Operating mode mismatch for the synchronous operation.
- **actInternalError** — Unspecified internal error.
- **actCmdFailed** — The execution of the command associated with the action failed.
- **actEndpointNotEnabled** — Action failed because the endpoint is not enabled.
- **actTimeoutError** — Action failed because the execution of the action timed out.
- **actCsumError** — Action failed because of an invalid checksum.
- **actIOError** — Action failed because of a hardware/bus error.
- **actStreamNotStarted** — Action failed because the stream is not started.
- **actStreamAlreadyStarted** — Action failed because the stream is already started.
- **actNvmNotPresent** — Unable to access NVM because it is not present.

enQueryResult

Enumeration `esb.enQueryResult`

Global enum for query result code.

- **qrySuccess** — Query was successful.
- **qryUnknownDevice** — The specified handle is invalid.
- **qryInvalidParam** — A parameter value is invalid or out of range, or a combination of parameters violates the constraints of the requested operation.
- **qryInternalError** — Unspecified internal error.
- **qryEndpointNotEnabled** — The query targets an endpoint that is not enabled.
- **qryNotSupported** — The requested operation or feature is not supported by this server.

enNotifResult

Enumeration `esb.enNotifResult`

Result codes for endpoint notifications and directive decoding errors.

- **ntfSuccess** — Notification was successful.
- **ntfIOError** — Hardware/bus error.
- **ntfBufferOverrun** — Buffer overrun occurred.
- **ntfProtocolError** — The server could not decode the received message.
- **ntfProtocolTooLarge** — The received message exceeds a limit of the server.
- **ntfProtocolUnsupported** — The received message has an unknown field or several members of a oneof.
- **ntfInternalError** — Unspecified internal error.
- **ntfFlushed** — Endpoint was flushed (e.g., UART break).
- **ntfCsumError** — Checksum error (e.g., UART parity error).
- **ntfFrameError** — Frame error (e.g., UART framing error).
- **ntfOutOfMemory** — Out of memory error.

enConfigResult

Enumeration `esb.enConfigResult`

Global enum for configuration result.

- **cfgSuccess** — Configuration was successful (default).

- **`cfgParamAdjusted`** — Configuration was successful, but some parameters were adjusted by the server.
- **`cfgNotSupported`** — The requested operation or feature is not supported by this server.
- **`cfgInvalidParam`** — A parameter value is invalid or out of range, or a combination of parameters violates the constraints of the requested operation.
- **`cfgInternalError`** — Unspecified internal error.
- **`cfgBusy`** — Server is busy and cannot accept the configuration at the moment.
- **`cfgNotAllowed`** — Configuration not allowed for the item being configured because it is read-only.

enAclResult

Enumeration `esb.enAclResult`

Global enum for action cancel result.

- **`aclSuccess`** — Action cancel operation was successful.
- **`aclPending`** — Action cancel operation was successful, but the action was already in progress.
- **`aclNotFound`** — Action cancel operation failed because the action was not found.
- **`aclInternalError`** — Unspecified internal error.

Chapter 3
Common Definitions

File: `rcp_common.proto`

Package: `esb`

Definitions that several files of the RCP protocol share.

This file holds the few definitions that other files share: endpoint types and queries, notification ownership, the timestamp, and the version.

3.1 Endpoints

The type of an endpoint tells which messages describe it: *GPIO Driver*, *UART Driver*, *SPI Driver*, *I2C Driver*, *ADC Driver*, *PWM Driver*, *PDM Driver*, *PCM Audio Driver*, *LIN Driver*, or *SENT Driver*. The server lists its endpoints with their handles and types in the result of the query *`rcp.dev.qryInterfaces`* (see *Identification and Capabilities*).

3.2 Timestamps and versions

Time is expressed in the format of the Precision Time Protocol (PTP, IEEE 802.1AS), see *PTP*. Besides absolute times of the PTP timer, the protocol also uses it for time spans, such as the offset, the period, and the delays of an action.

The server reports the version of the RCP protocol that it implements in its identity (see *Identification and Capabilities*).

3.3 Protocol

3.3.1 Messages

fldEndpointNotifInfo

Message `esb.fldEndpointNotifInfo`

Notification ownership and sequence state of an endpoint. An endpoint has at most one notification owner and sends its notifications only to that client.

Fields

- **`used`** — `bool`
  True if the endpoint has a notification owner.
- **`clientId`** — `uint32`
  Client ID of that owner, valid if *`used`* is true.
- **`seq`** — `uint32`
  Last notification sequence number, valid if *`used`* is true.

fldTstamp

Message `esb.fldTstamp`

Time expressed in seconds and nanoseconds. The containing field selects an absolute PTP time or a non-negative time span, e.g., an action period or delay. The seconds component is 32 bits.

Fields

- **`sec`** — `fixed32`
  Seconds component.
- **`nsec`** — `fixed32`
  Nanoseconds component.

fldVersion

Message `esb.fldVersion`

Version record.

Fields

- **`major`** — `uint32`
  The major version number.
- **`minor`** — `uint32`
  The minor version number.
- **`patch`** — `uint32`
  The patch number.

3.3.2 Enumerations

enEndpointType

Enumeration `esb.enEndpointType`

Endpoint types handled by the RCP protocol.

- **`devTypeGpio`** — General Purpose I/O Interface (GPIO).
- **`devTypeUart`** — Universal Asynchronous Receiver-Transmitter (UART) Interface.
- **`devTypeSpi`** — Serial Peripheral Interface (SPI).
- **`devTypeI2c`** — Inter-Integrated Circuit (I2C) Interface.
- **`devTypeAdc`** — Analog-to-Digital Converter (ADC) Interface.
- **`devTypePwm`** — Pulse-Width Modulation (PWM) Interface.
- **`devTypePdm`** — Pulse-Density Modulation (PDM) Interface.
- **`devTypePcm`** — Pulse-Code Modulation (PCM) Audio Interface.
- **`devTypeLin`** — Local Interconnect Network (LIN) Interface.
- **`devTypeSent`** — Single Edge Nibble Transmission (SENT) Interface.

enEndpointQuery

Enumeration `esb.enEndpointQuery`

Types of query that an endpoint answers.

- **`epqConfig`** — Get current endpoint configuration.
- **`epqNotifInfo`** — Get notification ownership and sequence state.

Chapter 4
Server

Messages that configure, control, and query the RCP server itself, as opposed to the endpoints that it contains.

The pages group the messages by topic:

- *RCP Server*: the containers of these messages and the commands that manage the server.
- *Identification and Capabilities*: the identity, the capabilities, and the list of endpoints.
- *Core Operations*: the identification number (DID), the network watchdog, the aggregation of notifications, the scheduler, and the NVM.
- *Pins*: the pins of the server and their assignment to the functions of the endpoints.
- *Network Configuration*: the addresses, VLANs, and clients of the server and the access to its PHY.
- *PTP*: the PTP timer, the time base for timed actions and timestamps.
- *PLCA*: the physical layer collision avoidance (PLCA) of 10BASE-T1S networks.
- *Topology Discovery*: the signal delays between the nodes of a 10BASE-T1S network.
- *Vendor-Specific Features*: the settings and the status that are specific to the vendor.

A client reads `rcp.info.fldCapabilities` to find out which optional features a server has.

4.1 RCP Server

File: `server/rcp_dev.proto`

Package: `esb.rcp.dev`

Configuration, commands, and queries of the RCP server itself.

This file defines the messages for the RCP server as a whole, as opposed to the endpoints (GPIO, SPI, etc.) that it contains. The server has the handle 0, see *Operating principle*. Its messages travel in the same containers as those of the endpoints: `fldConfig` in the configuration (`esb.fldConfig`), `fldCmd` in an action (`fldAction`), and `enQuery` in a query (`fldQuery`). The result of a command is a `fldCmdResult`, the result of a query a `fldQueryResult`.

4.1.1 Configuration, commands, and queries

The settings of the server are grouped by topic, and each group has a page of its own: the core operations (*Core Operations*), the PTP protocol (*PTP*), the PLCA (*PLCA*), the pins (*Pins*), the network (*Network Configuration*), and the vendor-specific features (*Vendor-Specific Features*). The commands manage the server itself: its power state, the storage of its configuration and actions, its scheduler, and its Ethernet line and PHY.

The queries read the configuration, the status, and the identification of the server. The identification queries are described in *Identification and Capabilities*. Not all queries have to be implemented: the query enumeration marks the ones that every ESB-compliant server implements, the ones that it implements if the hardware has the feature, and the optional ones.

4.1.2 Example

To set a DID and a watchdog timeout of one second, send a keep-alive, and read the status:

```
seq: 1
dir { config { rcp { server { did: 3  watchdogTimeoutMs: 1000 } }
} }
dir { action { rcp { op: opNone } } }
dir { query { rcp: qryServerStatus } }
```

4.1.3 Protocol

Messages

fldConfig *Message* `esb.rcp.dev.fldConfig`

Container of all RCP server configurations.

Fields

- **`cfg`** — *oneof*
  - **`server`** — *`core.fldServerConfig`*
    RCP server configuration (e.g., DID).
  - **`ptp`** — *`ptp.fldConfig`*
    PTP protocol configuration.
  - **`plca`** — *`plca.fldConfig`*
    PLCA configuration.
  - **`pins`** — *`pins.fldConfig`*
    RCP pins configuration.
  - **`net`** — *`net.fldConfig`*
    Network configuration.
  - **`phyReg`** — *`net.fldPhyReg`*
    PHY registers configuration.
  - **`vendor`** — *`vendor.fldConfig`*
    Vendor-specific configuration.
  - **`client`** — *`net.fldClientConfig`*
    Client configuration.

fldCmd *Message* `esb.rcp.dev.fldCmd`

Container for all RCP server commands.

Fields

- **`op`** — *`enCmdOperation`*
  Management operation to perform.
- **`param`** — *oneof*
  Command-specific parameters.
  - **`topDisc`** — *`tpd.fldCmd`*
    Topology discovery (*`opTopologyDiscovery`*).
  - **`phyReg`** — *`net.fldPhyReg`*
    PHY registers access (*`opManagePhyRegisters`*).
  - **`time`** — *`fldTstamp`*
    PTP time set (*`opSetPtpTime`*).
  - **`nvmOp`** — *`core.fldNvmOperation`*
    NVM operation parameters.

fldCmdResult *Message* `esb.rcp.dev.fldCmdResult`

Container for results of RCP commands.

Fields

- **res** — *oneof*
  - **topDiscResult** — *tpd.fldCmdResult*
    Topology discovery result.
  - **phyData** — *net.fldPhyData*
    PHY register data.

fldQueryResult *Message* `esb.rcp.dev.fldQueryResult`

Container for the results of all queries of the RCP server.

Fields

- **rep** — *oneof*
  - **identity** — *info.fldIdentity*
    Reply to *qryIdentity*.
  - **capabilities** — *info.fldCapabilities*
    Reply to *qryCapabilities*.
  - **interfaces** — *info.fldInterfaces*
    Reply to *qryInterfaces*.
  - **pins** — *pins.fldDescriptors*
    Reply to *qryPins*.
  - **server** — *core.fldServerStatus*
    Reply to *qryServerStatus*.
  - **net** — *net.fldStatus*
    Reply to *qryNetStatus*.
  - **ptp** — *ptp.fldStatus*
    Reply to *qryPtpStatus*.
  - **plca** — *plca.fldStatus*
    Reply to *qryPlcaStatus*.
  - **scheduler** — *core.fldSchedulerStatus*
    Reply to *qrySchedulerStatus*.
  - **nvm** — *core.fldNvmStatus*
    Reply to *qryNvmStatus*.
  - **vendor** — *vendor.fldStatus*
    Reply to *qryVendorStatus*.
  - **client** — *net.fldClientStatus*
    Reply to *qryClientStatus*.

Enumerations

enCmdOperation *Enumeration* `esb.rcp.dev.enCmdOperation`

RCP server management operations.

- **opNone** — This operation is a no-op and does not alter the state of the RCP server. However, it counts as a network watchdog refresh message, so the client can use it as a keep-alive message.
- **opResetDevice** — Reset the RCP server. If set, the RCP server reboots, clearing all dynamic configuration and resetting all internal states and variables.
- **opSaveConfig** — Saves the current RCP configuration into non-volatile memory.
- **opRestoreConfig** — Restores the last saved RCP configuration from non-volatile memory. If the operation fails, the current configuration does not change.
- **opSleep** — Commands the RCP server to enter deep-sleep mode. In deep-sleep mode, all RCP server functions are switched off, and all dynamic configuration and states are lost. A portion of the RCP server stays powered to detect wakeup signals from local sources or from the Ethernet bus. The wakeup signal triggers a full (re)boot of the server.

- **`opRestoreFactoryConfig`** — Forces the RCP server to erase the NVM content and reboot. The default HW configuration is then applied when the system comes back to life.
- **`opClearPwron`** — This command clears the *`core.fldServerStatus.pwrOn`* flag in the RCP server status. See *`core.fldServerStatus`* for more information.
- **`opTopologyDiscovery`** — Commands the RCP server to initiate topology discovery if the HW supports it. 10BASE-T1S PHYs can provide this feature.
- **`opManagePhyRegisters`** — Initiates an MDIO read or write operation to configure the PHY. See also *`net.fldMdioConfig`*.
- **`opSetPtpTime`**
  Sets the value for the internal PTP timer. Setting the PTP timer with this command can yield an error of several milliseconds, depending on the client latency and the network conditions.

  Warning

  Setting the PTP timer with this method can make scheduled actions fail or be delayed for a long time, depending on the direction of the time shift.

- **`opClearStoredActions`** — Clears all actions that are currently stored in the NVM. It does not affect actions loaded into volatile memory.
- **`opEnterSilent`**
  Enter silent mode. Features such as 10BASE-T1S topology discovery or cable fault detection need the line to be silent.

  Note

  The server sends back a confirmation once it has entered silent mode, unless suppressed explicitly.

- **`opExitSilent`** — Leave silent mode. If the server was not in silent mode, this command is ignored.
- **`opEnterFailsafe`** — Forces the RCP server into fail-safe mode.
- **`opCancelAllActions`** — Resets the RCP server scheduler, clearing all pending scheduled actions. This includes actions with a repetition count that still have pending executions. A running action always completes.
- **`opGlobalWakeup`**
  Makes the PHY send a global wakeup signal on the Ethernet line to wake up all sleeping nodes on the same network segment.

  Note

  Ethernet switches and PHYs can be programmed to propagate the wakeup signal further.

- **`opLoadStoredActions`**
  Loads the stored actions from non-volatile memory. Existing actions loaded in RAM are not canceled.

> **⚠ Warning**
>
> Executing this operation multiple times causes actions to be loaded multiple times into RAM.

- **`opManageUserData`** — Access the user-dedicated NVM, i.e., write or read user data to/from non-volatile memory. See *`core.fldNvmOperation`*.

enQuery *Enumeration* `esb.rcp.dev.enQuery`

Queries of the RCP server. Queries indicated with [M] are mandatory, meaning that ESB-compliant servers implement them. Queries marked with [H] are mandatory if the specific HW supports it. Queries marked with [O] are optional to implement.

- **`qryServerConfig`** — [M] Get current RCP server configuration.
- **`qryPtpConfig`** — [H] Get current PTP protocol configuration.
- **`qryPlcaConfig`** — [H] Get current PLCA configuration.
- **`qryPinConfig`** — [M] Get current RCP pins configuration.
- **`qryNetConfig`** — [M] Get current network configuration.
- **`qryRegConfig`** — [H] Get default register configuration.
- **`qryIdentity`** — [M] Get basic server information (vendor, serial, ...).
- **`qryCapabilities`** — [O] Get server capabilities (supported features).
- **`qryInterfaces`** — [O] Retrieve a list of supported endpoints (SPI, I2C, ...).
- **`qryPins`** — [O] Retrieve a list of pins and supported functions.
- **`qryServerStatus`** — [H] Get state of the server, whether fail-safe mode was activated or not.
- **`qryNetStatus`** — [O] Get current network status.
- **`qryPtpStatus`** — [H] Get current PTP protocol status.
- **`qryPlcaStatus`** — [H] Get current PLCA status.
- **`qrySchedulerStatus`** — [M] Get RCP Scheduler status.
- **`qryNvmStatus`** — [H] Get RCP NVM status.
- **`qryVendorStatus`** — [O] Get current vendor-specific status.
- **`qryClientStatus`** — [M] Get current client status.

4.2 Identification and Capabilities

File: `server/rcp_info.proto`

Package: `esb.rcp.info`

Identity, capabilities, and endpoints of the RCP server.

A client that connects to an unknown RCP server uses these queries to find out what the server is and what it can do. They are sent to the RCP server (handle 0, see *RCP Server*), and their results are the messages of this file: the identity of the server, its capabilities, and the list of its endpoints.

The identity holds the version of the RCP protocol that the server implements and the data that identifies the server, such as its factory-default MAC address, its vendor, and its product. The capabilities tell which PHY modes and which optional network features the server supports, e.g., PLCA (*PLCA*) or topology discovery (*Topology Discovery*). The list of the endpoints gives the handle of each endpoint, which identifies it in all configurations, actions, queries, and notifications, and its type. The type selects the driver that describes the messages of the endpoint, e.g., *SPI Driver* for the type *`devTypeSpi`*.

A client can read all three results with one directive:

```
seq: 1
dir { query { rcp: qryIdentity } }
dir { query { rcp: qryCapabilities } }
dir { query { rcp: qryInterfaces } }
```

4.2.1 Protocol

Messages

fldIdentity *Message* `esb.rcp.info.fldIdentity`

RCP server identity records.

Fields

- **`rcpVersion`** — *`fldVersion`*
  The current RCP protocol version implemented by the RCP server.
- **`macAddr`** — `bytes`
  The factory-default MAC address of the server.

  > **Warning**
  >
  > This can differ from the actual MAC address the RCP server is using if the client configured a new MAC address using *`net.fldConfig`*.

  > **Note**
  >
  > Some vendor implementations do not provide a default (globally-unique) MAC address permanently stored in the RCP server.

- **`vendor`** — *optional* `string`
  Optional UTF-8 string containing the RCP server vendor name.
- **`pid`** — *optional* `uint32`
  Optional product identifier (managed by the vendor).
- **`version`** — *optional* `uint32`
  Optional product version identifier (managed by the vendor).
- **`serial`** — *optional* `string`
  Optional product serial number/string (managed by the vendor).
- **`date`** — *optional* `uint32`
  Optional manufacturing date in UNIX format (seconds since 1st Jan 1970 00:00:00).
- **`sid`** — *optional* `uint32`
  Optional system-level unique identifier (not to be confused with the DID).

fldCapabilities *Message* `esb.rcp.info.fldCapabilities`

RCP server capability records.

Fields

- **`supportedPhyModes`** — *repeated* *`net.enPhyMode`*
  List of communication standards supported by the PHY.
- **`plcaSupported`** — `bool`
  If set, indicates the RCP server supports Physical Layer Collision Avoidance (PLCA) for the PHY modes that support it (e.g., 10BASE-T1S).
- **`topologyDiscoverySupported`** — `bool`
  If set, the RCP server supports the Topology Discovery feature for the PHY modes that support it (e.g., 10BASE-T1S).

- **sqiSupported** — bool
  If set, the RCP server supports the SQI feature for the PHY modes that support it (e.g., 100BASE-T1 and 10BASE-T1S).
- **wakeSleepSupported** — bool
  If set, the RCP server supports the OPEN Alliance TC10 wake/sleep feature for the PHY modes that support it (e.g., 100BASE-T1 and 10BASE-T1S).

fldInterfaces *Message* esb.rcp.info.fldInterfaces

RCP interface descriptors.

Fields

- **dsc** — *repeated* *fldInterfaceDescriptor*
  List of endpoint descriptors. Each interface has a type and a unique ID (handle) that can be used for configuration and actions.

fldInterfaces.fldInterfaceDescriptor *Message* esb.rcp.info.fldInterfaces.fldInterfaceDescriptc

Single instance of an endpoint descriptor.

Fields

- **hndl** — uint32
  The unique endpoint identifier (handle).

  > **Note**
  >
  > This number cannot be 0 as it is reserved for the RCP server itself.

- **type** — *enEndpointType*
  Endpoint type (SPI, I2C, ...)

4.3 Network Configuration

File: server/rcp_net.proto

Package: esb.rcp.net

Network (MAC and PHY) configuration and status of the RCP server.

This file defines the addresses, the VLANs, and the clients of the RCP server and the access to its PHY. The configuration is part of the server configuration (*rcp.dev.fldConfig.net*, *rcp.dev.fldConfig.client*, and *rcp.dev.fldConfig.phyReg*). The page *Network* describes how the server accepts and answers frames.

4.3.1 Subnet

The subnet prefix length (*fldConfig.subnetPrefixLength*) determines the subnet mask and also affects the subnet broadcast address. For example, the IP address 192.168.1.1 with a prefix length of 24 results in 192.168.1.1/24, which means that the subnet mask is 255.255.255.0.

4.3.2 VLANs

The server remembers the VLAN of each client. Clients with the same MAC or IP address but different VLAN IDs are different clients. Replies use the VLAN of the request, and unsolicited notifications use the VLAN that the client used when it subscribed.

4.3.3 Clients

The server learns a client from the messages that it receives from it and numbers it with a unique, non-zero client ID. A client with an IPv4 address is identified by this address, its port, and its VLAN ID, any other client by its MAC address and VLAN ID. The client ID tells who uses an endpoint (*fldEndpointNotifInfo*) and who scheduled an action (*rcp.core.fldSchedulerStatus*). The server sends the notifications of the endpoint and the results of the action to that client.

The client configuration (*rcp.dev.fldConfig.client*) registers clients in advance, so that the server knows their addresses before they send a message. It is saved with the server configuration (see *rcp.dev.opSaveConfig*), so the server knows these clients again after a restart, with the same client IDs. The query *rcp.dev.qryClientStatus* returns the clients that the server knows, with their client IDs.

4.3.4 PHY registers

The registers of the PHY are accessed over the management data input/output (MDIO) interface. A list of accesses writes or reads registers, and the values read come back in the order of the accesses. The command *rcp.dev.opManagePhyRegisters* executes the accesses once. In the server configuration (*rcp.dev.fldConfig.phyReg*), the same message configures PHY registers.

4.3.5 Example

To accept the VLANs 10 and 20 in addition to untagged frames and to send ARP or DHCP in VLAN 10:

```
seq: 1
dir { config { rcp { net { vlanIds: [10, 20]  defaultVlanId: 10 }
} } }
```

To register two clients, one in VLAN 10 and one with the IPv4 address 192.168.1.10 (`0xC0A8010A`) and the UDP port 5000, and then to read the clients that the server knows, with their client IDs:

```
seq: 2
dir {
   config {
      rcp {
         client {
            clients { macAddr: "\x00\x01\x02\x03\x04\x05"
vlanId: 10 }
            clients { macAddr: "\x00\x01\x02\x03\x04\x06"
ipAddr: 0xC0A8010A  port: 5000 }
         }
      }
   }
}
dir { query { rcp: qryClientStatus } }
```

4.3.6 Protocol

Messages

fldStatus *Message* `esb.rcp.net.fldStatus`

Network status records.

Fields

- **phyMode** — *enPhyMode*
  Current mode the embedded PHY is operating in.

- **lnkMode** — *enLinkMode*
  Current Ethernet link mode.
- **lnkSpeed** — *enLinkSpeed*
  Current Ethernet link speed.
- **ipAddr** — *optional* `fixed32`
  Current IPv4 address.
- **sqi** — *optional* `uint32`
  Current value of SQI+.
- **subnetPrefixLength** — *optional* `uint32`
  Current subnet prefix length.

fldConfig *Message* `esb.rcp.net.fldConfig`

Network configuration container.

Fields

- **macAddr** — *optional* `bytes`
  Configured MAC address. If set, the RCP server uses the specified MAC address for communications. When the MAC address changes, the RCP server sends the reply to the configuration operation with the old MAC address as source address. If not configured (default), the RCP server uses the factory-default MAC address, if available. Otherwise, it uses a fixed or randomly generated MAC address.
- **ipAddr** — *optional* `fixed32`
  If set, changes the current IPv4 address of the RCP server. When the IPv4 address changes, the RCP server sends the reply to the configuration operation with the old IP address as source address. If *ipAddr* is 0.0.0.0, the RCP server acquires an IPv4 address with DHCP or BOOTP. If it is 255.255.255.255, IPv4 is disabled. When IPv4 is enabled, the RCP server answers directives that arrive on the UDP port 7771 (0×1E5B) and uses the source port of the client as the destination port.

  > **Warning**
  >
  > Setting the IP address to 127.x.x.x fails with the *cfgInvalidParam* code.

- **subnetPrefixLength** — *optional* `uint32`
  Configures the subnetwork prefix length, which determines the subnet mask. This also affects the subnet broadcast address. For an example, see the page *Network Configuration*.
- **multicastAddrs** — *repeated* *fldMulticastItem*
  List of the multicast addresses that the RCP server accepts.
- **vlanIds** — *repeated* `uint32`
  If set, specifies a list of VLANs that the RCP server accepts. If this list is empty, the RCP server accepts only untagged frames, and silently drops all VLAN-tagged ones. The RCP server tracks the VLAN ID for the purpose of replying to clients:
  - Clients with the same MAC/IP address and different VLAN ID are considered different clients.
  - Replies preserve the original VLAN used by the client to communicate with the RCP server.

  Unsolicited RCP notifications use the VLAN that the client used for subscribing to the specific notification / endpoint.
- **defaultVlanId** — *optional* `uint32`
  The VLAN ID to be used for unsolicited non-RCP protocol frames, for example ARP announcements or DHCP. If set to 0 (default), non-RCP protocols use untagged frames.
- **gratuitousArpEnable** — *optional* `bool`
  If set to true, the server sends an ARP announcement after a successful IP address change. If set to false (default), the server does not announce its IP address on boot, regardless of

the DHCP or static IP settings.

fldConfig.fldMulticastItem *Message* `esb.rcp.net.fldConfig.fldMulticastItem`

Single entry of the multicast address list.

Fields

- **`addr`** — *oneof*
  - **`ipAddr`** — `fixed32`
    Multicast IPv4 address.
  - **`macAddr`** — `bytes`
    Multicast MAC address.

fldClientConfigItem *Message* `esb.rcp.net.fldClientConfigItem`

Address of a client that the RCP server registers in advance. If the client has an IPv4 address, the RCP server identifies it by this address, its UDP port, and its VLAN ID, and sends its messages to it in UDP datagrams. Otherwise, the RCP server identifies the client by its MAC address and its VLAN ID, and sends its messages to it in raw Ethernet frames.

Fields

- **`macAddr`** — `bytes`
  MAC address of the client. The RCP server uses it as the destination address of the Ethernet frames that carry its messages to the client.
- **`vlanId`** — `uint32`
  VLAN ID of the client. If 0 (default), the client does not use a VLAN, and the RCP server sends its messages to it in untagged frames. Otherwise, it sends them in frames that carry this VLAN ID in their tag.
- **`ipAddr`** — *optional* `fixed32`
  IPv4 address of the client. If set to a non-zero address, the RCP server sends its messages to the client in UDP datagrams to this address and to the *`port`*.
- **`port`** — *optional* `uint32`
  UDP port of the client, to which the RCP server sends its UDP datagrams. It is only used if *`ipAddr`* is set.

fldClientConfig *Message* `esb.rcp.net.fldClientConfig`

Registers clients in advance, i.e., before they send their first message. The RCP server learns a client from the messages that it receives from it. A registered client is known to the RCP server before it sends a message, so the RCP server can send messages to it without having received one from it first, e.g., the results of a stored action that it loads on boot. The client configuration is part of the server configuration. If it is saved (see *`rcp.dev.opSaveConfig`*), the RCP server knows the clients again after a restart, with the same client IDs, so that the stored actions that it loads still report to their clients.

Fields

- **`clients`** — *repeated* *`fldClientConfigItem`*
  Clients to register. For each entry, the RCP server adds the client and assigns a client ID to it. If the RCP server already knows the client, it keeps the client ID. The query *`rcp.dev.qryClientStatus`* returns the client IDs.

fldClientStatusItem *Message* `esb.rcp.net.fldClientStatusItem`

A client that the RCP server knows, with the client ID that the RCP server assigned to it. The address fields have the same meaning as those of *`fldClientConfigItem`*.

Fields

- **clientId** — uint32
  Client ID that the RCP server assigned to the client. It is unique among the clients that the RCP server knows. The ID identifies the notification owner of an endpoint (*fldEndpointNotifInfo.clientId*) or the client that scheduled an action (*rcp.core.fldSchedulerStatus.fldSchedulerItem.clientId*).
- **macAddr** — bytes
  MAC address of the client, 6 bytes.
- **vlanId** — *optional* uint32
  VLAN ID of the client. Not set if the client does not use a VLAN.
- **ipAddr** — *optional* fixed32
  IPv4 address of the client. Not set if the client does not have an IPv4 address.
- **port** — *optional* uint32
  UDP port of the client. Set together with *ipAddr*.

fldClientStatus *Message* esb.rcp.net.fldClientStatus

Status of the clients of the RCP server, returned by the query *rcp.dev.qryClientStatus*.

Fields

- **clients** — *repeated* *fldClientStatusItem*
  The clients that the RCP server knows: the ones that it learned from their messages, and the ones that were registered with *fldClientConfig*.

fldMdioConfig *Message* esb.rcp.net.fldMdioConfig

MDIO access container.

Fields

- **phyAddr** — uint32
  PHY MDIO address.
- **mmd** — *optional* uint32
  MMD address (0 = Clause 22).
- **regAddress** — uint32
  Register address.
- **data** — *optional* uint32
  Register data for write operations.

fldPhyReg *Message* esb.rcp.net.fldPhyReg

Container for a list of MDIO register accesses.

Fields

- **mdioList** — *repeated* *fldMdioConfig*
  List of MDIO accesses.
- **key** — *optional* uint32
  Key that gives access to vendor-specific capabilities of the PHY. The meaning of the key and of the accesses that it enables is implementation-specific (refer to the datasheet of the server).

fldPhyData *Message* esb.rcp.net.fldPhyData

Container for MDIO register read data.

Fields

- **data** — *repeated* uint32
  List of MDIO register data for read operations.

Enumerations

enPhyMode *Enumeration* `esb.rcp.net.enPhyMode`

Embedded Ethernet PHY modes.

- **`phyUnknown`** — Unknown or no PHY.
- **`phy10BASE_T1S`** — 10BASE-T1S PHY.
- **`phy100BASE_T1`** — 100BASE-T1 PHY.

enLinkMode *Enumeration* `esb.rcp.net.enLinkMode`

Ethernet link modes.

- **`lnkModeUnknown`** — Unknown or no link mode.
- **`lnkModeHalfDuplex`** — Half duplex link.

enLinkSpeed *Enumeration* `esb.rcp.net.enLinkSpeed`

Embedded Ethernet link speeds.

- **`lnkSpeedUnknown`** — Unknown or no link speed.
- **`lnkSpeed10M`** — 10 Mbps supported.
- **`lnkSpeed100M`** — 100 Mbps supported.

4.4 Pins

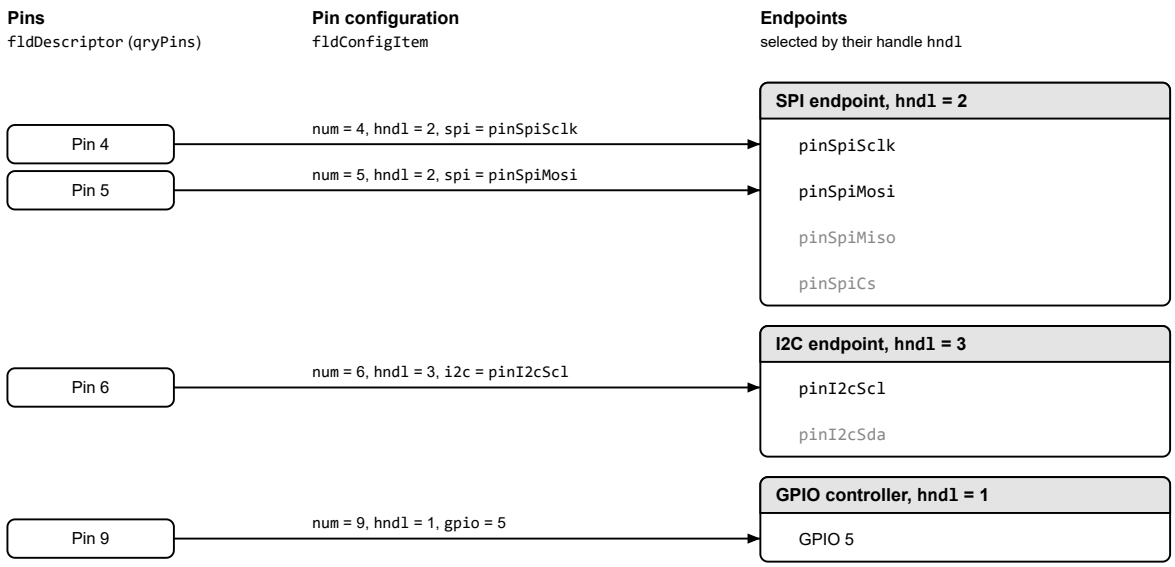
File: `server/rcp_pins.proto`

Package: `esb.rcp.pins`

Pins of the RCP server: descriptors and configuration.

The RCP server has a number of physical pins. A pin serves a function of an endpoint, identified by its handle, e.g., the clock of an SPI endpoint or a GPIO. Depending on the server, the mapping of the pins to the endpoints is either static or configurable. On a server with a static mapping, each pin has a fixed endpoint and function. On a server with a configurable mapping, the pin configuration assigns each pin to an endpoint and to a function of that endpoint.

This file defines the descriptors that tell what the pins can do and the configuration that sets them up. The configuration is part of the server configuration (*`rcp.dev.fldConfig.pins`*). The figure shows the configuration of a server with a configurable mapping:



Each configuration item also sets, per pin, the pull-up or pull-down resistor (`pullMode`), the driver (`driverMode`) and the pad strength (`padStrength`). Example data. The available pins and functions depend on the server.

4.4.1 Pin descriptors and functions

A query returns a descriptor for each pin: its unique identifier, its capabilities, such as input, output, driver modes, or weak pull resistors, and the endpoints and functions that the pin can serve. On a server with a static mapping, that is exactly one endpoint and function. The functions of the pins depend on the type of the endpoint, e.g., clock, data in, data out, and chip-select for an SPI endpoint. There is one enumeration of functions for each group of endpoint types.

4.4.2 Pin configuration

The configuration identifies a pin by the identifier of its descriptor. On a server with a configurable mapping, it assigns the pin to an endpoint. The function within the endpoint is either fixed by the handle or selected explicitly, if the server supports it. On a server with a static mapping, the endpoint and the function are fixed. In both cases, the configuration sets the electrical properties of the pin, such as the pull resistor, the driver mode, and the pad strength. A configuration that the pin or the server does not support fails with *`cfgNotSupported`*. The configuration can be read back with a query.

4.4.3 Example

To use the pins 4 and 5 as clock and data output of the SPI endpoint with the handle 2, and the pin 9 as GPIO 5 with a pull-up resistor:

```
seq: 1
dir {
   config {
      rcp {
         pins {
            pin { num: 4  hndl: 2  spi: pinSpiSclk }
            pin { num: 5  hndl: 2  spi: pinSpiMosi }
            pin { num: 9  hndl: 1  gpio: 5  pullMode: pullUp }
         }
      }
   }
}
```

4.4.4 Protocol

Messages

fldDescriptors *Message* `esb.rcp.pins.fldDescriptors`

RCP server pin descriptors.

Fields

- **`dsc`** — *repeated* *`fldDescriptor`*
  List of pin descriptors.

fldDescriptors.fldDescriptor *Message* `esb.rcp.pins.fldDescriptors.fldDescriptor`

Single instance of a pin descriptor.

Fields

- **`num`** — `uint32`
  Unique pin identifier within the controller.
- **`caps`** — `uint32`
  Pin capabilities. *`caps`* is a bit mask, according to the *`enCapability`* enum.
- **`fun`** — *repeated* *`fldFunction`*
  List of endpoints/functions the pin can be associated with. If not present (empty), all handles and functions are supported. On a server with a static mapping, the list holds the one endpoint and function that the pin is fixed to.

fldDescriptors.fldDescriptor.fldFunction *Message* `esb.rcp.pins.fldDescriptors.fldDescriptor.fl`

Fields

- **`hndl`** — `uint32`
  Handle of the endpoint.
- **`sub`** — *oneof*
  - **`uart`** — *`enPinFuncUart`*
    UART/LIN subfunction.
  - **`spi`** — *`enPinFuncSpi`*
    SPI/ADC subfunction.
  - **`i2c`** — *`enPinFuncI2c`*
    I2C subfunction.
  - **`pcm`** — *`enPinFuncPcm`*
    PCM subfunction.
  - **`pdm`** — *`enPinFuncPdm`*
    PDM subfunction.

fldConfig *Message* `esb.rcp.pins.fldConfig`

Container for the pin configuration message.

Fields

- **`pin`** — *repeated* *`fldConfigItem`*
  List of pin configurations.

fldConfig.fldConfigItem *Message* `esb.rcp.pins.fldConfig.fldConfigItem`

Single instance of a pin configuration.

Fields

- **num** — uint32
  Unique pin identifier within the RCP server, as reported by *dev.qryPins*.
- **hndl** — *optional* uint32
  If set, associates the pin with a specific endpoint instance. The exact pin function within the specified endpoint (e.g., SPI MISO or SPI MOSI) can be fixed based on the specified endpoint handle, or it can be further specified using the *fun* parameter (within this message), if supported by the server hardware. If the pin cannot be associated with the specified endpoint instance, the configuration operation fails with the *cfgNotSupported* error code.
- **pullMode** — *optional* *enPull*
  If set, enables or disables the pin's pull-up/down resistor when the endpoint supports it and the pin is in push-pull mode. If the pin does not support the specified *pullMode*, the configuration operation fails with the *cfgNotSupported* error code.
- **driverMode** — *optional* *enDriver*
  If set, changes the pin driver configuration. If the pin does not support the specified *driverMode*, the configuration operation fails with the *cfgNotSupported* error code.
- **padStrength** — *optional* uint32
  If set, defines the pad strength in microamperes (uA). The specific server may round this parameter to the nearest supported value. To retrieve the actual value set by the HW, the client can use the *dev.qryPinConfig* query.
- **fun** — *oneof*
  If set, defines the pin function within the endpoint specified by the *hndl* field. If a server implementation does not support the explicit selection of the pin function (i.e., the endpoint's pin locations are fixed) and this field is set, the configuration fails with the *cfgNotSupported* error code.
  - **uart** — *enPinFuncUart*
    UART/LIN pin functions.
  - **spi** — *enPinFuncSpi*
    SPI/ADC pin functions.
  - **i2c** — *enPinFuncI2c*
    I2C pin functions.
  - **pcm** — *enPinFuncPcm*
    PCM pin functions.
  - **pdm** — *enPinFuncPdm*
    PDM pin functions.
  - **gpio** — uint32
    GPIO pin index (0-based index within the controller).

Enumerations

enCapability *Enumeration* esb.rcp.pins.enCapability

Capabilities of a pin, used as bit mask values.

- **capNone** — No capabilities.
- **capInput** — Pin can be configured as input.
- **capOutput** — Pin can be configured as output.
- **capDrvPushPull** — Pin supports push-pull driver mode.
- **capDrvOpenDrain** — Pin supports open-drain driver mode (cannot drive strong '1').
- **capDrvOpenSource** — Pin supports open-source driver mode (cannot drive strong '0').
- **capTristate** — Pin supports tristate mode.
- **capRpu** — Pin has a weak pull-up resistor.
- **capRpd** — Pin has a weak pull-down resistor.
- **capIrq** — Pin can be used as an IRQ source for notifications.

enPinFuncUart *Enumeration* `esb.rcp.pins.enPinFuncUart`

Pin functions for the UART and LIN endpoints.

- **`pinUartTx`** — Transmit Data.
- **`pinUartRx`** — Receive Data.
- **`pinUartRts`** — Ready To Send.
- **`pinUartCts`** — Clear To Send.

enPinFuncSpi *Enumeration* `esb.rcp.pins.enPinFuncSpi`

Pin functions for the SPI and the ADC endpoints.

- **`pinSpiSclk`** — Serial Clock.
- **`pinSpiMosi`** — Master Out Slave In.
- **`pinSpiMiso`** — Master In Slave Out.
- **`pinSpiCs`** — Chip-Select.

enPinFuncI2c *Enumeration* `esb.rcp.pins.enPinFuncI2c`

Pin functions for the I2C endpoint.

- **`pinI2cScl`** — Serial Clock.
- **`pinI2cSda`** — Bidirectional serial data.

enPinFuncPcm *Enumeration* `esb.rcp.pins.enPinFuncPcm`

Pin functions for the PCM audio endpoint.

- **`pinPcmSck`** — Serial Clock.
- **`pinPcmSd`** — Serial Data.
- **`pinPcmWs`** — Word-Select / LR / Frame.

enPinFuncPdm *Enumeration* `esb.rcp.pins.enPinFuncPdm`

Pin functions for the PDM endpoint.

- **`pinPdmDin`** — Data In.
- **`pinPdmClk`** — Clock Out.

enPull *Enumeration* `esb.rcp.pins.enPull`

Pin pull-up/down mode.

- **`pullNone`** — Disable pull-up/down.
- **`pullUp`** — Enable pull-up resistor.
- **`pullDw`** — Enable pull-down resistor.

enDriver *Enumeration* `esb.rcp.pins.enDriver`

GPIO/PWM pin driver mode.

- **`drvPushPull`** — Push-pull: actively drives both ‘0’ and ‘1’.
- **`drvOpenDrain`** — Open-drain: actively drives ‘0’ only (cannot drive strong ‘1’).
- **`drvOpenSource`** — Open-source: actively drives ‘1’ only (cannot drive strong ‘0’).

4.5 PLCA

File: `server/rcp_plca.proto`

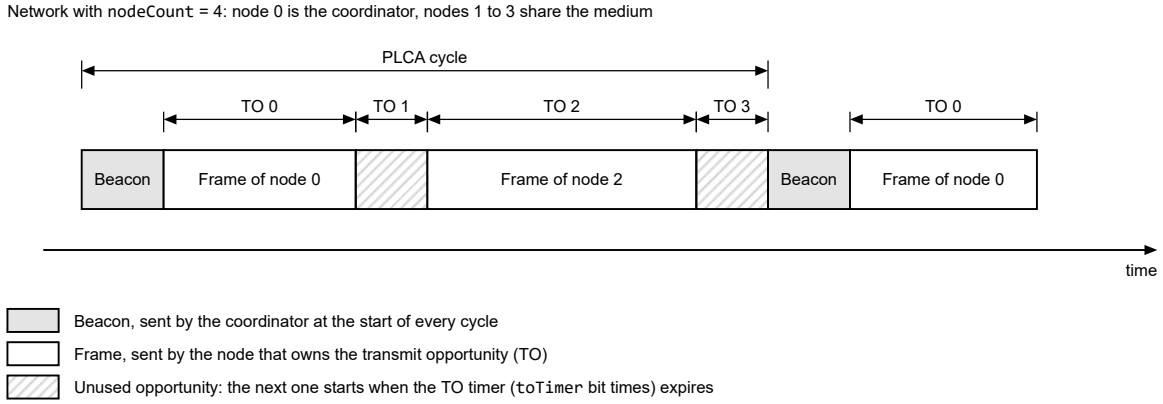
Package: `esb.rcp.plca`

Physical layer collision avoidance (PLCA) configuration and status of the RCP server.

10BASE-T1S is a multidrop network: all nodes share the same wire pair. PLCA lets them take turns instead of detecting collisions. The RCP server supports it if *`rcp.info.fldCapabilities.plcaSupported`* is set (see *Identification and Capabilities*). The configuration is part of the server configuration (*`rcp.dev.fldConfig.plca`*).

4.5.1 PLCA cycle

One node of the network is the coordinator. It has the node ID 0, and every other node has a unique node ID. The coordinator starts each PLCA cycle with a beacon. After the beacon, every node gets one transmit opportunity (TO) in the order of the node IDs, and may send a frame in it. A node that has nothing to send lets its opportunity pass. The opportunity then lasts a configurable number of bit times, the TO timer, before the next one starts. The cycle ends after the opportunity of the last node, and the coordinator sends the next beacon.



The time of a cycle depends on the number of nodes, their frames, and the TO timer for the unused opportunities. If bursts are enabled, a node can send more than one frame in its opportunity.

4.5.2 Protocol

Messages

fldStatus *Message* `esb.rcp.plca.fldStatus`

Data structure representing the PLCA status.

Fields

- **`status`** — `bool`
  The global PLCA operational status (True = UP).
- **`rxInTo`** — *optional* `bool`
  Diag: multiple nodes sharing the same node ID were detected.
- **`unexpB`** — *optional* `bool`
  Diag: multiple coordinators were detected.
- **`bcnBfTo`** — *optional* `bool`
  Diag: wrong PLCA node count detected.

fldConfig *Message* `esb.rcp.plca.fldConfig`

Data structure representing the PLCA configuration.

Fields

- **`enable`** — *optional* `bool`
  PLCA administrative state.
- **`nodeId`** — *optional* `uint32`
  Unique ID assigned to the node (0 = Coordinator, 255 = disabled).
- **`nodeCount`** — *optional* `uint32`
  Maximum number of nodes on the network (only for coordinator, default = 8).
- **`toTimer`** — *optional* `uint32`
  Transmit Opportunity Timer (default = 32).
- **`burstTimer`** — *optional* `uint32`
  Burst Timer in bit times (default = 128).
- **`burstCount`** — *optional* `uint32`
  PLCA burst count (0 = disabled, default).

4.6 PTP

File: `server/rcp_ptp.proto`

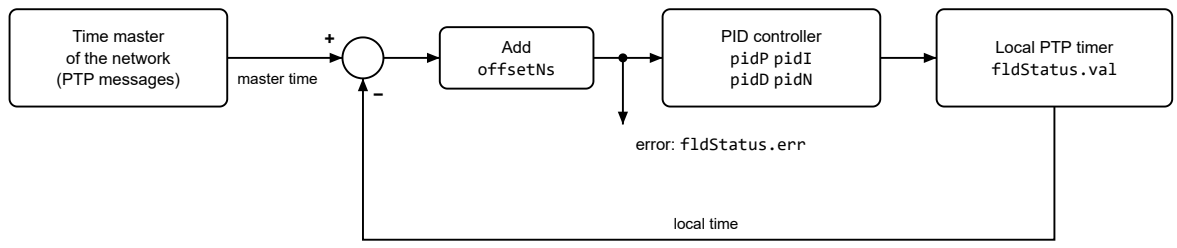
Package: `esb.rcp.ptp`

Precision Time Protocol (PTP) configuration and status of the RCP server.

The RCP server has a PTP timer, a clock that is synchronized with the time of the network by the Precision Time Protocol (PTP, IEEE 802.1AS). The timer provides the time base of the server: actions can be scheduled at a point in time (*`fldActionTimed`*), and notifications and results carry timestamps (*`fldTstamp`*). All servers of a network that follow the same time master share a common time, which allows synchronized actions on different servers. The configuration is part of the server configuration (*`rcp.dev.fldConfig.ptp`*).

4.6.1 Synchronization

While the PTP protocol is enabled, the server compares its timer with the time of the master and corrects the timer with a control loop, in which the error is the input of a PID controller. The parameters of the controller tune the loop, and an offset compensates systematic delays, such as the wire length.



The timer drives timed actions and the timestamps of reports. `fldStatus.synced` tells whether it is synchronized.
The synchronization is lost when no synchronization is received for `syncTimeoutMs`.

The client can also set the timer with a command, e.g., before it uses timed actions or enables the PTP protocol, see *`rcp.dev.enCmdOperation`*.

4.6.2 Example

To enable the PTP protocol with the proportional gain 4.0 (`pidP: 64`), the integral gain 0.5 (`pidI: 8`), and an offset of 500 ns for the delay of the wire, to declare the loss of synchronization after one second without a synchronization message, and then to read the status:

```
seq: 1
dir {
   config {
      rcp {
         ptp { enable: true  syncTimeoutMs: 1000  offsetNs: 500
pidP: 64  pidI: 8  pidD: 0  pidN: 0 }
      }
   }
}
dir { query { rcp: qryPtpStatus } }
```

4.6.3 Protocol

Messages

fldConfig *Message* `esb.rcp.ptp.fldConfig`

PTP protocol settings. The gains *`pidP`*, *`pidI`*, and *`pidD`* have four fractional bits: the gain is the field value divided by 16. The supported coefficients and their ranges are server-dependent. Refer to the datasheet. The PID controller adds three terms: the error multiplied by the proportional gain, the sum of the errors multiplied by the integral gain, and the change of the error since the previous synchronization message multiplied by the derivative gain. The sum is shifted left by N bits, i.e., multiplied by 2 to the power of N, and adjusts the rate of the PTP timer.

Fields

- **`enable`** — *optional* `bool`
  If set, enables or disables the PTP protocol. While the protocol is enabled, the RCP server corrects its PTP timer with the time of the PTP master. While it is disabled, the RCP server does not correct the timer, but the client can still set the timer with the command *`rcp.dev.opSetPtpTime`*.
- **`syncTimeoutMs`** — *optional* `uint32`
  If set, configures the timeout for losing synchronization in milliseconds. If the RCP server receives no synchronization message from the PTP master within this time, it declares the loss of synchronization and clears *`fldStatus.synced`* until the next synchronization message arrives. If 0, the timeout is disabled, and the RCP server never declares the loss of synchronization.
- **`offsetNs`** — *optional* `uint32`
  If set, configures the error offset for synchronization in nanoseconds. This value is added to the calculated error before it is used for clock adjustment, effectively shifting the clock reference point. This can be used to compensate for systematic delays in the system, for example the wire length. The error that the RCP server reports in *`fldStatus.err`* does not include the offset.
- **`pidP`** — *optional* `uint32`
  If set, sets the “P” coefficient of the PID algorithm, i.e., the proportional gain by which the error is multiplied.
- **`pidI`** — *optional* `uint32`
  If set, sets the “I” coefficient of the PID algorithm, i.e., the integral gain by which the sum of the errors since the start of the synchronization is multiplied.
- **`pidD`** — *optional* `uint32`
  If set, sets the “D” coefficient of the PID algorithm, i.e., the derivative gain by which the change of the error since the previous synchronization message is multiplied.

- **pidN** — *optional* `uint32`
  If set, sets the N shift of the PID algorithm: the sum of the three terms of the controller is shifted left by N bits, i.e., multiplied by 2 to the power of N, before it adjusts the rate of the PTP timer. The shift scales the three gains together.

fldStatus *Message* `esb.rcp.ptp.fldStatus`

PTP protocol status, returned by the query *`rcp.dev.qryPtpStatus`*.

Fields

- **val** — *`fldTstamp`*
  The current value of the PTP timer, read when the RCP server handles the query. If the timer is neither synchronized (see *`synced`*) nor set with a command, its value does not correspond to the time of the master.
- **err** — *`fldTstamp`*
  The calculated error: the difference between the time of the PTP master and the PTP timer, determined with the last synchronization message and compensated with *`fldConfig.offsetNs`*. It is an absolute value, so its sign is not reported.
- **synced** — `bool`
  If set, the PTP timer is currently synchronized: the RCP server received a synchronization message from the PTP master, and it has not declared the loss of synchronization since, see *`fldConfig.syncTimeoutMs`*.

4.7 Topology Discovery

File: `server/rcp_tpd.proto`

Package: `esb.rcp.tpd`

Commands for the topology discovery of 10BASE-T1S networks.

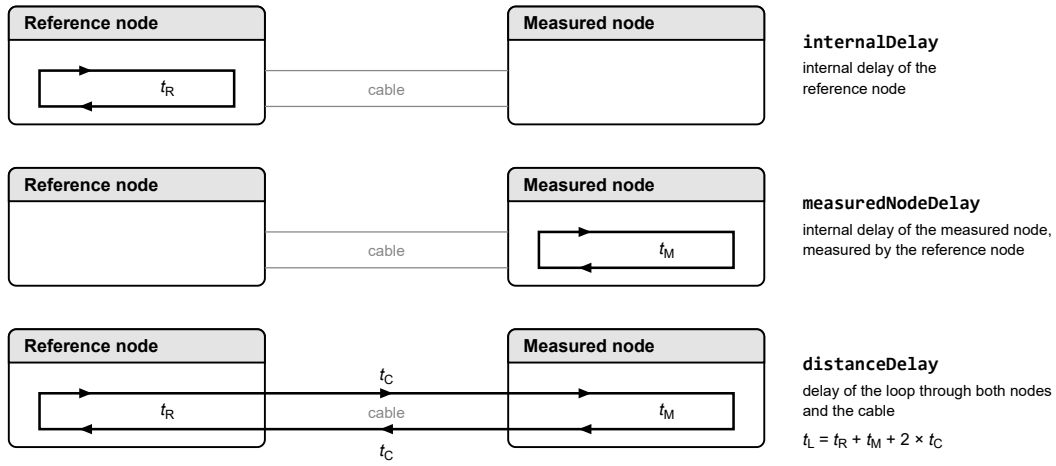
On a 10BASE-T1S network all nodes share the same line. Topology discovery measures signal delays on that line, from which a client can derive the layout of the network. The RCP server supports it if *`rcp.info.fldCapabilities.topologyDiscoverySupported`* is set (see *Identification and Capabilities*). The details of the measurement are PHY-dependent, refer to the datasheet.

The measurement involves two nodes, a reference node and a measured node. The client starts it on both nodes with the command *`rcp.dev.opTopologyDiscovery`*, each with its own role. The nodes then measure together over the line: first the internal delays of the nodes, then the delay between them. A node waits for the other node at most for the timeout of its command. The measurement requires the line to be silent. The commands *`rcp.dev.opEnterSilent`* and *`rcp.dev.opExitSilent`* switch an RCP server to and from the silent mode for this purpose.

The server answers with the delays that it measured and the status of the measurement. The delays are counts, which the section Distance converts into the distance between the nodes.

4.7.1 Distance

The result of the reference node contains the three delays that the distance needs. Each one is a count: the number of passes of a pulse around its loop within the duration of the measurement. The figure shows the three loops:



Each delay is the number of passes of a pulse around its loop within the duration `durationMs` of the measurement. The result of the reference node contains all three delays.

The time of one pass around a loop is the duration divided by the count. With the duration T = *`fldCmdResult.durationMs`* × 10^6 ns, the delays are t_R = T / *`fldCmdResult.internalDelay`* for the reference node, t_M = T / *`fldCmdResult.measuredNodeDelay`* for the measured node, and t_L = T / *`fldCmdResult.distanceDelay`* for the loop through both nodes.

The loop through both nodes passes the cable twice, once in each direction, so t_L = t_R + t_M + 2 × t_C, where t_C is the delay of the cable in one direction. The delay of the cable and the distance d between the nodes follow from it:

t_C = (t_L - t_R - t_M) / 2

d = t_C / v

where v is the propagation delay of the cable in nanoseconds per meter.

4.7.2 Example

To measure for 10 ms, with a timeout of one second, as the measured node:

```
seq: 1
dir {
   action {
      rcp { op: opTopologyDiscovery  topDisc { role:
roleMeasuredNode  durationMs: 10  timeoutMs: 1000 } }
   }
}
```

To start the other node as the reference node, send it the same command with `role: roleReferenceNode`.

To derive the distance from the result of the reference node with `internalDelay: 32906`, `distanceDelay: 15800`, `durationMs: 16`, and `measuredNodeDelay: 34918`, take the duration T = 16 × 10^6 ns. The delays are then t_R = 486.23 ns, t_M = 458.22 ns, and t_L = 1012.66 ns, so the delay of the cable is t_C = (1012.66 - 486.23 - 458.22) / 2 = 34.10 ns. With v = 5 ns/m, the distance is d = 34.10 / 5 = 6.82 m.

4.7.3 Protocol

Messages

fldCmd *Message* `esb.rcp.tpd.fldCmd`

Container for the topology discovery operation.

Fields

- **role** — *enCmdRole*
  Role of the node in the measurement. The measurement needs two nodes with different roles, so the client sends the command to both of them.
- **durationMs** — uint32
  Duration of each delay measurement in milliseconds. A longer duration improves the precision of the measured delays, but makes the whole measurement longer.
- **timeoutMs** — uint32
  Timeout of the measurement in milliseconds. The RCP server waits at most this time for the other node to take part in the measurement. If the other node does not respond in time, the measurement fails with an error status, see *enCmdStatus*.

fldCmdResult *Message* esb.rcp.tpd.fldCmdResult

Container for the topology discovery result.

Fields

- **internalDelay** — uint32
  Internal delay of the node, i.e., the delay that the node itself adds to the signal. The delay between the two nodes includes the internal delays, see *distanceDelay*. It is a count, see the page *Topology Discovery* for the conversion into a time.
- **distanceDelay** — uint32
  Delay between the two nodes, measured over the line. The client derives the distance between the nodes from it, taking the internal delays into account, see *internalDelay* and *measuredNodeDelay*. For the calculation, see the page *Topology Discovery*.
- **durationMs** — uint32
  Duration of each delay measurement in milliseconds, the same as in *fldCmd.durationMs*. The client needs it to convert the delays into times, see the page *Topology Discovery*.
- **measuredNodeDelay** — uint32
  Internal delay of the measured node, as measured by the reference node. Only the reference node reports it. In the result of the measured node, it is 0.
- **status** — *enCmdStatus*
  Status of the measurement. The delays are only valid if it is *statusSuccess*.

Enumerations

enCmdRole *Enumeration* esb.rcp.tpd.enCmdRole

Role of a node in the topology discovery.

- **roleMeasuredNode** — Start topology discovery as the measured node. The measured node waits for the reference node, measures its own internal delay, and then takes part in the measurement of the delay between the two nodes.
- **roleReferenceNode** — Start topology discovery as the reference node. The reference node starts the exchange with the measured node. It measures its own internal delay and that of the measured node, and then the delay between the two nodes.

enCmdStatus *Enumeration* esb.rcp.tpd.enCmdStatus

Topology discovery status.

- **statusSuccess** — Measurement was successful.
- **statusAutoErr** — Error in the automatic sequence of the measurement: the other node did not continue the sequence within the timeout, see *fldCmd.timeoutMs*.
- **statusMeasErr** — Error during the measurement of the delay between the two nodes, e.g., because no signal was received in time.

- **`statusDlyMeasErr`** — Error during the measurement of an internal delay, e.g., because no signal was received in time.

4.8 Core Operations

File: `server/rcp_core.proto`

Package: `esb.rcp.core`

Core operations of the RCP server.

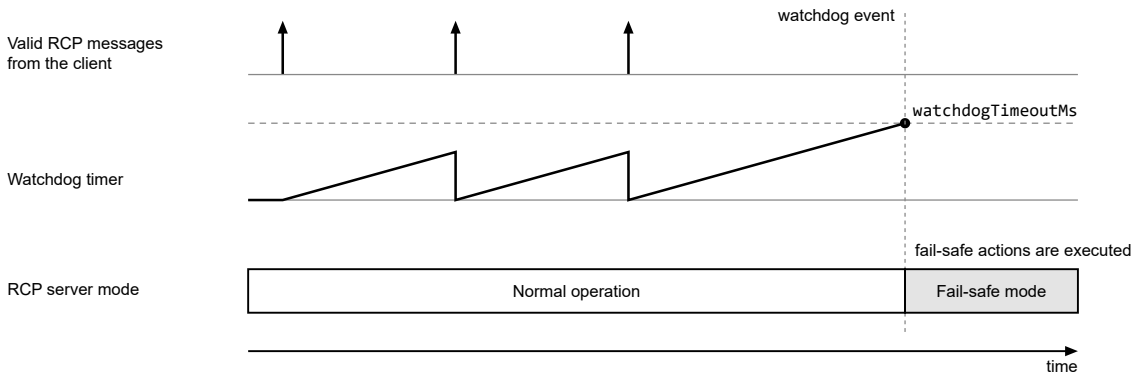
This file defines the core functions of the RCP server: its identification number, the network watchdog with the fail-safe mode, the aggregation of notifications, the scheduler, and the non-volatile memory (NVM). They are configured with *`fldServerConfig`* (see *`rcp.dev.fldConfig`*) and reported by status queries. The page *Higher-level behavior* describes the fail-safe mode and the NVM, and the page *Scheduler* the scheduler.

4.8.1 Server identification

The identification number (DID) of an RCP server and the DID mask of an operation (*`fldOperation.deviceIdMask`*) select the servers that execute an operation of a directive that is broadcast or multicast, see *Messages*. A server executes the operation only if the bitwise AND of its DID and the mask is not zero. All other servers ignore the operation. A server with the DID 0 therefore ignores every operation that has a mask. Each set bit of a DID makes the server a member of the group that the same bit of a mask selects. For example, three servers with the DIDs 0b001, 0b010, and 0b100 are each selected by a mask with their own bit, and the mask 0b101 selects the first and the third server. A server with the DID 0b011 executes every operation whose mask has the bit 0 or the bit 1 set.

4.8.2 Network watchdog

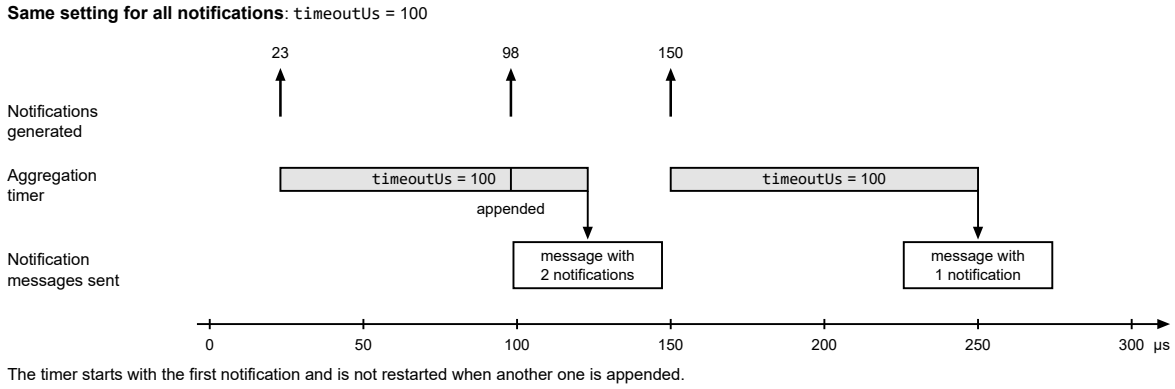
The network watchdog detects the loss of the client. It is disabled by default, and a non-zero *`fldServerConfig.watchdogTimeoutMs`* enables it. While the watchdog is enabled, every valid RCP message restarts the watchdog timer. A client that wants to keep the server out of the fail-safe mode sends a valid RCP message at least once within the timeout. The command *`rcp.dev.opNone`* only restarts the timer, so a client can use it as a keep-alive message when it has nothing else to send. The figure shows the timer:



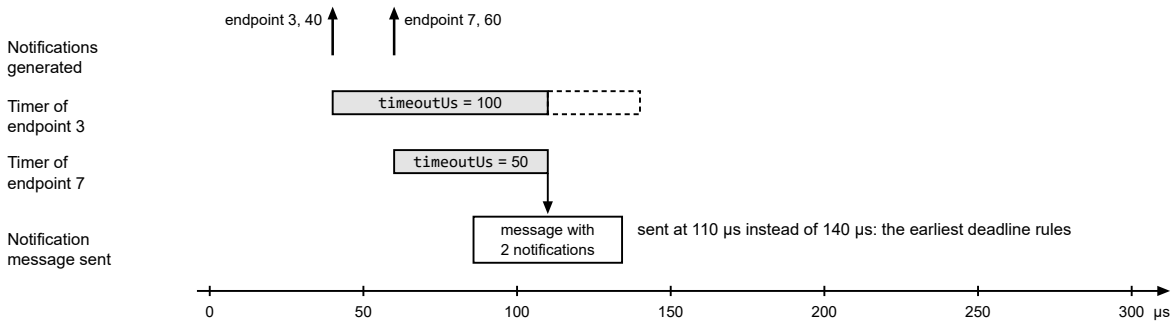
In the figure, each valid RCP message from the client starts the timer again. After the third message, no valid message arrives within the timeout. The timer reaches *`fldServerConfig.watchdogTimeoutMs`* and the watchdog event occurs. The server changes from normal operation to the fail-safe mode and executes the fail-safe actions, see the page *Higher-level behavior*.

4.8.3 Aggregation of notifications

To save bandwidth, the RCP server collects the notifications of the endpoints in unsolicited notification messages (see *`fldNotif`*). Per endpoint, a timeout limits how long the server may wait after the first notification before it sends the message. The timer is not restarted when another notification is appended, and of several endpoints with different timeouts, the earliest deadline applies. A message is also sent when it reaches a configurable number of notifications or bytes. The figure shows an example for one endpoint and an example for two endpoints with different timeouts:



The timer starts with the first notification and is not restarted when another one is appended.



In the first example, the timeout is 100 µs and notifications are generated at 23 µs, 98 µs, and 150 µs. The first notification starts the timer, the second one is appended to the same message, and at 123 µs the server sends the message with two notifications. The third notification starts a new message, which the server sends at 250 µs with one notification.

In the second example, endpoint 3 has a timeout of 100 µs and endpoint 7 one of 50 µs. Endpoint 3 generates a notification at 40 µs and endpoint 7 at 60 µs. The deadline of endpoint 3 would be 140 µs, but that of endpoint 7 is 110 µs, so the server sends one message with both notifications at 110 µs.

4.8.4 Protocol

Messages

fldConfigAggregation *Message* `esb.rcp.core.fldConfigAggregation`

Message for configuring aggregation of notifications.

Fields

- **`hndl`** — `uint32`
  Handle of the endpoint generating notifications.

- **`timeoutUs`** — `uint32`
  Timeout in microseconds for aggregating multiple endpoint notifications into a single unsolicited notification message. The RCP server schedules the message in its MAC TX queue no later than this time after the first notification. Appending another notification does not restart the timer. If 0 (default), the RCP server optimizes for latency and sends the message as soon as the endpoint generates an event. Notifications can still be aggregated while an outgoing message waits in the MAC queue. A longer timeout trades latency for bandwidth. If endpoints have different timeouts, the earliest deadline applies. For examples, see the page *Core Operations*.

  > **Note**
  >
  > The message can be sent earlier if it exceeds the maximum allowed size (buffer full).

fldServerConfig *Message* `esb.rcp.core.fldServerConfig`

Core settings of the RCP server.

Fields

- **`did`** — *optional* `uint32`
  If set, changes the identification number (DID) of the RCP server. The DID selects the servers that execute an operation of a directive that is broadcast or multicast, see *`fldOperation.deviceIdMask`*.

  > **Note**
  >
  > A client can derive the DID from the hardware-unique identifier *`info.fldIdentity.sid`* that the query *`dev.qryIdentity`* returns. Some vendors do not provide a *`info.fldIdentity.sid`*.

- **`watchdogTimeoutMs`** — *optional* `uint32`
  Configures the network watchdog functionality. If set to 0 (default), the network watchdog feature is disabled. If set to a non-zero value, the RCP server expects the client to send RCP messages periodically. If no valid RCP messages are received within the specified number of milliseconds since the last valid RCP message was received, the watchdog event is triggered and the RCP server enters fail-safe mode. If fail-safe operations are defined when the fail-safe mode is entered, they are executed. The client can define these fail-safe operations after booting, or the RCP server can load them from the NVM. The flag *`fldAction.failsafe`* selects fail-safe actions.
- **`agg`** — *repeated* *`fldConfigAggregation`*
  Aggregation configuration for each endpoint generating notifications.
- **`unsolicitedOperationLimit`** — *optional* `uint32`
  Configures the maximum number of notifications allowed within a single notification message. If set to 0 (default), there is no limit to the number of notifications that can be stored in a single notification message. If set to a non-zero value, the RCP server sends an unsolicited notification message as soon as the number of pending notifications reaches this limit.
- **`syncActionsEnable`** — *optional* `bool`
  In some applications, synchronous actions can negatively affect the overall determinism of the system. Setting this option to false forces the scheduler to treat synchronous actions as asynchronous. This means that best-effort actions are scheduled asynchronously, generate an immediate *`actPending`* reply, and run as soon as possible. This option is enabled by default.
- **`aggregationSizeBytes`** — *optional* `uint32`
  Sets the threshold in bytes after which an unsolicited message is scheduled to the MAC TX queue, regardless of how many notifications are still pending. If set to 0

(default) the RCP server implicitly limits the size according to the maximum Ethernet frame size. The RCP server tries to aggregate as many notifications as possible in a single unsolicited message, also based on the *unsolicitedOperationLimit* and the *fldConfigAggregation* options. When *aggregationSizeBytes* is set to a non-zero value, if at any time while aggregating notifications the size of the message exceeds this value, the message is scheduled to be sent. The actual message size can therefore exceed the *aggregationSizeBytes* value, depending on the configured buffer size of the endpoints generating notifications.

fldServerStatus *Message* esb.rcp.core.fldServerStatus

Container to retrieve RCP server status information.

Fields

- **liv** — bool
  If set, the RCP server operates normally, i.e., it is alive. The flag is cleared while the RCP server is in fail-safe mode.
- **pwrOn** — bool
  This flag is set when the RCP server powers on, and can only be cleared by the client. Polling this flag detects unwanted resets of the RCP server. To clear this flag, use the *dev.opClearPwron* command.

fldNvmStatus *Message* esb.rcp.core.fldNvmStatus

Container to retrieve NVM status information.

Fields

- **present** — bool
  NVM is present and ready to be used.
- **hasConfig** — bool
  True if the NVM contains a valid configuration.
- **storedActions** — uint32
  Number of actions stored in the NVM, 0 if loading them failed (see *actionsError*).
- **actionsError** — bool
  True if loading the stored actions failed: a corrupted action (CRC mismatch), an NVM read error, or a missing resource. No stored action is loaded then and *storedActions* is 0. The flag is also set when the scheduler refuses a stored action. The other actions stay loaded then, and the count is kept. It stays set until the stored actions are cleared or successfully loaded again.
- **configError** — bool
  True if the stored configuration could not be loaded (e.g., a CRC mismatch). This is detected on boot and when the configuration is restored. The stored configuration is not applied then (*hasConfig* is false). It stays set until the configuration is saved, cleared, or successfully loaded again.

fldNvmOperation *Message* esb.rcp.core.fldNvmOperation

Message to perform user-NVM read/write operations. This message can be used to perform user-defined read/write accesses into the user-dedicated NVM in the server, if available. The RCP server does not interpret or use the user-dedicated NVM data for any purpose other than making it accessible to the user. Therefore, the format and structure of the user-dedicated NVM are entirely user-defined. The RCP server rejects attempts to read/write past the available user-dedicated NVM.

Fields

- **address** — uint32
  Byte address in user-dedicated NVM.

- **op** — *oneof*
  Selects a read or a write.
  - **rsize** — `uint32`
    Length of data to read.
  - **wdata** — `bytes`
    Data to write to NVM.

fldSchedulerStatus *Message* `esb.rcp.core.fldSchedulerStatus`

Container to retrieve information about the RCP server scheduler.

Fields

- **actions** — *repeated* *`fldSchedulerItem`*
  List of scheduled actions.

fldSchedulerStatus.fldSchedulerItem *Message* `esb.rcp.core.fldSchedulerStatus.fldSchedulerIte`

Single instance of a scheduled action item.

Fields

- **uid** — `uint32`
  Unique identifier of the scheduled action, see *`fldActionResult.uid`*.
- **hndl** — `uint32`
  Endpoint handle the action is associated with.
- **failsafe** — `bool`
  True if the action is a fail-safe mode action.
- **repeatCount** — `uint32`
  Remaining repetitions (0xFFFFFFFF = infinite).
- **seq** — `uint32`
  Last sequence number of the (repeated) action.
- **clientId** — `uint32`
  Client ID of the action.
- **tag** — *optional* `uint32`
  Tag given by the client, if any, see *`fldAction.tag`*.

4.9 Vendor-Specific Features

File: `server/rcp_vendor.proto`

Package: `esb.rcp.vendor`

Vendor-specific configuration and status of the RCP server.

The settings and the status that are specific to the vendor of an RCP server are not part of the common ESB specification, so this file defines only the containers for them, and the vendor adds the fields. The configuration is part of the server configuration (*`rcp.dev.fldConfig.vendor`*). The status is returned by the optional query *`rcp.dev.qryVendorStatus`*. Refer to the datasheet of the server for the fields and their valid values.

4.9.1 Protocol

Messages

fldConfig *Message* `esb.rcp.vendor.fldConfig`

Vendor-specific configuration.

fldStatus *Message* `esb.rcp.vendor.fldStatus`

Vendor-specific status.

Chapter 5
Endpoints

Messages for the endpoints of an RCP server, such as SPI, I2C, GPIO, UART, ADC, or PWM. A client addresses an endpoint by its handle, see *Operating principle*.

Each type of endpoint has a page of its own. It defines the configuration of the endpoint, the commands that an action carries, and, if the endpoint produces data or events by itself, the notifications that it sends. The endpoint types are:

- *GPIO Driver*: general-purpose input/output pins, which a client sets, clears, and reads with masks and which can send notifications about events.
- *SPI Driver*: a serial peripheral interface (SPI) controller that runs bus transactions in the SPI modes 0 to 3.
- *I2C Driver*: an inter-integrated circuit (I2C) controller that builds bus transactions from transfer tokens.
- *UART Driver*: a universal asynchronous receiver-transmitter that sends and receives blocks of data words.
- *LIN Driver*: the commander of a local interconnect network (LIN) bus, which sends the frame headers.
- *PWM Driver*: a generator of a square wave with a configurable frequency and duty cycle.
- *ADC Driver*: an analog-to-digital converter that samples at a fixed rate and delivers the processed values.
- *SENT Driver*: a receiver of single edge nibble transmission (SENT) messages of a sensor.
- *PDM Driver*: a receiver of a pulse-density modulation (PDM) stream, e.g., from a digital microphone.
- *PCM Audio Driver*: a pulse-code modulation (PCM) audio interface that streams digital audio in I2S or time-division multiplexing (TDM) frames.

The *pin configuration* assigns the pins to the endpoints. On a server with a static mapping, the pins are fixed.

5.1 GPIO Driver

File: `endpoints/gpio/gpio_dev.proto`

Package: `esb.gpio`

GPIO configuration, actions, queries, and notifications for RCP.

A GPIO endpoint is a controller for a group of general-purpose input/output (GPIO) pins. Because commands and notifications use 32-bit masks, a controller handles up to 32 pins.

5.1.1 Pins and masks

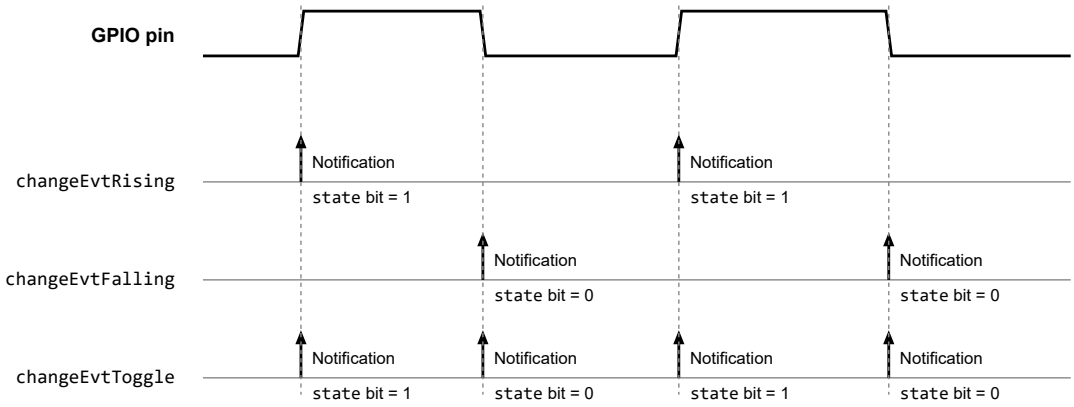
The GPIO number of a pin is its bit position in the masks of commands and notifications: GPIO #0 is the least significant bit (LSb). The pin configuration assigns the GPIO numbers to the physical pins and sets their electrical properties, see *`rcp.pins.fldConfig.fldConfigItem.gpio`*.

A command combines operations on any number of pins: it configures the direction of the pins, reads their state, and drives, clears, or toggles them. A pin is only affected by an operation if its bit is set in the mask of that operation. For example, a *`fldCmd.clr`* mask of 0×09 (binary

00001001) clears GPIO #0 and GPIO #3 at the same time. The states that a command reads are returned in the action result, see *fldActionResult.gpio*.

5.1.2 Pin change events

An input can report its changes to the client without being polled. Its configuration selects the changes that generate an event (see *fldPinConfig.event*). An event generates a notification, unless the pin is configured as quiet (see *fldPinConfig.quiet*), and it can trigger actions, see *fldActionEvent*. The figure shows a pin and the notifications that each event type generates:



The pin needs to be configured with `fldPinConfig.event`. The notification carries the PTP time of the change if PTP is enabled.

A notification carries the state of all pins of the controller and tells which of them changed.

5.1.3 Example

To report the rising edges of GPIO #3 and to drive GPIO #0 high:

```
seq: 1
dir { config { hndl: 1  gpio { pin { num: 3  event:
changeEvtRising  debounceNs: 1000 } } } }
dir { action { hndl: 1  gpio { out: 0x01  set: 0x01 } } }
```

5.1.4 Protocol

Messages

fldPinConfig *Message* `esb.gpio.fldPinConfig`

Configuration message for GPIO pins.

Fields

- **`num`** — `uint32`
  Index of the pin within the controller, starting from 0. It is the GPIO number of the pin, which is its bit position in the masks of *fldCmd* and *fldNotif*.
- **`event`** — *optional* *enChangeEvent*
  Configures which changes of the pin generate an event. An event generates an unsolicited notification, unless *quiet* is set, and it can trigger actions, see *fldActionEvent*. If PTP is configured and enabled, the notification contains the value of the PTP timer at the time the pin changed its state.
- **`quiet`** — *optional* `bool`
  If set, suppresses notification reports for this pin. The change event can still be used to trigger other actions.

- **debounceNs** — *optional* `uint32`
  Debounce time in nanoseconds.

fldConfig *Message* `esb.gpio.fldConfig`

GPIO configuration message.

Fields

- **pin** — *repeated* *`fldPinConfig`*
  Configuration for each GPIO pin.

fldCmd *Message* `esb.gpio.fldCmd`

Container for GPIO commands. Each field is a mask with one bit per GPIO. A set bit applies the operation of the field to the GPIO whose number equals the bit position, starting with bit 0 (LSb). For an example, see the page *GPIO Driver*.

Fields

- **out** — `uint32`
  Mask of pins to (re)configure as output.
- **inp** — `uint32`
  Mask of pins to (re)configure as input.
- **get** — `uint32`
  Mask of pins to get state for.
- **set** — `uint32`
  Mask of pins to set state for.
- **clr** — `uint32`
  Mask of pins to clear state for.
- **tgl** — `uint32`
  Mask of pins to toggle state for.

fldNotif *Message* `esb.gpio.fldNotif`

GPIO change notification.

Fields

- **state** — `uint32`
  Current state of all GPIO pins for this controller (endpoint). The LSb represents the status of GPIO #0 within this controller.
- **change** — `uint32`
  Mask of pins that changed state. A bit set in the mask at position N (starting from LSb) indicates that the GPIO #N changed state since the last notification.

Enumerations

enChangeEvent *Enumeration* `esb.gpio.enChangeEvent`

Pin change that generates an event.

- **changeEvtNone** — No event (default).
- **changeEvtRising** — Generate an event when the pin transitions from 0 to 1.
- **changeEvtFalling** — Generate an event when the pin transitions from 1 to 0.
- **changeEvtToggle** — Generate an event any time the pin changes state.

5.2 SPI Driver

File: `endpoints/spi/spi_dev.proto`

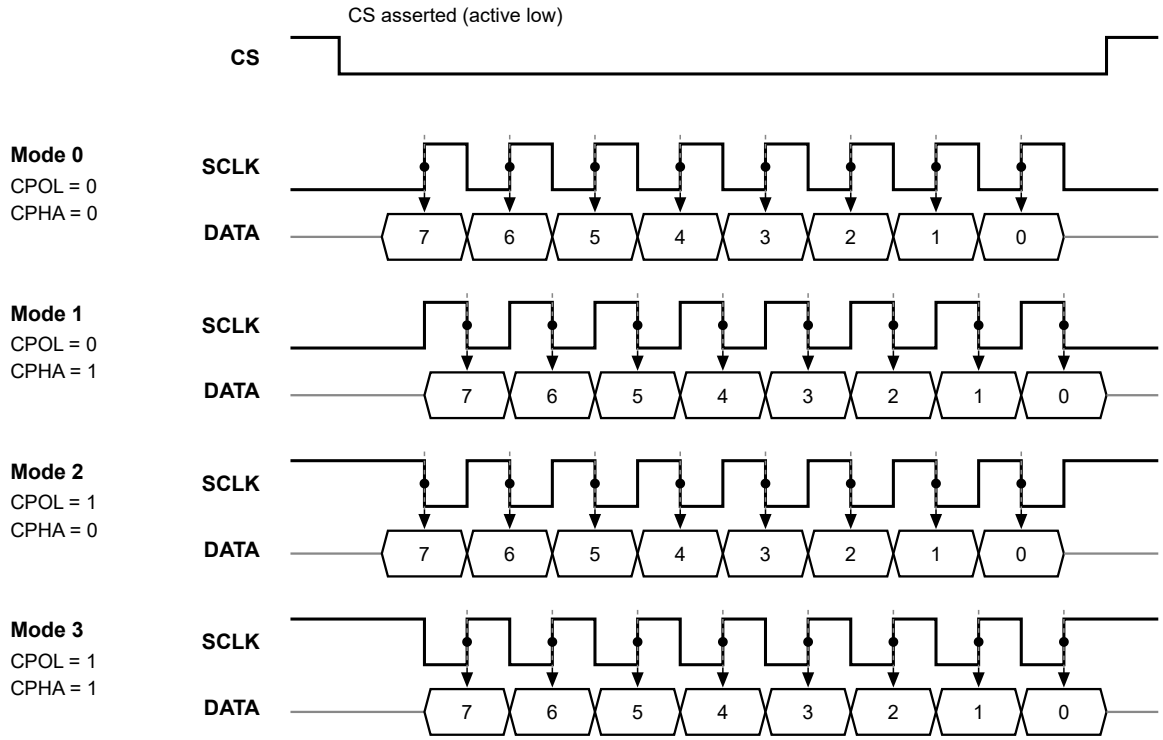
Package: `esb.spi`

SPI configuration, actions, queries, and notifications for RCP.

An SPI endpoint is a serial peripheral interface (SPI) controller, also called the “master”. It drives the clock SCLK and the chip-select CS, shifts data out on MOSI, and samples MISO. The client first configures the bus with a *`fldConfig`* message and then performs bus transactions with *`fldCmd`* commands. The pins SCLK, MOSI, MISO, and CS are assigned to the endpoint with the pin configuration, see *`rcp.pins.enPinFuncSpi`*.

5.2.1 Clock polarity and phase

The clock polarity and the clock phase select one of the four SPI modes. The polarity is the idle level of SCLK. The phase decides which clock edge samples the data: the leading edge (the first edge after the idle level) or the trailing edge (the second edge of each clock period). The transmitter changes its data on the other edge. The figure shows the four modes with the most significant bit (MSb) first. The circles mark the sampling edges.



- Sampling edge: the receiver takes over the DATA value here. The bits are numbered for `msbFirst` set.

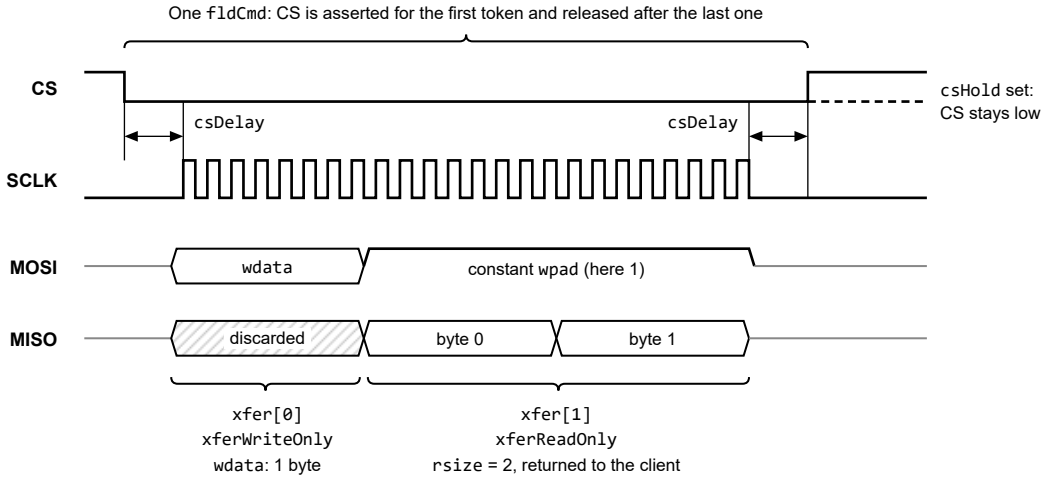
  Leading edge: first clock edge after the idle level. Trailing edge: second clock edge of each period.

5.2.2 Transactions

Each command is one SPI transaction. The chip-select is asserted before the first transfer token and released after the last one, unless the command keeps it asserted so that a following command can continue the transaction. The tokens are executed in order. A token writes bytes, reads bytes, or does both at the same time. The bytes that all tokens return are concatenated, in the order of the tokens, in the reply to the command.

The figure shows a command with two tokens: a write-only token that sends one byte, followed by a read-only token that reads two bytes while MOSI stays at a constant level. SCLK runs

continuously from the first to the last bit. CS is asserted a configurable delay before the first clock edge and released the same delay after the last one. The hardware can add a minimum delay on top of this. Refer to the datasheet of the server.



Shown for CPOL = 0, CPHA = 0, MSb first.

A command uses the configuration of the endpoint. It can override some of the parameters for itself, e.g., to use a lower clock speed for one device.

5.2.3 Example

To read a two-byte identifier from a device with the command byte 0×9F in SPI mode 0, at 1 MHz:

```
seq: 1
dir {
   config { hndl: 2  spi { cpol: false  cpha: false  msbFirst:
true  speedHz: 1000000 } }
}
dir {
   action {
      hndl: 2
      spi {
         xfer { xft: xferWriteOnly  wdata: "\x9F" }
         xfer { xft: xferReadOnly   rsize: 2 }
      }
   }
}
```

The identifier is returned in *fldActionResult.data*.

5.2.4 Protocol

Messages

fldCmdXfer *Message* `esb.spi.fldCmdXfer`

SPI transfer token, building block to perform I/O operations. Three transfer types are supported:

- full-duplex: a number of bytes (*wdata*) is transferred via MOSI and the same number of bytes, sampled from MISO, is returned.
- write-only: as full-duplex, but the data from MISO is discarded and no buffer is returned to the client.

- read-only: no data is transferred on MOSI, which is set to the constant value *wpad* during the transfer. Instead, *rsize* bytes are sampled from MISO and returned to the client.

Fields

- **xft** — *enXferType*
  Selects the transfer type (full-duplex, write-only, read-only).
- **wdata** — bytes
  When *xft* is set to *xferFullDuplex* or *xferWriteOnly*, the size of the transfer is set by the length of *wdata*, which contains the data to be sent over the MOSI pin.
- **rsize** — uint32
  When *xft* is set to *xferReadOnly*, *rsize* determines the transfer length in bytes. This amount of data is appended to the data of the action result (*fldActionResult.data*).
- **wpad** — bool
  When *xft* is set to *xferReadOnly*, the MOSI pin is set according to the value of *wpad* (False = 0, True = 1) for the duration of the transfer.

fldConfig *Message* esb.spi.fldConfig

SPI configuration message.

Fields

- **msbFirst** — *optional* bool
  If set, configures whether the data is sent/received MSB or LSB first.
- **cpol** — *optional* bool
  If set, configures the clock polarity.
- **cpha** — *optional* bool
  If set, configures the clock/data phase.
- **cs** — *optional* uint32
  If set, configures the default pin to be used as the chip-select. The number is the unique pin identifier as reported by *rcp.dev.qryPins*.
- **csDelay** — *optional* uint32
  CS to SCLK delay in half clock periods. The same delay applies to both sides of the transfer. This is in addition to the minimum delay.
- **speedHz** — *optional* uint32
  The clock speed in hertz (Hz). If the requested speed is fully supported by the RCP server, the configuration is accepted with the *cfgSuccess* return code. The RCP server is allowed to round the requested speed to the nearest supported speed within a reasonable tolerance (refer to the datasheet of the server). In this case, the configuration is accepted with the *cfgParamAdjusted* warning code and the client can retrieve the actual (rounded) speed by querying the configuration of the endpoint.

fldCmd *Message* esb.spi.fldCmd

Container for SPI commands.

Fields

- **param** — *optional* *fldConfig*
  If set, overrides the specified configuration fields for this command only.
- **xfer** — *repeated* *fldCmdXfer*
  List of transfer tokens. The chip-select is asserted on the first transfer token, and it is de-asserted after all tokens have been processed.
- **csHold** — bool
  Keep CS low after the transaction.

Enumerations

enXferType *Enumeration* `esb.spi.enXferType`

SPI transfer type.

- **`xferFullDuplex`** — Perform R/W concurrently.
- **`xferWriteOnly`** — Just write, discard data from MISO.
- **`xferReadOnly`** — Just read, set MOSI to the PAD value.

5.3 I2C Driver

File: `endpoints/i2c/i2c_dev.proto`

Package: `esb.i2c`

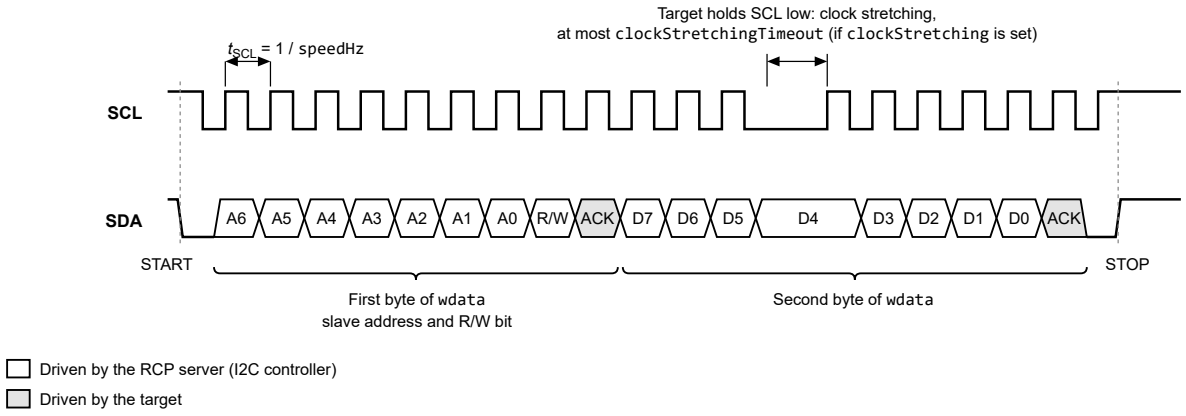
I2C configuration, actions, queries, and notifications for RCP.

An I2C endpoint is an inter-integrated circuit (I2C) controller. It drives the clock SCL and the bidirectional data line SDA of a bus on which one or more attached devices, called targets, respond. The pins are assigned to the endpoint with the pin configuration, see *`rcp.pins.enPinFuncI2c`*.

The endpoint is a low-level interface: it generates the bus conditions and moves bytes, but it does not know the protocol of the attached targets. The client builds each bus transaction from transfer tokens, including the target address and the R/W bit that start it.

5.3.1 Bit timing

The figure shows a write transaction of two bytes. The first byte holds the 7-bit target address and the R/W bit. After each byte the target answers with an acknowledge (ACK). A target that needs more time can hold SCL low, which is called clock stretching. In the example, the target stretches the clock during one bit.

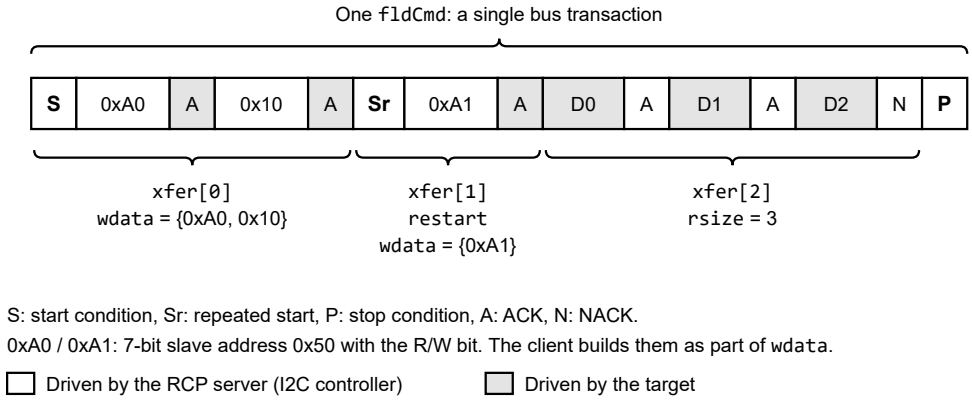


The clock stretching timeout (*`fldConfig.clockStretchingTimeout`*) limits how long the RCP server waits for a target to release SCL before it aborts the current transfer.

5.3.2 Transactions

Each command is one transaction: a start condition, the transfer tokens in order, and a stop condition. A token either writes bytes or reads a number of bytes. A token can be preceded by a repeated start condition, which allows a transaction to change the direction without releasing the bus. The bytes that all read tokens return are collected, in order, in the reply to the command.

The figure shows a typical register read of an EEPROM at the 7-bit address 0×50: the first token writes the address byte (with the R/W bit cleared) and the register address, the second token restarts and sends the address byte again with the R/W bit set, and the third token reads three bytes.



5.3.3 Example

The register read of the figure:

```
seq: 1
dir {
   action {
      hndl: 3
      i2c {
         xfer { wdata: "\xA0\x10" }
         xfer { restart: true  wdata: "\xA1" }
         xfer { rsize: 3 }
      }
   }
}
```

5.3.4 Protocol

Messages

fldConfig *Message* `esb.i2c.fldConfig`

I2C bus configuration.

Fields

- **`speedHz`** — *optional* `uint32`
  The clock speed in hertz (Hz). If the requested speed is fully supported by the RCP server, the configuration is accepted with the *`cfgSuccess`* return code. The RCP server is allowed to round the requested speed to the nearest supported speed within a reasonable tolerance (refer to the datasheet of the server). In this case, the configuration is accepted with the *`cfgParamAdjusted`* warning code and the client can retrieve the actual (rounded) speed by querying the configuration of the endpoint.
- **`clockStretchingTimeout`** — *optional* `uint32`
  If set, configures the timeout for clock stretching: the longest time that the RCP server waits for a target to release SCL before it aborts the current transfer. The timeout is given in quarter periods of the SCL clock.

- **`clockStretching`** — *optional* `bool`
  If set, enables clock stretching on the bus.

fldCmdXfer *Message* `esb.i2c.fldCmdXfer`

I2C transfer token, the building block of an I2C command. A token is either a write or a read transfer.

> **Note**
>
> This is a low-level I2C interface that gives the client direct control of the bus. The client implements the I2C protocol of the attached device(s), including the generation of the target address and the R/W bit.

Fields

- **`xft`** — *oneof*
  - **`wdata`** — `bytes`
    If set, the I2C controller performs a write transfer of the specified amount of data (size of *`wdata`*). The controller expects an ACK from the attached device for each byte transferred. Otherwise, the command fails with the *`actIOError`* code.
  - **`rsize`** — `uint32`
    Determines the transfer length in bytes. This amount of data is appended to the data of the action result (*`fldActionResult.data`*).
- **`restart`** — `bool`
  If set, generates a repeated start condition before the current transfer occurs.

fldCmd *Message* `esb.i2c.fldCmd`

Container for I2C commands.

Fields

- **`speedHz`** — *optional* `uint32`
  If set, overrides the clock speed in hertz (Hz) for this command only. The RCP server is allowed to round the requested speed, as described for *`fldConfig.speedHz`*.
- **`xfer`** — *repeated* *`fldCmdXfer`*
  List of transfer tokens.

5.4 PWM Driver

File: `endpoints/pwm/pwm_dev.proto`

Package: `esb.pwm`

PWM configuration and actions for RCP.

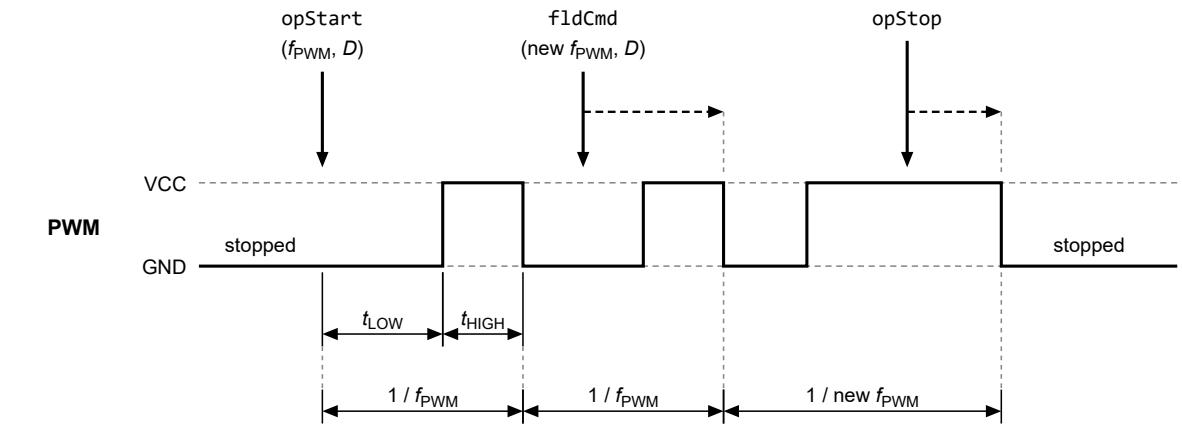
A PWM endpoint generates a periodic square wave with a configurable frequency and duty cycle.

After configuration, the endpoint is stopped and its output pin is low. A *`fldCmd`* message with *`fldCmd.op`* set to *`opStart`* starts PWM. The command can set a frequency f_{PWM} and a duty cycle D at the same time ($0 \le D \le 1$). When the endpoint processes the start instruction, it first emits a low pulse lasting $t_{LOW} = (1 - D) / f_{PWM}$, then a high pulse lasting $t_{HIGH} = D / f_{PWM}$.

A new *`fldCmd`* message can change the PWM frequency and the duty cycle at any time. To avoid glitches, the endpoint applies the new values at the end of a full cycle. Similarly, *`fldCmd.op`* = *`opStop`* pulls the output pin low only after the active low-high cycle has completed.

The figure shows the output pin when PWM is started, when the frequency and the duty cycle are changed, and when PWM is stopped:

PWM output: each cycle is a low pulse followed by a high pulse



$t_{LOW} = (1 - D) / f_{PWM}$ and $t_{HIGH} = D / f_{PWM}$, with $0 \le D \le 1$.

A new `fldCmd` or `opStop` waits (dashed arrow) for the end of the running cycle.

5.4.1 Protocol

Messages

fldConfig *Message* `esb.pwm.fldConfig`

Fields

- **`speedHz`** — *optional* `uint32`
  The clock speed in hertz (Hz). If the requested speed is fully supported by the RCP server, the configuration is accepted with the *`cfgSuccess`* return code. The RCP server is allowed to round the requested speed to the nearest supported speed within a reasonable tolerance (refer to the datasheet of the server). In this case, the configuration is accepted with the *`cfgParamAdjusted`* warning code and the client can retrieve the actual (rounded) speed by querying the configuration of the endpoint.
- **`duty`** — *optional* `fixed32`
  32-bit left-aligned duty cycle value X:
  - `0`: output stuck to GND.
  - `0xFFFFFFFF`: output stuck to VCC.
  - `0x80000000`: 50% (default).
  - Otherwise: duty cycle = X / 2^{32}.

fldCmd *Message* `esb.pwm.fldCmd`

Container for PWM commands.

Fields

- **`op`** — *optional* *`enCmdOperation`*
  PWM operation.
- **`duty`** — *oneof*
  - **`dutySet`** — `fixed32`
    32-bit left-aligned duty cycle value, same as *`fldConfig.duty`*.
  - **`dutyAdj`** — `sint32`
    Adjust the duty cycle by this amount. Adjustments can be negative. If the result would be invalid, i.e., exceeding the boundaries of a 32-bit unsigned integer, the duty cycle is clipped.

- **`freq`** — *oneof*
  - **`freqSet`** — `uint32`
    Set frequency in hertz (Hz), same as *`fldConfig.speedHz`*.
  - **`freqAdj`** — `sint32`
    Adjust frequency by this amount in hertz (Hz), can be negative. Under-and overflows are handled as with *`dutyAdj`*.

Enumerations

enCmdOperation *Enumeration* `esb.pwm.enCmdOperation`

PWM operation types.

- **`opStart`** — Start PWM.
- **`opStop`** — Stop PWM.

5.5 UART Driver

File: `endpoints/uart/uart_dev.proto`

Package: `esb.uart`

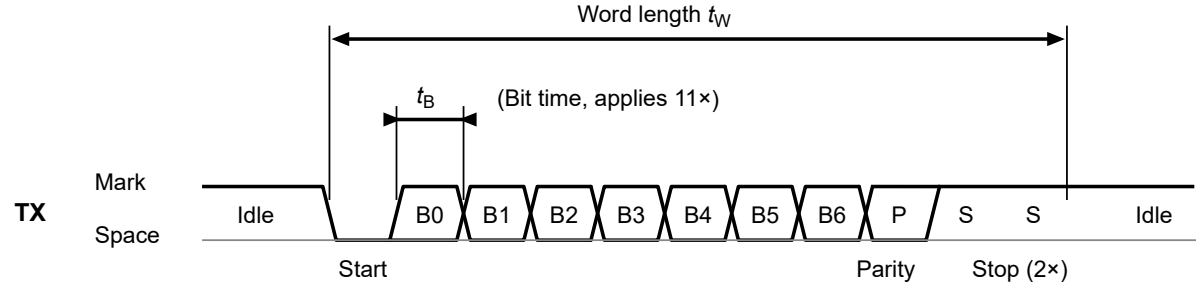
UART configuration, actions, and notifications for RCP.

A UART endpoint is a universal asynchronous receiver-transmitter (UART). It sends and receives blocks of data words. The transmitter and the receiver work independently, but both use the same line configuration. The client configures the endpoint with a *`fldConfig`* message before it uses it. The pins TX and RX and, for hardware flow control, CTS and RTS are assigned to the endpoint with the pin configuration, see *`rcp.pins.enPinFuncUart`*.

5.5.1 Transmission timing

The UART sends blocks of words on the pin TX. By default, the idle level is high (called “mark”), and start bits and breaks are low (called “space”). Setting *`fldConfig.txPolInv`* inverts the polarity of the pin, including the data bits.

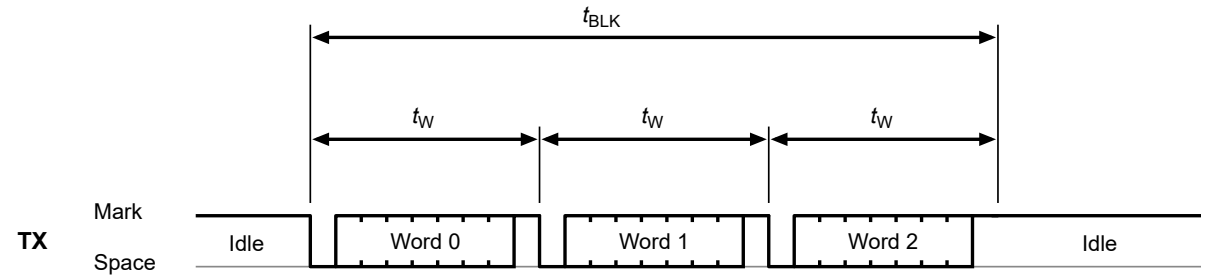
A UART word consists of a start bit, data bits, an optional parity bit, and stop bits. The least significant bit is sent first. The parameters *`fldConfig.speedBps`*, *`fldConfig.numDataBits`*, *`fldConfig.parityBit`*, and *`fldConfig.numStopBits`* configure how the word is structured in time. The figure shows a word with seven data bits, even parity, and two stop bits.



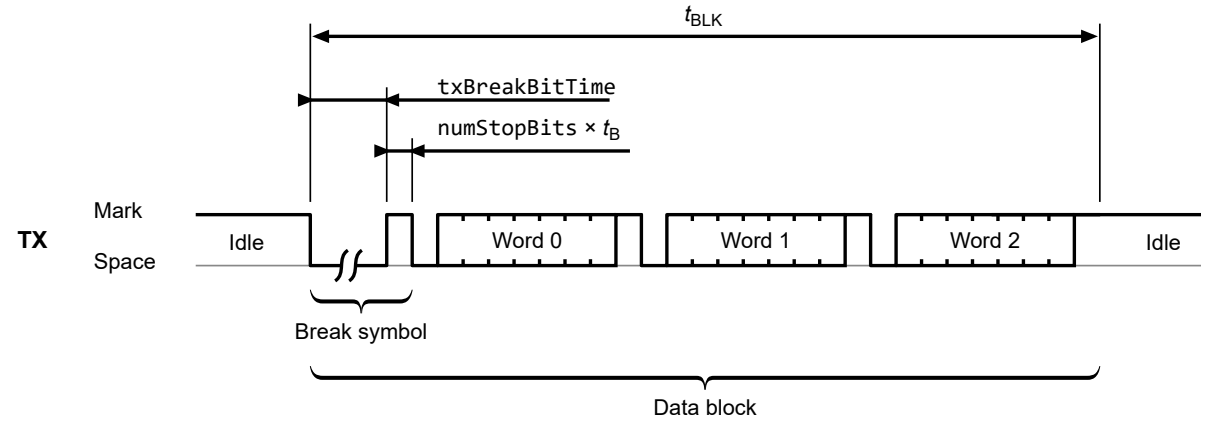
The bit time is t_B = 1 / *`fldConfig.speedBps`*. The word time t_W is the time from the start of the start bit to the end of the last stop bit:

t_W = (1 + *`fldConfig.numDataBits`* + {1 if *`fldConfig.parityBit`* is not *`parityNone`*} + *`fldConfig.numStopBits`*) × t_B

The UART sends one block of data words for each *`fldCmd`* message. In the simplest case, the command contains only the data to send, and the UART concatenates all words without a pause between them. The block time t_{BLK} is then Length(*`fldCmd.txData`*) × t_W:

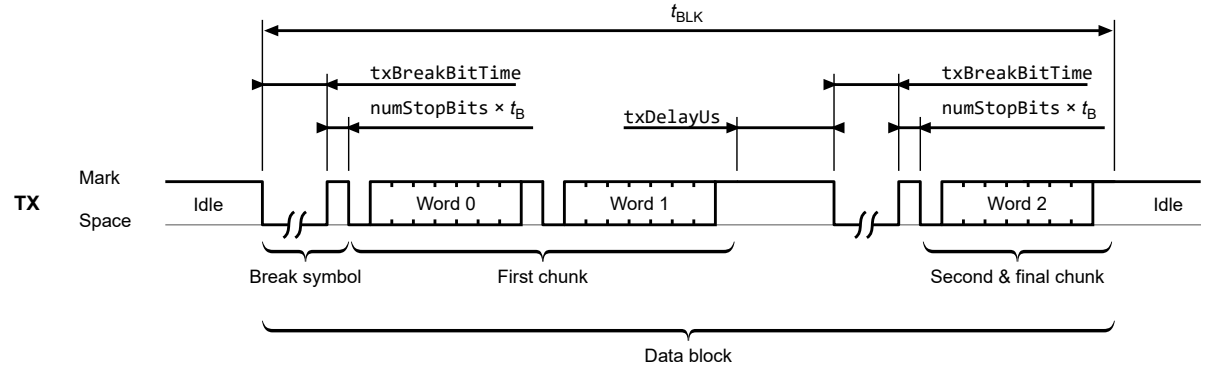


The client can set *fldCmd.txBreak* to send a break before the data. A break is a space that lasts *fldConfig.txBreakBitTime* bit times. The client should choose a break that is longer than the longest space of normal data, so that a UART receiver can tell the break from an all-zero word. A mark follows the break and separates it from the start bit of the first word, which is a space as well. The mark lasts as long as the stop bits of a word, i.e., *fldConfig.numStopBits* bit times. The figure shows three words with a break:



The block time is now t_{BLK} = t_B × (*fldConfig.txBreakBitTime* + *fldConfig.numStopBits*) + Length(*fldCmd.txData*) × t_W.

fldCmd.chunkSize splits a block into smaller "chunks". Each chunk contains that many words, except for the last one, which is smaller if the block size is not a multiple of the chunk size. With the default chunk size of 0, a block consists of one chunk. The UART concatenates the words of a chunk as before, adds the pause *fldConfig.txDelayUs* between the chunks, and sends the break, if commanded, before each chunk:



In the general case, the block time is

t_{BLK} = t_B × (*fldConfig.txBreakBitTime* + *fldConfig.numStopBits*) × N_{Ch} + *fldConfig.txDelayUs* / 10^6 × (N_{Ch} - 1) + Length(*fldCmd.txData*) × t_W

where N_{Ch} is the number of chunks in the block: N_{Ch} = ⌈ Length(*fldCmd.txData*) / *fldCmd.chunkSize* ⌉, or 1 if *fldCmd.chunkSize* is 0. Without *fldCmd.txBreak*, the first term is omitted.

5.5.2 Reception

The UART receives on the pin RX while *fldConfig.rxEnable* is set. The receiver uses the same nominal timing as the transmitter. Details such as the sampling moment, oversampling, and resynchronization are implementation-specific (refer to the datasheet of the server).

The receiver validates each word. A word without the expected stop bits is a framing error, and a word with a wrong parity bit is a parity error. The receiver reports both with a notification (criterion 2 below).

To save bandwidth between the client and the RCP server, the receiver buffers the received words before it sends them. It sends a *fldNotif* notification to the client as soon as one of six criteria is met. *fldNotif.data* holds the accumulated words from the buffer unless stated otherwise. The client should evaluate *fldNotif.code* to process the notification correctly.

The receiver checks the criteria in this order:

1. The endpoint could not process the received data quickly enough and lost data. This can happen when the link between the client and the RCP server is overloaded and notifications cannot be sent as fast as the UART receives words. *fldNotif.code* is *ntfBufferOverrun*. *fldNotif.data* can contain the words received after the data loss. If the word received after the loss is corrupted, e.g., by a parity error, *fldNotif.data* is empty and another notification follows immediately, according to criterion 2. Likewise, if a break follows the overrun immediately, a second notification with empty data and the code *ntfFlushed* follows immediately.
2. The receiver detects a parity error or a framing error. *fldNotif.code* is *ntfCsumError* or *ntfFrameError*, and the last byte of *fldNotif.data* is the erroneous word, which is likely corrupted.
3. The receiver detects a break. *fldNotif.code* is *ntfFlushed*, and the last byte of *fldNotif.data* is the last word received before the break. *fldNotif.data* is empty if the receiver got no word before the break. The minimum duration of a space that the receiver detects as a break is implementation-specific (refer to the datasheet of the server).
4. The number of words in the buffer equals or exceeds *fldConfig.rxLength*. This criterion is met intrinsically if the buffer is full. *fldNotif.code* is *ntfSuccess*.
5. The last word in the buffer was received more than *fldConfig.rxTimeoutBitTime* bit times ago, and *fldConfig.rxTimeoutEnable* is set. *fldNotif.code* is *ntfSuccess*.
6. The last received word equals *fldConfig.rxTerminalByte*, and *fldConfig.rxTerminalByteEnable* is set. *fldNotif.code* is *ntfSuccess*, and the last byte of *fldNotif.data* is the terminal byte.

5.5.3 Reception filtering

The receiver can filter the received data, so that the client receives only fragments of a data stream.

By default, the receiver collects all received words. Setting *fldConfig.rxStartByteEnable* makes the receiver ignore incoming data until it correctly receives the start byte *fldConfig.rxStartByte*. The receiver puts the start byte into the buffer and collects the data again until it receives the terminal byte *fldConfig.rxTerminalByte*. After that, it ignores incoming data again until the next start byte.

The receiver treats a start byte that arrives while it is collecting like any other word. It keeps ignoring data when a terminal byte arrives while it is ignoring.

The notifications include the start byte and the terminal byte that change the filtering state. As a consequence, after the client sets *fldConfig.rxStartByteEnable*, the first received byte is always the start byte. After a terminal byte, the next received byte is always a start byte.

Filtering affects only the collection of valid data and not the notification criteria above:

- A break that arrives while the receiver is ignoring data still causes a notification with the code *ntfFlushed*. *fldNotif.data* is empty, because the receiver has collected no words (criterion 3). A break that arrives while the receiver is collecting causes a separate notification as well.
- Parity and framing errors are always reported to the client. While the receiver is collecting, the behavior is the same as without filtering, and all bytes in *fldNotif.data* are valid except the last. While the receiver is ignoring, *fldNotif.data* contains only the erroneous byte.
- When the receiver starts ignoring data, it always sends a notification, and the last byte is *fldConfig.rxTerminalByte* (criterion 6).

Only correctly received words change the filtering state. Errors have two consequences:

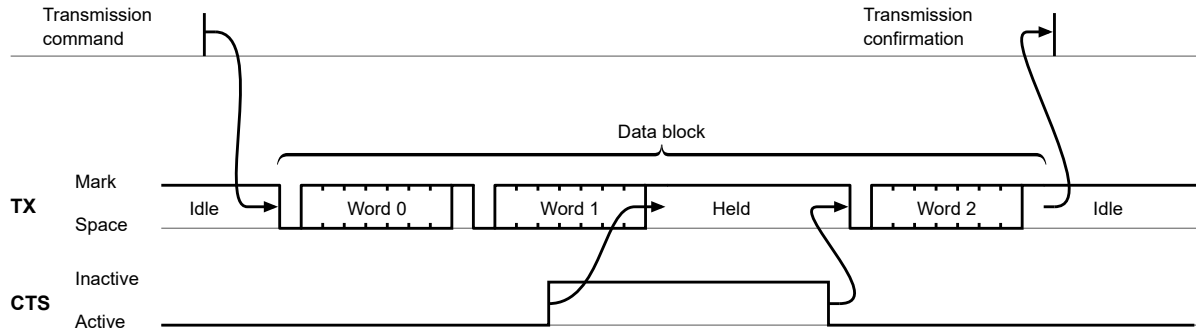
- A start byte with a framing or parity error does not start the collection, and the client misses the fragment.
- A terminal byte with an error does not stop the collection, and the client keeps receiving notifications that it would rather ignore.

5.5.4 Flow control

The UART supports hardware and software flow control, which can be enabled together.

Setting *fldConfig.hwFlowControl* enables hardware flow control. The signal CTS needs an input pin and the signal RTS an output pin.

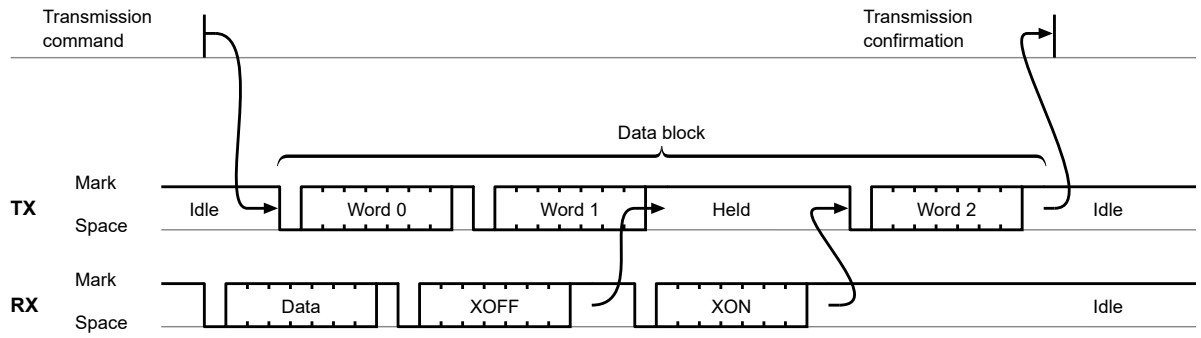
While CTS is active, the transmitter may send data. Otherwise, it holds the pending data. The transmitter stops per word, not per block, so unfinished commands stay pending as long as CTS is inactive:



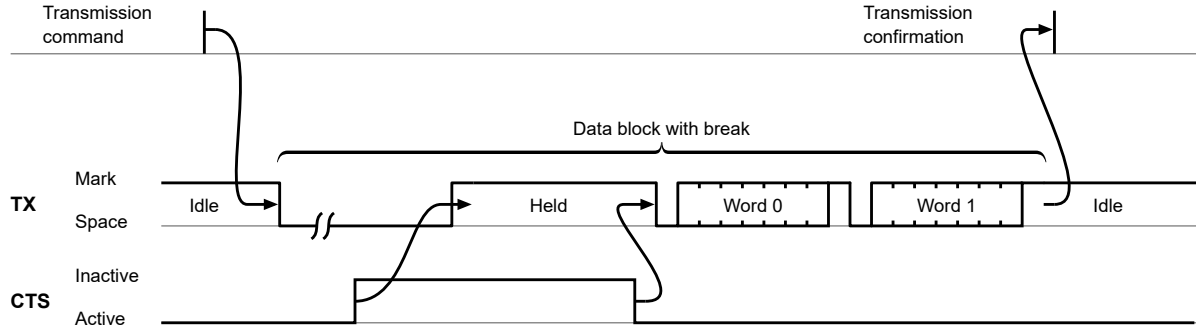
The receiver signals through RTS whether it can accept further words. RTS is active after initialization. It becomes inactive when the endpoint cannot guarantee that further incoming words can be processed without data loss. If RTS becomes inactive while a word is being received, the receiver still accepts that word without data loss.

By default, CTS and RTS are active low. *fldConfig.hwFlowControlPolInv* inverts their polarity.

Software flow control throttles the transmitter only. *fldConfig.swFlowControl* enables it. It needs no additional hardware signal and relies on in-band signaling in the data that the UART receives. When the incoming data stream contains an “XOFF” character (0×13, 19), the transmitter finishes the ongoing word, if any, and stops until it receives an “XON” character (0×11, 17):



For both kinds of flow control, the transmitter handles a break like a data word. If the transmission is suspended while a break is being sent, the transmitter completes the break and waits before the first data word:



5.5.5 Example

To configure the UART for 115200 bps, 8 data bits, no parity, and one stop bit, to receive lines that end with a line feed, and to send a command line:

```
seq: 1
dir {
   config {
      hndl: 7
      uart {
         speedBps: 115200  numDataBits: 8  numStopBits: 1
parityBit: parityNone
         rxEnable: true  rxTerminalByteEnable: true
rxTerminalByte: 10
      }
   }
}
dir { action { hndl: 7  uart { txData: "AT\r\n" } } }
```

The receiver collects words until the terminal byte 0×0A (line feed) and then sends a notification with *fldNotif.code* set to *ntfSuccess* and the received line in *fldNotif.data*.

5.5.6 Protocol

Messages

fldConfig *Message* `esb.uart.fldConfig`

Message sent by the client to configure the UART. The line parameters *speedBps*, *numDataBits*, *numStopBits*, and *parityBit* apply to both the receiver and the transmitter. The parameters with the prefix `tx` affect only the transmitter, and those with the prefix `rx` affect only

the receiver. The flow control parameters are *swFlowControl*, *hwFlowControl*, and *hwFlowControlPolInv*.

Fields

- **speedBps** — *optional* uint32
  The bit rate in bits per second (bps). If the requested speed is fully supported by the RCP server, the configuration is accepted with the *cfgSuccess* return code. The RCP server is allowed to round the requested speed to the nearest supported speed within a reasonable tolerance (refer to the datasheet of the server). In this case, the configuration is accepted with the *cfgParamAdjusted* warning code and the client can retrieve the actual (rounded) speed by querying the configuration of the endpoint.
- **numDataBits** — *optional* uint32
  The number of data bits per word (e.g., 8 in 8N1).
- **numStopBits** — *optional* uint32
  The number of stop bits (e.g., 1 in 8N1).
- **parityBit** — *optional* *enParityBit*
  The type of the parity bit (e.g., N for none in 8N1).
- **txPolInv** — *optional* bool
  When false, the TX pin is active low. When true, the TX pin is active high.
- **rxPolInv** — *optional* bool
  When false, the RX pin is active low. When true, the RX pin is active high.
- **txDelayUs** — *optional* uint32
  The minimum pause in microseconds between two chunks of a block (see *fldCmd.chunkSize*).
- **swFlowControl** — *optional* bool
  If true, enables software flow control (XON/XOFF).
- **hwFlowControl** — *optional* bool
  If true, enables hardware flow control (RTS/CTS).
- **hwFlowControlPolInv** — *optional* bool
  If true, inverts the polarity of the hardware flow control signals (RTS/CTS).
- **txBreakBitTime** — *optional* uint32
  The duration of a transmitted break in bit times.
- **rxEnable** — *optional* bool
  Enables the UART receiver. Received-data notifications are sent only when it is enabled.
- **rxLength** — *optional* uint32
  The number of received words that triggers a notification (length-triggered mode). 0 selects the size of the UART buffer. The configuration fails with the *cfgInvalidParam* error code if the length exceeds the size of the UART buffer.
- **rxTimeoutEnable** — *optional* bool
  Enables the time-triggered mode.
- **rxTimeoutBitTime** — *optional* uint32
  The timeout of the time-triggered mode in bit times.
- **rxTerminalByteEnable** — *optional* bool
  Enables the terminator-triggered mode.
- **rxTerminalByte** — *optional* uint32
  The terminal byte (e.g., 10 for the line feed `'\n'`).
- **rxStartByteEnable** — *optional* bool
  Enables the start-byte-triggered mode. The mode also needs *rxTerminalByteEnable*.
- **rxStartByte** — *optional* uint32
  The start byte (e.g., 36 for `'$'`).
- **rxSuppressLoopback** — *optional* bool
  If true, the UART discards one received word for each word it transmits, without checking whether the timing or the data content of the received word is identical. On half-duplex links, this prevents the transmitted data from being looped back to the client.

> **Note**
>
> Enabling this makes it impossible to detect transmission errors, e.g., due to collisions.

fldCmd *Message* `esb.uart.fldCmd`

Message sent by the client to transmit data over the UART.

Fields

- **`txData`** — `bytes`
  Bytes to send. At least 1 byte is required.
- **`txBreak`** — `bool`
  If true, the UART sends a break before each chunk of data (see *`chunkSize`*). With the default *`chunkSize`*, it sends a single break.
- **`chunkSize`** — `uint32`
  If non-zero, the UART sends the data in chunks of this size, with the pause *`fldConfig.txDelayUs`* between the chunks. If 0 (default), it sends the data in one chunk.

  > **Note**
  >
  > The maximum chunk size depends on the server (refer to the datasheet of the server).

Enumerations

enParityBit *Enumeration* `esb.uart.enParityBit`

Parity bit of a UART word.

- **`parityNone`** — No parity bit.
- **`parityEven`** — Even parity.
- **`parityOdd`** — Odd parity.
- **`parityMark`** — Mark parity: the parity bit is always 1.
- **`paritySpace`** — Space parity: the parity bit is always 0.

5.6 PCM Audio Driver

File: `endpoints/pcm/pcm_dev.proto`

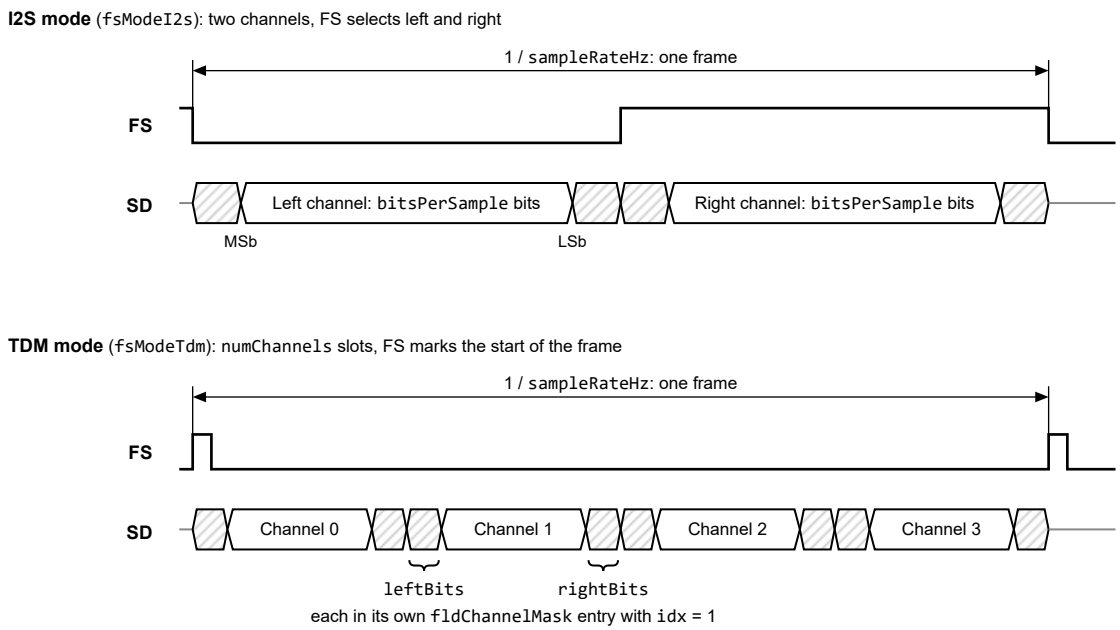
Package: `esb.pcm`

PCM audio interface configuration, actions, queries, and notifications for RCP.

A PCM audio endpoint streams digital audio as pulse-code modulation (PCM) samples: either to a converter or codec (output) or from a source such as a microphone (input). The frame format is either I2S, with two channels, or time-division multiplexing (TDM), with several channels. The endpoint uses the pins SCK (bit clock), SD (serial data), and WS (word select, also called frame sync or FS), see *`rcp.pins.enPinFuncPcm`*. Before streaming, the client configures the endpoint with a *`fldConfig`* message.

5.6.1 Frame formats

The audio is organized in frames. One frame carries one sample of every channel, so the frame rate is the sample rate. In I2S mode, a frame has two channels, left and right, and the level of FS tells which of them is on SD. In TDM mode, a frame has several channels, up to the maximum that the server supports, which follow each other, and FS marks the start of the frame.



The alignment of FS with the first data bit, the polarity of FS and of the bit clock, and the bit order of the samples are configurable. A channel slot can be longer than the sample. The padding bits are not part of the audio and can be ignored to save bandwidth.

5.6.2 Streaming

The client starts and stops the stream with a command. In output mode, the audio samples are sent to the endpoint in the command. In input mode, the endpoint collects the received samples in a buffer and delivers them to the client in notifications, see *fldNotif.data*.

5.6.3 Example

To receive stereo audio at 48 kHz with 24 bits per sample, MSb first:

```
seq: 1
dir {
   config {
      hndl: 5
      pcm {
         dir: dirIn  fsMode: fsModeI2s  sampleRateHz: 48000
bitsPerSample: 24  msbFirst: true
         rxBufferSizeBytes: 1024
      }
   }
}
dir { action { hndl: 5  pcm { op: opStart } } }
```

5.6.4 Protocol

Messages

fldChannelMask *Message* `esb.pcm.fldChannelMask`

Container for specifying which portion of the data stream to ignore.

Fields

- **idx** — `uint32`
  Index of the channel, starting from 0.
- **padding** — *oneof*
  - **leftBits** — `uint32`
    Number of ignored padding bits before the first data bit in each channel.
  - **rightBits** — `uint32`
    Number of ignored padding bits after the last data bit in each channel.

fldConfig *Message* `esb.pcm.fldConfig`

PCM configuration message.

Fields

- **dir** — *optional* *enDir*
  PCM direction.
- **fsMode** — *optional* *enFsMode*
  PCM FS mode.
- **fsInv** — *optional* `bool`
  FS output inverted if true.
- **clkInv** — *optional* `bool`
  FS clk inverted if true.
- **fsAlign** — *optional* `bool`
  If true, the leading edge of fs is aligned with the first data bit. If false, the trailing edge of fs is aligned with the first data bit.
- **msbFirst** — *optional* `bool`
  If set, configures whether the data is sent/received MSB or LSB first.
- **rxBufferSizeBytes** — *optional* `uint32`
  Length of the buffer in bytes. Setting this to 0 configures the default buffer length. Exceeding the buffer size supported by the server causes the configuration to fail.
- **sampleRateHz** — *optional* `uint32`
  Sample rate in hertz (Hz).
- **numChannels** — *optional* `uint32`
  Number of channels. In I2S mode, this configuration is ignored, as the number of channels is fixed. In TDM mode, this can be set to any number up to the maximum supported by the server.
- **bitsPerSample** — *optional* `uint32`
  Number of bits per audio sample.
- **channelMask** — *repeated* *fldChannelMask*
  Configuration for ignoring data in specific channels, to save bandwidth.

fldCmd *Message* `esb.pcm.fldCmd`

Container for PCM command.

Fields

- **op** — *optional* *enCmdOperation*
  Operation to perform.
- **txData** — `bytes`
  Audio stream to transmit when in output mode.

Enumerations

enDir *Enumeration* `esb.pcm.enDir`

PCM direction.

- **`dirIn`** — PCM configured as input (RX).
- **`dirOut`** — PCM configured as output (TX).

enFsMode *Enumeration* `esb.pcm.enFsMode`

PCM frame sync mode.

- **`fsModeI2s`** — I2S mode.
- **`fsModeTdm`** — TDM mode.

enCmdOperation *Enumeration* `esb.pcm.enCmdOperation`

PCM operation types.

- **`opStart`** — Start PCM.
- **`opStop`** — Stop PCM.

5.7 PDM Driver

File: `endpoints/pdm/pdm_dev.proto`

Package: `esb.pdm`

PDM configuration, actions, queries, and notifications for RCP.

A PDM endpoint receives a pulse-density modulation (PDM) stream, for example from a digital microphone. The endpoint drives the clock CLK and samples the data line DATA, see the pin functions *`rcp.pins.enPinFuncPdm`*.

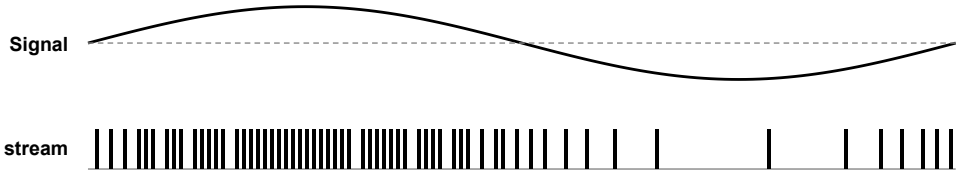
5.7.1 Principle

A PDM stream is a one-bit stream with a high bit rate. The amplitude of the signal is encoded in the density of the 1 bits: many 1 bits stand for a large positive value, many 0 bits for a large negative value. Converting the stream to audio samples with a lower rate requires filtering and decimation. This conversion is not part of the protocol. The endpoint streams the sampled bits to the client.

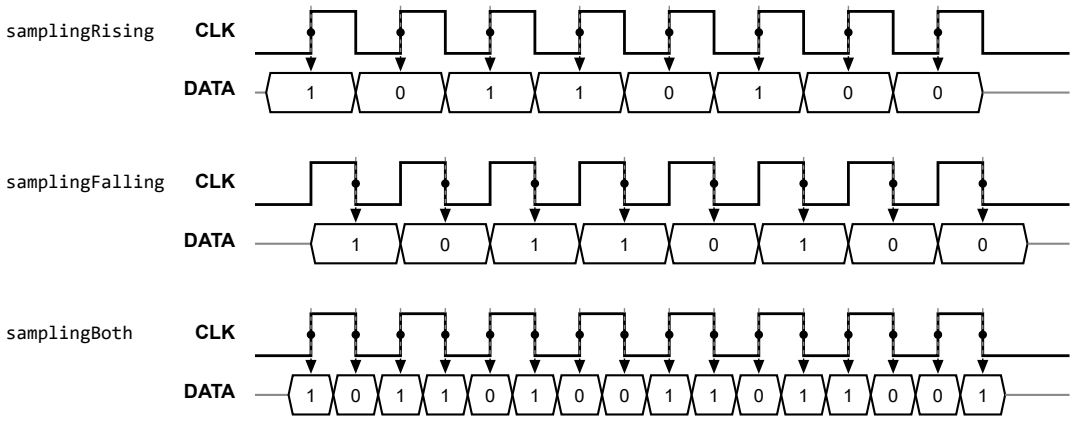
5.7.2 Clock and sampling edge

The endpoint takes over DATA on the rising edge, on the falling edge, or on both edges of the clock. With both edges, two bits are received per clock period, so the bit rate is twice the clock speed.

Principle: the density of the 1 bits follows the amplitude of the signal



Sampling edge (`enSamplingEdge`): the endpoint drives CLK and takes over DATA at the marked edges



5.7.3 Streaming

The client starts and stops the stream with a command. The endpoint collects the sampled bits in a buffer and delivers them to the client in notifications, see *`fldNotif.data`*.

5.7.4 Example

To record a PDM microphone with a 3.072 MHz clock, sampled on the rising edge:

```
seq: 1
dir { config { hndl: 6  pdm { speedHz: 3072000  samplingEdge:
samplingRising  bufferSizeBytes: 1024 } } }
dir { action { hndl: 6  pdm { op: opStart } } }
```

5.7.5 Protocol

Messages

fldConfig *Message* `esb.pdm.fldConfig`

PDM configuration message.

Fields

- **`bufferSizeBytes`** — *optional* `uint32`
  Length of the buffer in bytes. Setting this to 0 configures the default buffer length. Exceeding the buffer size supported by the server causes the configuration to fail.
- **`samplingEdge`** — *optional* *`enSamplingEdge`*
  PDM sampling edge configuration.
- **`speedHz`** — *optional* `uint32`
  The clock speed in hertz (Hz). If the requested speed is fully supported by the RCP server, the configuration is accepted with the *`cfgSuccess`* return code. The RCP server is allowed to round the requested speed to the nearest supported speed within a

reasonable tolerance (refer to the datasheet of the server). In this case, the configuration is accepted with the *cfgParamAdjusted* warning code and the client can retrieve the actual (rounded) speed by querying the configuration of the endpoint.

fldCmd *Message* `esb.pdm.fldCmd`

Container for PDM command.

Fields

- **op** — *optional* *enCmdOperation*
  PDM operation.

Enumerations

enSamplingEdge *Enumeration* `esb.pdm.enSamplingEdge`

PDM sampling edge configuration options.

- **samplingRising** — PDM configured to sample DATA on the CLK rising edge.
- **samplingFalling** — PDM configured to sample DATA on the CLK falling edge.
- **samplingBoth** — PDM configured to sample DATA on both CLK edges, rising and falling.

enCmdOperation *Enumeration* `esb.pdm.enCmdOperation`

PDM operation types.

- **opStart** — Start PDM.
- **opStop** — Stop PDM.

5.8 ADC Driver

File: `endpoints/adc/adc_dev.proto`

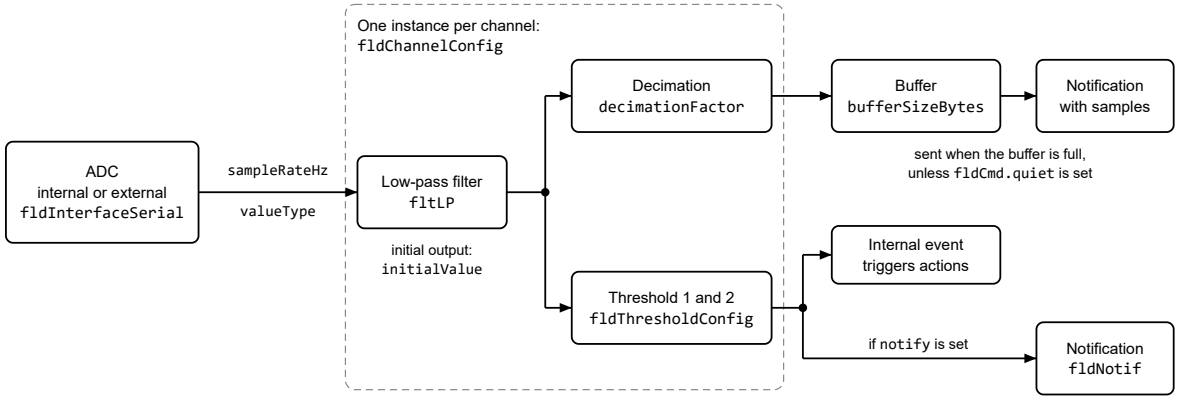
Package: `esb.adc`

ADC configuration, actions, queries, and notifications for RCP.

An ADC endpoint samples analog values at a fixed rate, post-processes them in the RCP server, and delivers the result to the client. The ADC is either part of the RCP server or an external device connected through a serial interface, see *fldInterfaceSerial*. Before using the ADC, the client configures it with a *fldConfig* message.

5.8.1 Processing chain

The samples are fed into the processing at the configured sample rate. If the ADC has several channels, each of them has its own processing chain: a low-pass filter smooths the samples, a decimator reduces the number of reported samples, and two thresholds evaluate the filtered values.



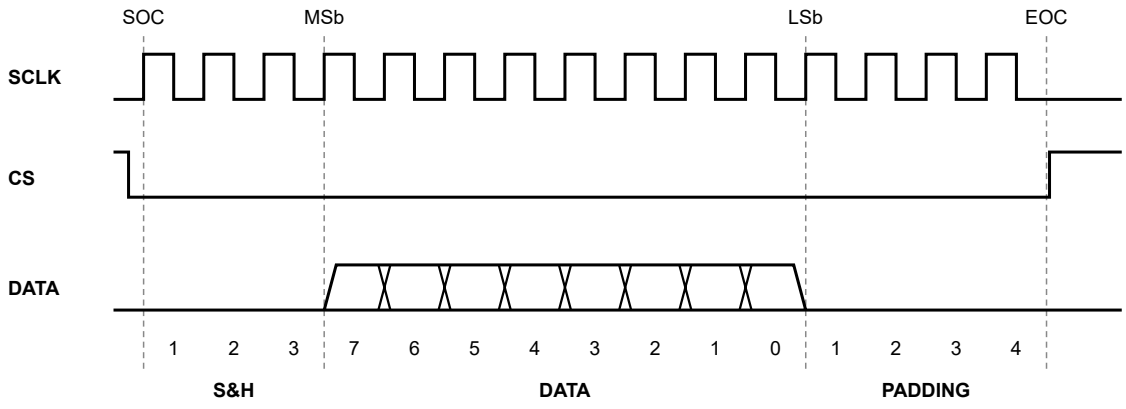
5.8.2 Streaming and thresholds

The client starts and stops the ADC with a command. The samples are collected in a buffer, and the RCP server sends them to the client in a notification when the buffer is full (streaming), see *esb.fldNotif.data*. The format of the samples in the buffer is server-dependent. Refer to the datasheet.

A threshold fires when the filtered value crosses a configured level. A fired threshold generates an internal event that can trigger actions with low latency, see *fldActionEvent*, and it can also send a notification, see *fldNotif*. Streaming and thresholds are independent of each other and can be used at the same time.

5.8.3 External ADC

An external ADC is read through a serial interface with three pins, similar to SPI: the RCP server generates the clock SCLK and the chip-select CS, and it samples the data output of the ADC. The RCP server starts a conversion by asserting CS. After a number of padding clock cycles, which give the ADC time to sample and hold, the data bits follow. The RCP server then waits for more padding clock cycles, which some ADCs need to reset their internal state, before it de-asserts CS. The figure shows an acquisition of 8 data bits, most significant bit first, with 3 padding clock cycles before them and 4 after them, at clock polarity 0 and clock phase 0:



SOC = start of conversion, EOC = end of conversion

5.8.4 Example

To sample channel 0 at 1 kHz, filter it, report every fourth sample, and be notified when it exceeds 800:

```
seq: 1
dir {
   config {
      hndl: 4
      adc {
         sampleRateHz: 1000
         bufferSizeBytes: 256
         channels {
            channelId: 0
            fltLP: 3
            decimationFactor: 2
            threshold1 { value: 800  event: thresholdEvtAbove
notify: true }
         }
      }
   }
}
dir { action { hndl: 4  adc { op: opStart } } }
```

5.8.5 Protocol

Messages

fldInterfaceSerial *Message* `esb.adc.fldInterfaceSerial`

The ESB protocol allows ADC endpoints to be internal or external to the RCP server. This interface allows configuration of an external ADC endpoint. The external ADC serial interface assumes the ADC has only three pins, similar to an SPI interface. In fact, the pin configuration message is the same as that of an SPI endpoint (see *rcp.pins.enPinFuncSpi*).

- *rcp.pins.pinSpiSclk*: the clock from the RCP server to the external ADC
- *rcp.pins.pinSpiMiso*: the ADC serial data output pin (input to RCP server)
- *rcp.pins.pinSpiCs*: the chip-select, marking the start of the sampling period

The parameters *dataBits*, *leftPaddingBits*, *rightPaddingBits*, *cpol*, *cpha* and *msbFirst* define the length of a single acquisition. For an example, see the page *ADC Driver*.

The RCP server starts the conversion by asserting the chip select (CS). The *leftPaddingBits* defines how many clock cycles to wait before the first valid data bit appears at the ADC data pin. This allows time for the ADC to perform sample & hold, depending on the ADC type and performance. The *dataBits* parameter tells the RCP server how many data bits to sample. This should be set according to the ADC resolution. Finally, the *rightPaddingBits* parameter sets how many clock cycles to wait before de-asserting the chip-select. Some ADCs require these extra clock cycles to reset the internal state before a new conversion can happen. The total number of clock cycles for a single acquisition is therefore the sum of *leftPaddingBits*, *dataBits*, and *rightPaddingBits*. The *cpol*, *cpha* and *msbFirst* parameters are the same as those used for a normal SPI interface. See *spi.fldConfig* for more information.

Fields

- **`speedHz`** — *optional* `uint32`
  The clock speed in hertz (Hz). If the requested speed is fully supported by the RCP server, the configuration is accepted with the *cfgSuccess* return code. The RCP server is allowed to round the requested speed to the nearest supported speed within a reasonable tolerance (refer to the datasheet of the server). In this case, the configuration is accepted with the *cfgParamAdjusted* warning code and the client can retrieve the actual (rounded) speed by querying the configuration of the endpoint.
- **`dataBits`** — *optional* `uint32`
  Number of bits in each ADC sample.

- **`leftPaddingBits`** — *optional* `uint32`
  Number of clock cycles before the first data bit.
- **`rightPaddingBits`** — *optional* `uint32`
  Number of clock cycles after the last data bit.
- **`cpol`** — *optional* `bool`
  Clock polarity (same as SPI CPOL).
- **`cpha`** — *optional* `bool`
  Clock phase (same as SPI CPHA).
- **`msbFirst`** — *optional* `bool`
  If true, ADC data is captured MSb first.
- **`csDelay`** — *optional* `uint32`
  CS to SCLK delay in half clock periods. This parameter defines the additional delay between the chip select assertion and the first clock edge, as well as the delay between the last clock edge and the de-assertion of the chip-select. The total delay is calculated as (*`csDelay`* + 1) / (2 * *`speedHz`*) seconds.

fldThresholdConfig *Message* `esb.adc.fldThresholdConfig`

Each ADC channel has configurable thresholds to generate internal events and notifications. The thresholds evaluate the value at the ADC filter output, that is, after all configured on-chip post-processing except decimation. When a threshold is enabled and triggered, it generates an internal event that can be used to fire pre-configured (re)actions for use cases requiring low latency. In addition to the internal event, a threshold can be configured to send a notification message when triggered.

Fields

- **`value`** — *optional* `uint32`
  Unsigned threshold value, see *`enValueType`*.
- **`event`** — *optional* *`enThresholdEvent`*
  Configures which threshold crossings generate an event.
- **`notify`** — *optional* `bool`
  If set, sends a notification to the client when an event occurs.

fldChannelConfig *Message* `esb.adc.fldChannelConfig`

Configuration for each ADC channel post-processor.

Fields

- **`fltLP`** — *optional* `uint32`
  Filter factor for the ADC channel. This value determines the strength of the low-pass filter applied to the ADC samples. The filter transfer function is: OUT[i] = (IN[i] + *`fltLP`* * OUT[i - 1]) / (*`fltLP`* + 1) Setting *`fltLP`* to 0 (default) disables low-pass filtering (OUT[i] = IN[i]).
- **`decimationFactor`** — *optional* `uint32`
  Decimation factor for the ADC channel. This value determines the number of samples reported to the client. The decimator reports 1 sample every 2^N samples, where N is *`decimationFactor`*. Setting *`decimationFactor`* = 0 (default) disables decimation (all samples are reported to the client). Decimation affects streamed samples only. Thresholds evaluate the filtered samples before decimation, see *`fldThresholdConfig`*.
- **`initialValue`** — *optional* `uint32`
  Initial value for the ADC channel filter output. This value is the initial output value of the ADC post-processing block when the ADC starts. This can be used to avoid glitches on the configured thresholds.
- **`valueType`** — *optional* *`enValueType`*
  Configures the value type of the ADC samples for this channel.
- **`threshold1`** — *optional* *`fldThresholdConfig`*
  Threshold configuration for the first threshold event.

- **threshold2** — *optional* *fldThresholdConfig*
  Threshold configuration for the second threshold event.
- **channelId** — *optional* uint32
  If the ADC device supports multiple channels, this parameter represents the channel number identifier, that is, the channel multiplexer (MUX) selector number. This identifier can be used to instruct the ADC to sample channels in a specific order / sequence.

fldConfig *Message* esb.adc.fldConfig

ADC post-processing configuration.

Fields

- **sampleRateHz** — *optional* uint32
  The ADC sample rate in hertz (Hz). This parameter configures the speed at which ADC samples are fed into the ADC post-processing block. If an external ADC interface is used, the time interval between two CS pin assertions is 1 / *sampleRateHz*. If the requested frequency is fully supported by the RCP server, the configuration is accepted with the *cfgSuccess* return code. The RCP server is allowed to round the requested frequency to the nearest supported speed within a reasonable tolerance (refer to the datasheet of the server). In this case, the configuration is accepted with the *cfgParamAdjusted* warning code and the client can retrieve the actual (rounded) speed by querying the configuration of the endpoint.

  > **Note**
  >
  > On some implementations, the sample rate can be synchronous to the internal PTP clock.

  > **Warning**
  >
  > Do not confuse the sample rate with the external ADC clock speed, which is configured with the *fldInterfaceSerial* message.

- **bufferSizeBytes** — *optional* uint32
  This parameter sets the size in bytes of the internal ADC buffer. When the amount of sampled data exceeds *bufferSizeBytes*, the RCP server sends out a notification message including all the captured samples.
- **channels** — *repeated* *fldChannelConfig*
  Configuration for each channel.
- **interface** — *oneof*
  Defines the ADC physical interface. If none of the options is selected, the configuration targets the first available internal ADC.
  - **serial** — *fldInterfaceSerial*
    Configuration for the SPI ADC interface.

fldCmd *Message* esb.adc.fldCmd

Container for ADC commands.

Fields

- **op** — *optional* *enCmdOperation*
  ADC operation.
- **quiet** — bool
  If set, suppresses data streaming to the client.

fldNotif *Message* `esb.adc.fldNotif`

Container for ADC event notifications.

Fields

- **`channelId`** — `uint32`
  ID of the channel that generated the event.
- **`event`** — *`enNotifEvent`*
  Type of the event.
- **`value`** — `uint32`
  Value that triggered the event.

Enumerations

enThresholdEvent *Enumeration* `esb.adc.enThresholdEvent`

Threshold crossing that generates an event.

- **`thresholdEvtNone`** — No trigger (disables the threshold).
- **`thresholdEvtAbove`** — Trigger if the filtered output rises above *`fldThresholdConfig.value`*.
- **`thresholdEvtBelow`** — Trigger if the output falls below *`fldThresholdConfig.value`*.
- **`thresholdEvtCrossing`** — Trigger if the output crosses *`fldThresholdConfig.value`* in either direction.

enValueType *Enumeration* `esb.adc.enValueType`

This enumeration defines the ADC data format as produced by the internal or external ADC endpoint. This setting affects the ADC post-processing and filtering arithmetic.

> **Note**
>
> The threshold value (*`fldThresholdConfig.value`*), the initial value (*`fldChannelConfig.initialValue`*) and the value in a notification (*`fldNotif.value`*) are unsigned integers, regardless of the selected format.

- **`valueUnsigned`** — Standard n-bit unsigned sample data (10010101 = 149 dec.)
- **`valueComplement`** — Signed two's complement number (10010101 = -107 dec.)
- **`valueSignBit`** — Sign bit plus integer part, i.e., the MSb is used as sign bit (10010101 = -21 dec.)

enCmdOperation *Enumeration* `esb.adc.enCmdOperation`

ADC operation types.

- **`opStart`** — Start the ADC in free-running mode.
- **`opStop`** — Stop the ADC.
- **`opSingle`** — Capture a single buffer, then stop.

enNotifEvent *Enumeration* `esb.adc.enNotifEvent`

Type of ADC event.

- **`eventThreshold1`** — An event occurred on threshold 1.
- **`eventThreshold2`** — An event occurred on threshold 2.

5.9 SENT Driver

File: `endpoints/sent/sent_dev.proto`

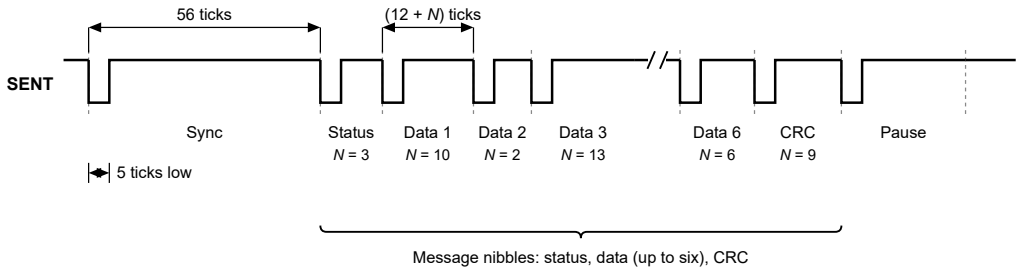
Package: `esb.sent`

SENT configuration, actions, queries, and notifications for RCP.

A SENT endpoint receives messages of the single edge nibble transmission (SENT) protocol (SAE J2716). A sensor sends them periodically over a single wire, and the endpoint only listens. The client configures the endpoint with a *`fldConfig`* message, starts it with a command, and receives the messages in notifications.

5.9.1 Message format

A SENT message is a sequence of pulses. Each pulse starts with a falling edge, stays low for 5 ticks, and is measured from its falling edge to the next one. The tick time t_{tick} is defined by the sensor. There is no tick time parameter. The synchronization pulse, 56 ticks long, lets the receiver measure the tick time. Each nibble pulse carries a four-bit value N in its length of 12 + N ticks: the first nibble is the status, then up to six data nibbles follow, and the last one is the cyclic redundancy check (CRC). An optional pause pulse can be used to give all messages the same length.



1 tick = t_{tick} (set by the sensor). The pause pulse is optional (`expectPause`). The field `polInv` inverts the polarity.

5.9.2 Streaming

The client starts and stops the reception with a command. The endpoint collects the received messages in a buffer and flushes it to the client in notifications when it is full, see *`fldNotif.data`*. The format of the data in the buffer is server-dependent. Refer to the datasheet.

5.9.3 Example

To receive messages with one status nibble, six data nibbles, and one CRC nibble from a sensor that sends a pause pulse:

```
seq: 1
dir { config { hndl: 9  sent { frameLength: 8  expectPause: true
bufferSizeBytes: 256 } } }
dir { action { hndl: 9  sent { op: opStart } } }
```

5.9.4 Protocol

Messages

fldConfig *Message* `esb.sent.fldConfig`

SENT configuration message.

Fields

- **`bufferSizeBytes`** — *optional* `uint32`
  Size of the SENT data buffer in bytes. The endpoint flushes the data to the client when the buffer is full.
- **`polInv`** — *optional* `bool`
  If true, inverts the polarity of the SENT signal.
- **`expectPause`** — *optional* `bool`
  If true, expects a pause field after the nibble transmission.
- **`frameLength`** — *optional* `uint32`
  Expected length of the SENT frame in nibbles.
- **`timeoutEnable`** — *optional* `bool`
  If true, enables the reception timeout: when no edge occurs on the SENT signal for the time set by *`timeout`*, the endpoint flushes the buffer to the client before it is full, in a notification with the code *`ntfIOError`*.
- **`timeout`** — *optional* `uint32`
  The time without an edge on the SENT signal after which the reception times out, see *`timeoutEnable`*. It is given as a number of ticks. The tick time is the one that the receiver measures from the synchronization pulse. The timer restarts with every edge, so the reception times out if the timeout is shorter than the longest pulse, which is the pause pulse if the sensor sends one. The number of ticks per step and the valid range are server-dependent. Refer to the datasheet.

fldCmd *Message* `esb.sent.fldCmd`

Container for SENT commands.

Fields

- **`op`** — *optional* *`enCmdOperation`*
  SENT operation.

Enumerations

enCmdOperation *Enumeration* `esb.sent.enCmdOperation`

SENT operation types.

- **`opStart`** — Start SENT.
- **`opStop`** — Stop SENT.

5.10 LIN Driver

File: `endpoints/lin/lin_dev.proto`

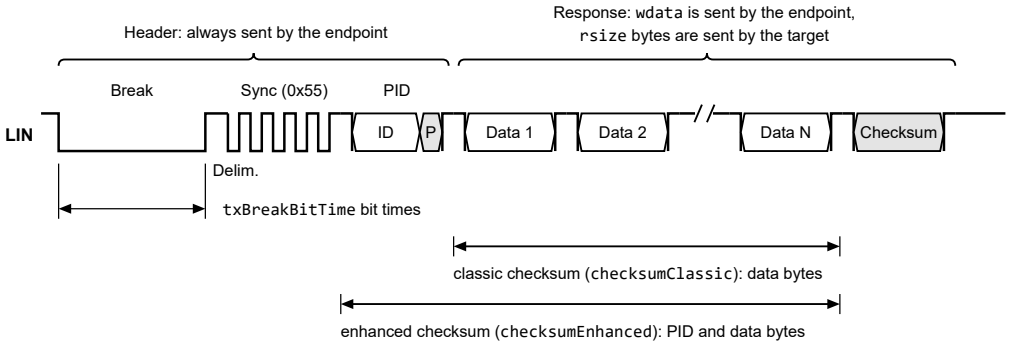
Package: `esb.lin`

LIN configuration, actions, queries, and notifications for RCP.

A LIN endpoint is the commander of a local interconnect network (LIN) bus, also called the master. A LIN frame consists of a header and a response. The endpoint always sends the header. The response comes either from the endpoint itself or from a target (responder). The endpoint uses the pins TX and RX of the UART pin functions, which the client connects to a LIN transceiver, see *`rcp.pins.enPinFuncUart`*.

5.10.1 Frame

Every byte on the bus has a start bit, eight data bits (LSb first), and a stop bit. A frame starts with the break, a long low level followed by a delimiter. The synchronization byte 0×55 lets the target measure the bit time. The protected identifier (PID) holds the six-bit frame identifier and two parity bits. The response holds the data bytes and a checksum.



Each byte: start bit (low), eight data bits (LSb first), stop bit (high). Bit time t_B = 1 / `speedBps`. P: parity bits.

5.10.2 Commands

A command sends one frame. It either publishes the response, with the bytes that the endpoint sends, or requests it, with the number of bytes that the target sends. The bytes read are returned in the reply to the command. The endpoint can calculate the parity bits of the identifier and the checksum of the response itself, or leave both to the client.

5.10.3 Example

To read a four-byte response from the frame with the identifier 0×12 at 19200 bits per second (bps):

```
seq: 1
dir { config { hndl: 8  lin { speedBps: 19200  identifierParity:
true  checksumType: checksumEnhanced } } }
dir { action { hndl: 8  lin { frameId: 0x12  rsize: 4 } } }
```

5.10.4 Protocol

Messages

fldConfig *Message* `esb.lin.fldConfig`

LIN configuration message.

Fields

- **`speedBps`** — *optional* `uint32`
  The bit rate in bits per second (bps). If the requested speed is fully supported by the RCP server, the configuration is accepted with the *`cfgSuccess`* return code. The RCP server is allowed to round the requested speed to the nearest supported speed within a reasonable tolerance (refer to the datasheet of the server). In this case, the configuration is accepted with the *`cfgParamAdjusted`* warning code and the client can retrieve the actual (rounded) speed by querying the configuration of the endpoint.
- **`identifierParity`** — *optional* `bool`
  If set to true, the endpoint automatically calculates and appends the parity bits to the identifier. Otherwise, the *`fldCmd.frameId`* field includes the parity bits.

- **checksumType** — *optional* *enChecksumType*
  LIN checksum type.
- **rxSuppressLoopback** — *optional* `bool`
  If used with a LIN transceiver that loops back the transmitted data to the receiver, setting this to true suppresses the loopback.
- **txBreakBitTime** — *optional* `uint32`
  The break condition duration in bit times.

fldCmd *Message* `esb.lin.fldCmd`

Container for LIN command.

Fields

- **mode** — *oneof*
  - **wdata** — `bytes`
    Bytes to send. If checksum is enabled, checksum byte(s) are appended automatically.
  - **rsize** — `uint32`
    Number of bytes to read. If checksum is enabled, checksum byte(s) are read and verified automatically.
- **frameId** — `uint32`
  Identifier of the LIN frame. If *fldConfig.identifierParity* is false, it includes the parity bits.
- **checksumType** — *optional* *enChecksumType*
  If set, overrides the checksum configuration for this command.

Enumerations

enChecksumType *Enumeration* `esb.lin.enChecksumType`

Checksum type for LIN messages.

- **checksumNone** — Checksum disabled. The client has to handle it.
- **checksumClassic** — Classic checksum (without identifier).
- **checksumEnhanced** — Enhanced checksum (with identifier).